

分类号： TP242.6

学校代码： 10112

密 级： 公开

太 原 理 工 大 学

硕 士 学 位 论 文

(专 业 学 位)

轮腿形态可转换机器人结构及其运动仿真研究

姓 名： 刘欣

学 号： 2021520118

培养单位： 机械与运载工程学院

专业领域： 机械工程

研究方向： 机器人技术

导师姓名： 程建明 高级工程师

企业导师： 刘 晓 教授级高工

论文提交日期： 2024 年 6 月

A Dissertation Submitted to
Taiyuan University of Technology
In partial fulfillment of the requirement
For the degree of Master

**Research on Structure and Motion Simulation of a
Transformable Wheel-legged Robot**

By

Xin Liu

College of Mechanical and Vehicle Engineering
June 2024

摘要

人类文明的进步就是不断适应自然改变自然的过程，从原始的石器到现在的高科技工具都对人类社会的发展做出了卓越的贡献。机器人是人们改造自然的一种高级形态工具，行走机器人通常采用轮式、履带式或足式结构，不同结构各有优势。轮式机器人适合在平坦地形上高速稳定行驶；履带式机器人适合较复杂的松软不太平整的地形行驶；而足式机器人越障能力强，适合在复杂地形运动。单一结构机器人难以兼顾稳定、高速、适合不同地形行驶要求。

针对上述问题本文研究了一种新型轮腿可变形机器人，本机器人在平坦地形时用四轮平稳高速行驶，在复杂地形时变化为腿足模式运动。本轮腿变形机构从拱桥结构得到启发：将轮子拆分成外缘铰接的若干节，轮子直径方向设置电动推杆，推杆收缩长度到直径时为轮形结构，此时轮子就如同拱桥的结构，具有较好的刚性，可承受较大负载；电动推杆伸直后轮子拉长变形为腿形结构。本轮腿机构运动由同一组关节模组驱动，充分利用了机器人驱动系统和动力资源。

论文在 SolidWorks 下研究开发了本轮腿机器人的三维模型，利用 SolidWorks 的 Motion 模块对机器人轮腿变形机构进行运动仿真；利用 Ansys 对关键零件进行有限元力学分析，验证了结构的合理性；基于齐次坐标变换矩阵，完成对机器人腿部的运动学建模；利用 MATLAB 计算了腿部的工作空间；基于 VMC 算法完成对机器人整体的运动学建模；使用 Python 编写运动控制算法；利用机器人运动仿真软件 Webots 对机器人进行运动仿真实验；根据仿真结果优化了控制算法。最后使用 3D 打印制作轮腿变形机构样机，并进行了一体化关节模组的控制实验，验证了本机器人结构的可行性。

关键词：轮腿机器人；可变形；结构设计；多运动模式；运动控制；关节模组

ABSTRACT

The progress of human civilization lies in the continuous adaptation to and transformation of nature. From primitive stone tools to today's advanced technology, each has made remarkable contributions to the development of human society. Robots represent an advanced form of tools through which humans have modified nature. Conveyor robots typically adopt wheel, track, or legged structures, each with its own advantages. Wheeled robots are suitable for high-speed stable travel on flat terrain, tracked robots are suitable for traveling on complex, soft, and uneven terrain, while legged robots exhibit strong obstacle-crossing capabilities and are suitable for movement in complex terrain. A single-structured robot finds it difficult to balance stability, speed, and suitability for different terrain requirements.

This study addresses the design of a novel transformable wheel-leg robot. The robot achieves smooth and high-speed movement on flat terrain using four wheels, and transitions to a legged mode for navigation in complex terrains. Inspired by the arch bridge structure, the wheel-leg transformation mechanism divides the wheels into several sections hinged along the outer edge. Electric push rods are arranged in the diameter direction of the wheels; when the push rods retract to match the diameter, the wheels form a wheel-like structure resembling an arch bridge, providing good rigidity to withstand heavy loads. When the electric push rods extend, the wheels elongate and transform into a leg-like structure. The motion of this wheel-leg mechanism is driven by the same set of joint motors, effectively utilizing the drive system and power resources.

The three-dimensional model of the wheel-leg robot was designed in SolidWorks, and the Motion module of SolidWorks was used to simulate the motion of the robot's wheel-leg transformation mechanism. Ansys was utilized to conduct finite element analysis on key components to validate the structural integrity. Kinematic modeling of the robot's legs was completed based on homogeneous coordinate transformation matrices, while the workspace of the legs was computed using MATLAB. The overall kinematic modeling of the robot was achieved through the use of the VMC algorithm, and motion control algorithms were developed using Python. Furthermore, motion simulation experiments for the robot were conducted using the robot motion simulation software Webots. Control algorithms were optimized based on the simulation results. Finally, a prototype of the wheel-leg transformation mechanism was 3D printed, and integrated joint motor control experiments were conducted to verify the feasibility of the robot's structure.

Keywords: wheel-legged robots; transformable; mechanism design; motion control; joint module

目 录

第 1 章 绪论.....	1
1.1 课题研究背景与意义.....	1
1.2 四足机器人研究现状.....	2
1.2.1 四足机器人国外研究现状.....	2
1.2.2 四足机器人国内研究现状.....	5
1.3 轮腿复合机器人研究现状	8
1.3.1 轮腿复合式机器人国外研究现状.....	8
1.3.2 轮腿复合式机器人国内研究现状.....	12
1.4 国内外研究现状总结.....	17
1.5 本文主要研究内容.....	18
第 2 章 轮腿机器人总体结构设计	21
2.1 引言.....	21
2.2 轮腿变形机构设计.....	21
2.2.1 拱桥结构分析.....	21
2.2.2 变形原理.....	22
2.2.3 结构设计.....	23
2.2.4 有限元仿真分析.....	26
2.3 机器人总体结构设计.....	29
2.3.1 结构方案与驱动方式选择.....	29
2.3.2 机器人总体结构及参数.....	29
2.3.3 一体化关节模组设计.....	30
2.3.4 导电滑环设计.....	33
2.3.5 腿部连接件设计.....	34
2.4 本章小结.....	35
第 3 章 机器人运动学分析	37
3.1 引言.....	37
3.2 单腿运动学及工作空间分析	37
3.2.1 正运动学分析.....	37
3.2.2 工作空间分析.....	43
3.2.3 逆运动学分析.....	44
3.3 单腿雅可比矩阵分析.....	46
3.4 本章小结.....	48
第 4 章 运动控制系统及控制方法	49
4.1 引言.....	49

4.2 控制系统.....	49
4.2.1 上位机系统.....	49
4.2.2 关节电机.....	50
4.2.3 电机控制方法.....	51
4.3 机器人整机运动控制方案	54
4.3.1 四足模式控制方案.....	54
4.3.2 机器人步态规划.....	59
4.3.3 四轮模式控制方案.....	61
4.3.4 机器人模式切换运动规划.....	63
4.4 本章小结.....	64
第5章 运动控制仿真实验	65
5.1 引言.....	65
5.2 仿真环境搭建.....	65
5.2.1 软件介绍.....	65
5.2.2 环境搭建.....	65
5.3 四足模式仿真.....	66
5.4 四轮模式仿真.....	72
5.5 本章小结.....	73
第6章 样机实验	75
6.1 引言.....	75
6.2 变形机构实验测试.....	75
6.3 关节模组控制实验.....	77
6.4 本章小结.....	80
第7章 结论与展望	81
7.1 全文总结.....	81
7.2 展望.....	81
参考文献.....	83
附录 A 运动控制算法程序实现.....	87

第1章 绪论

1.1 课题研究背景与意义

随着社会的不断发展和科技的飞速进步,机器人技术作为一项具有广泛应用前景的前沿科技已经逐渐融入我们的生活和工作中,成为各行各业的重要工具,从生产制造到医疗保健,再到紧急救援和探险科学,机器人已经展现出了无限的应用潜力。其中,地面移动机器人一直是研究的热点之一,在众多领域中都有着广泛的应用需求。例如,紧急救援机器人可以在自然灾害、事故或紧急情况下执行危险任务,减少人员伤亡;探险科学中,机器人可以被用于探测险恶的环境,如火山、沉船或无人地区;在军事领域,机器人可以执行侦察、排除地雷和运送任务;此外,工业制造中的自动化、仓储物流和服务业等领域也对移动机器人提出了需求。智能制造是“中国制造 2025”的主攻方向^[1],机器人是重要组成部分,也是我国研究的一个重要方面。机器人技术是综合了机械电子、自动化控制、传感器技术、计算机技术、新型材料、仿生技术和人工智能等多领域多学科的复杂高科技技术,被认为是对未来新兴产业发展具有重要意义的高新技术之一^[2,3]。随着人工智能、传感技术和材料科学的不断进步,机器人技术已经取得了显著的突破,现代机器人不再局限于单一的工作模式和环境,而是可以在多种工况和不同环境下执行任务^[4]。这种多模式多工况的机器人已经成为研究的热点,各种复合式机器人如轮腿机器人、轮履机器人等应运而生。

地面移动机器人根据其移动方式可以分为四类:轮式、腿式、履带式和复合式^[5]。轮式机器人具有自重轻、承载大、机构简单、驱动和控制相对方便、行走速度快、机动灵活、工作效率高等优点^[6],但对复杂未知非结构化环境的适应能力差^[7]。履带机器人控制系统相对简单并且环境适应能力强,但其结构笨重,速度较慢,能耗较大,多用于一些特殊场景,如军事装备方面^[8,9]。腿足式移动机器人因其腿部具有较大的活动范围,展现出高机动越野能力,使其能够用于轮式、履式平台难以适应的复杂地形环境^[10]。然而,这种类型的机器人在动力和结构上的复杂性,以及多自由度的特点,意味着需要驱动众多部件,这不仅增加了电控系统设计的难度,也导致了较高的能源消耗,因此其使用通常被限制在特殊的应用环境中^[11,12]。复合式机器人综合了多种移动结构的特点,兼具不同移动方式的优势,近年来备受学界关注。轮腿机器人作为复合式机器人的一种,结合了轮式和腿式移动方式,同时具备轮式机器人的快速移动能力和腿足式机器人的越障能力,可以适应各种复杂地形和任务需求^[13]。对轮腿机器人的研究不仅有助于推动机器人技术的进步,还可以为各个领域提供更多的应用解决方案。

本课题提出一种新型多运动方式轮腿复合式移动机器人,独特的结构使其具有出色的地形适应能力,能够根据任务需求切换运动方式,从而提高工作效率。在不平坦和崎

岖的地形环境中,可以通过切换到腿式移动方式,越过各种障碍,如沟渠、斜坡和楼梯,这在紧急救援和探险任务中尤为重要,能够在危险地区执行任务,减少人员伤亡;在平坦路面上,轮腿机器人可以切换为轮式运动模式,可降低能耗,更快速稳定地移动,提高任务执行效率,这对于工业制造和物流领域尤为重要,可以提高生产效率和减少成本。轮腿机器人的多功能性使其在各种领域都有广泛的应用潜力,除了紧急救援、探险和军事应用外,轮腿机器人还可以用于环境探测、农业、医疗保健和娱乐等领域。轮腿机器人的研究成果可以促进机器人技术的创新和进步,为推动机器人技术的广泛应用打下坚实的基础。

1.2 四足机器人研究现状

四足机器人是一种具有四条腿的机器人,通常被设计成模仿动物的行走方式。它们可以具备不同的形状、尺寸和能力,但它们的共同特点是使用四条腿行走,提供优秀的稳定性和适应性,相比于一些其他类型的机器人,如轮式或履带式机器人,四足机器人在复杂、不规则或崎岖的地形上表现更出色。

1.2.1 四足机器人国外研究现状

国外在四足机器人方面相关研究起步较早。**WalkingTruck** 是美国 1968 年研制的世界上第一个有计算机参与的足式机器人^[14],是早期尝试在机器人领域模仿生物步态的先驱之一,如图 1-1 所示。这台 **WalkingTruck** 四足机器人由液压驱动,每一条腿都有三个自由度,可以模拟类似于动物行走的步态。尽管这台机器人在技术上取得了一定的成就,展示了四足行走的潜力,但由于操作复杂性和成本问题,它没有被广泛采用。

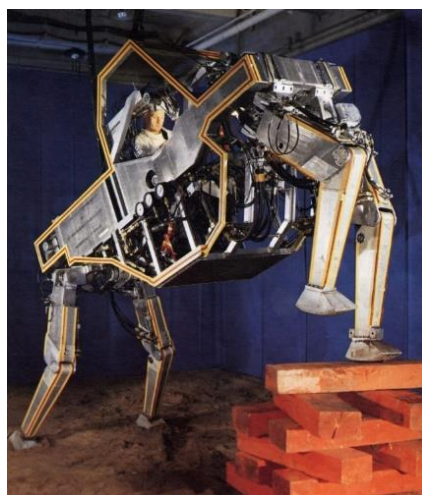


图 1-1 walkingTruck 四足机器人^[14]

Fig.1-1 walkingTruck^[14]

进入 21 世纪,随着科技的进步,以及控制理论和技术、传感器技术的发展,四足

机器人的研究也随之蓬勃发展。LittleDog^[15]是一款由麻省理工学院人工智能实验室（MITAILab）开发的四足机器人，是其系列机器人中的一员，如图 1-2 所示。LittleDog 被设计用于探究机器人在多变环境下的步行与移动技能。该机器人采取腿式结构，配备一个集中式控制单元，具有极高的运动灵活性及障碍跨越功能。借助精确协同工作的传感器与控制系统，Little Dog 能够适应多种地形，如爬楼梯、走斜坡，展示出卓越的平衡与稳定表现。它拥有 12 个主动关节，每个关节均由高增益的伺服电机提供动力，但早期由电机直接驱动的机器人存在承重能力低、动态反应不足和抗冲击力较弱等问题。

为改进上述一系列问题，美国波士顿动力公司于 2005 年成功研制出了四足机器人 Bigdog^[16]，如图 1-3 所示。Bigdog 的身高为 1 米，体重为 109 公斤，其动力来自一台输出约 15 马力的水冷式二冲程内燃机。这台发动机负责驱动液压泵，而动作执行则依靠通过两级航空航天级伺服阀控制的低摩擦液压缸完成，每条腿装配了 4 个液压动力单元以驱动关节，并设有第 5 个被动运动自由度。Bigdog 能够承载高达 154 公斤的重量，在快速移动过程中展现出了优异的载重性能。此外，它还具备强大的抗干扰能力，即便在负重情况下也能稳定通过雪地和冰面。



图 1-2 LittleDog 四足机器人^[15]

Fig.1-2 LittleDog^[15]



图 1-3 Bigdog 四足机器人^[16]

Fig.1-3 Bigdog^[16]

波士顿动力公司又设计了 LS3 来帮助士兵搬运重物^[17]，如图 1-4 所示。LS3 是一种崎岖地形机器人，被设计用于在各种地形下为海军陆战队和士兵提供支持，能够携带 400 磅的装备和足够的燃料，持续执行长达 24 小时、20 英里的任务。该机器人利用计算机视觉自动跟踪领导者，无需专门驾驶员。此外，它还可以通过地形感知和全球定位系统准确导航至指定目的地。尽管 BigDog 和 LS3 在性能方面表现出色，但由于噪音问题较为突出，最终没有得到广泛应用。LS3 机器人的推出对机器人领域的发展起到了一定的推动作用，同时也凸显了在实际应用中需要解决的一些技术难题，如噪音问题的处理和环境适应性等。

波士顿动力在经过十余年的技术迭代后，于 2015 年推出了 Spot 机器人^[18]，如图 1-5 所示。Spot 机器人站立高度为 0.94 米，体重 75 公斤，具备在携带 45 公斤负载下的奔

跑能力。Spot 改用锂电池供电以驱动其液压系统，从而实现了更为安静的操作。它还装备了多样的传感器和视觉系统，这使得 Spot 能够感知并适应其周边环境，尤其是在复杂地形中的表现尤为突出，不过，这种设计也导致了续航力的降低。



图 1-4 LS3 四足机器人^[17]

Fig.1-4 LS3^[17]

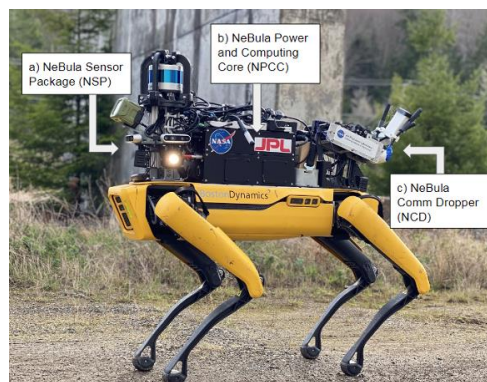


图 1-5 spot 四足机器人^[18]

Fig.1-5 spot^[18]

苏黎世理工大学(ETH)研发了一种具有卓越机动性和动态运动能力的四足机器人 ANYmal^[19]，如图 1-6 所示，每条腿有三个驱动关节，大腿和小腿的连接长度约为 250 毫米，总重量略低于 30 公斤，为了实现这种轻量化设计，主体和腿段由铝和碳纤维制成。ANYmal 的关节单元采用了物理柔性驱动器(SEA)技术，是一个高度集成的弹性致动器，它由高转矩电机、谐波驱动齿轮、柔性旋转弹簧和编码器构成，具有出色的负载能力和抗冲击性，非常适合在复杂和危险的工厂环境中操作。此外，ANYmal 还配备了深度相机、雷达和红外热像仪等传感器，使其能够在黑暗环境中进行定位、识别环境并规划路径。



图 1-6 ANYmal 四足机器人^[19]

Fig.1-6 ANYmal^[19]

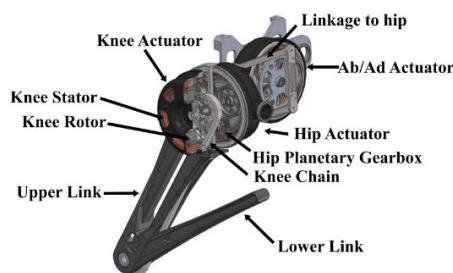
麻省理工学院 (MIT) 研发的猎豹系列机器人 Cheetah^[20], Cheetah2^[21], Cheetah3^[22], mini Cheetah^[23]，如图 1-7 所示为 Cheetah3^[22]，其重量约为 40kg，其设计沿用了先前猎豹 1 和猎豹 2 的动力系统设计理念，特别是选用了能够实现反向驱动的单级行星齿轮减

速器和设计以减小惯性的低惯量腿部，配备了高扭矩密度的电机。每个执行器都由一个定制的高扭矩密度电机组成，该电机与一个单级 7.67:1 行星齿轮减速器相连。为了最小化腿部的惯性，腿部采用串行驱动方式，并且每条腿的髋关节和膝关节的致动器被放置于髋部以实现同轴配置。Cheetah3 无需依赖于安装在关节或脚部的力感应器和扭矩感应器，就能通过本体感知来精确控制对地面的反作用力。



(a) 整机

(a) Monolithic structure



(b) 腿部结构

(b) Leg structure

图 1-7 猎豹 3 四足机器人^[22]Fig.1-7 Cheetah3^[22]

1.2.2 四足机器人国内研究现状

国内四足机器人的发展虽然相对于国外较晚，但也取得了一定的成果。以 1991 年上海交通大学的研究团队为例，研发了 JTUWM 系列^[24-26]的机器人，其中包括 JTUWM-II 型机器人，如图 1-8 所示，该机器人在足部安装了多种传感器，用于环境识别，并完成相应的运动步态规划，以适应各种路面情况下的行走。由于四足机器人的控制结构复杂、计算需求庞大、对实时性要求高，JTUWM-II 型机器人采用了分布式计算机控制结构，有效提高了系统的并行处理和实时控制能力。这些研发成果为四足机器人技术的进一步发展奠定了坚实的基础，为我国在机器人领域的研究和发展提供了有益的经验 and 启示。



图 1-8 JTUWM-II 型机器人

Fig.1-8 JTUWM-II

中国科学技术大学于 2007 年开发了一款专为月球探测任务设计的四足机器人，如图 1-9 所示。开发团队创新性地引入了中枢模式振荡器^[27](CPG, Central Pattern Generator)，该技术受到生物学原理的启发，用以确保机器人在各种环境下的稳定步态，这一系统通过融合反射机制，实现了机器人的自动避障功能。此外，团队还运用了强化学习技术，通过神经网络构建了一个强化学习框架，并开发出基于神经网络的 Q 学习算法，这一进步使得四足机器人能够在未知的环境中进行自主导航并优化其行为。这些技术的应用不仅为四足机器人在面对复杂环境时的操作提供了新的策略，也拓宽了其应用领域的范围。

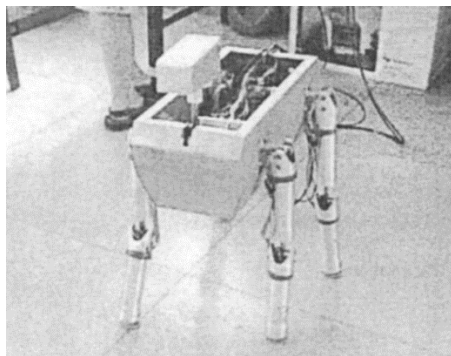


图 1-9 月球勘测四足机器人

Fig.1-9 Lunar Survey Quadruped robot

2011 年，作为国家 863 计划的一部分，我国多个高等教育机构着手开展了液压驱动四足机器人的研究工作。在这一背景下，山东大学成功研发了 Scalf 机器人^[28]，该机器人的设计采纳了液压驱动技术，如图 1-10 所示。Scalf 的体积为长 1.0 米、宽 0.49 米、高 1.0 米，重量大约 123 公斤，每条腿配备 3 个关节，能连续运行约 40 分钟。得益于液压驱动技术，它能够在负载 50 公斤的情况下以 1.28 米/秒的速度移动，并能攀爬 10 度的斜面，展现出与 Bigdog 相媲美的侧向冲击防御能力。不过，Scalf 使用发动机作为动力源，因此在移动时会产生较大的噪音。



图 1-10 Scalf 四足机器人^[28]

Fig.1-10 Scalf Quadruped robot^[28]

2014 年,哈尔滨工业大学的研究团队开发出了一款采用液压驱动的四足机器人^[29],如图 1-11。为降低机器人的整体重量,该机器人的构架采用了碳纤维复合材料,其腿部设计包括由对称液压缸提供动力的四个主动运动自由度,并辅以装有并联弹簧的直线滑动自由度。机器人内部集成了发动机、恒压变量泵、电池、电源模块、主控制系统、惯性导航单元以及各类液压系统组件,实现了完全无缆化操作。该机器人的自重约为 150 公斤,能够承载高达 52 公斤的重量,其设计中还特别加入了分布于四周的钢板以模拟不同的负载条件。

上海交通大学推出了一款具有并联腿部结构的四足机器人,名为小象^[30],如图 1-12 所示。由于其腿部并联结构的优点,增强了机器人的承载力,能够支撑超过 50 公斤负载;并研发了电机液压复合驱动器,可实现力位解耦控制。小象的体重低于 130 公斤,拥有 12 个自由度,最高速度能达到 4 公里/小时,且能攀爬最大 20 度的斜坡。它不仅能够实现慢行、小跑和爬坡等多种运动模式,还具有更优秀的运动稳定性。

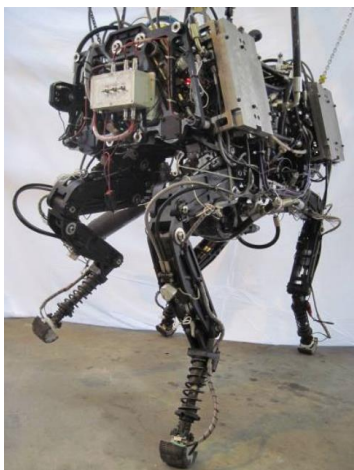


图 1-11 哈尔滨工业大学的四足机器人^[29]

Fig.1-11 The robot in HIT^[29]

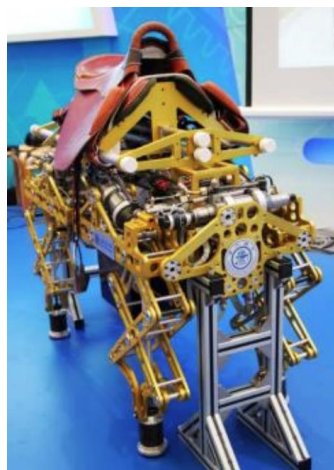


图 1-12 小象机器人^[30]

Fig.1-12 baby elephant^[30]

浙江大学对四足机器人的研究领域做出了显著贡献,特别是开发了赤兔和绝影系列机器人。绝影^[31,32]是一种由电机驱动的“机器狗”,如图 1-13 所示,体长 80 厘米,站高 60 厘米,体重 40 公斤,能够承载超过 10 公斤的重量,并且最高速度可达每小时 8 公里。这款机器人不仅走得快而且行走稳定,还装备了先进的环境感知系统,增强了其在多种应用场景下的实用性。绝影的四肢设计既包括轨迹控制也引入了力量控制功能,实现机器人的高动态控制,使其能够快速调整自身步态,保持稳定行走。

除了在高校科研领域,越来越多的公司也开始涉足四足机器人的研发,其中宇树科技公司走在前列。该公司于 2017 年发布了商用型四足机器人 Laikago^[33],如图 1-14 所示,Laikago 采用自主研发的大扭矩一体式关节模组驱动,噪音控制优秀,功率密度大,每条腿有 3 个自由度,机器人具有较强的抗干扰能力。经过技术迭代,宇树科技于 2020 年推出了 A1 四足机器人^[34],如图 1-15 所示,采用 MPC 控制算法加直接力控,运动性

能大大提升，速度也提升至最大 3.3m/s，能够行走和奔跑在不同的地形上，展现卓越的运动性能。该机器人还配备了丰富的传感器系统，包括激光雷达、摄像头等，以实现环境感知和导航功能。



图 1-13 绝影四足机器人^[32]

Fig.1-13 Shadow quadruped robot^[32]



图 1-14 Laikago 四足机器人

Fig.1-14 Laikago Quadruped Robot



图 1-15 A1 四足机器人

Fig.1-15 A1 Quadruped Robot

1.3 轮腿复合机器人研究现状

1.3.1 轮腿复合式机器人国外研究现状

轮腿复合式机器人与四足机器人的发展存在着紧密的联系，随着科技的不断进步和机器人技术的发展，轮腿复合式机器人在一定程度上弥补了传统四足机器人在速度和效率方面的不足。轮腿复合式机器人结合了轮式机器人和四足机器人的优势，既可以在平滑道路上快速移动，又可以在复杂环境中灵活行走。由于其高效的移动性能和优秀的越障性能，最早被应用于外星球探索。

苏联的 Lunokhod 1^[35]是 1970 年代初成功发射的月球探测车，成为了世界上第一辆成功登陆月球并执行任务的无人车辆，它于 1970 年 11 月 17 日由“月球 16 号”号探测器成功送达月球表面。如图 1-16 所示，车身两侧对称分布着八个轮子，通过两侧车轮的

差速控制实现转向功能，它还具备腿式运动模式，可以跨越 40 厘米高的障碍物，以及攀登 30 度的斜坡。美国于 1997 年向火星发射了“探路者号”，搭载了一款名为“Sojourner”的轮腿复合式机器人^[36]，它是人类历史上第一辆在火星表面执行任务的无人探测车。如图 1-17 所示，Sojourner 火星车采用了六轮驱动的设计，每个车轮都可以独立转动和悬挂，以提高车辆在不平坦的火星表面上的通过能力和稳定性，Sojourner 在轮式运动状态下能够保持 60m/h 的速度，得益于腿式连杆机构使其能够跨越 15 厘米高的障碍物。



图 1-16 月球车一号

Fig.1-16 Lunokhod-1

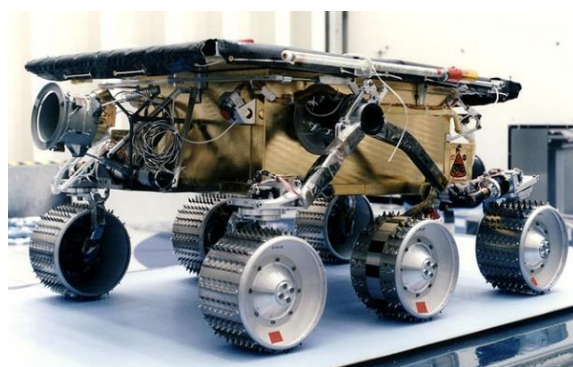
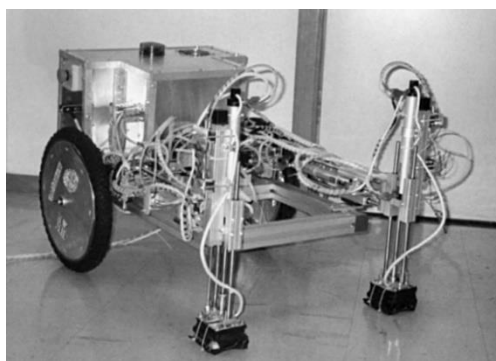


图 1-17 Sojourner 火星探测车

Fig.1-17 Sojourner Mars rover

轮腿复合式机器人在其他许多领域同样得到了广泛的应用。Wheeleg^[37]机器人（图 1-18）的设计和制造是为了研究轮/腿混合结构在崎岖地形上的运动能力，设想的可能应用是人道主义排雷和对火山等非结构化环境的勘探。Wheeleg 机器人有两个气动前腿，每个前腿有三个自由度，两个后轮通过使用两个不同的直流电机独立驱动，主要想法是使用后轮来承载机器人的大部分重量，并使用前腿来提高对表面的抓地力和攀爬障碍物。

图 1-18 Wheeleg 机器人^[37]Fig.1-18 The Wheeleg robot^[37]

2008 年，德国人工智能中心的 EichM 团队研制了一款名为 ASGUARD^[38]的多功能楼梯越障机器人，如图 1-19 所示，该机器人采用了异形轮设计，由五条均匀布置的柔性直线型轮辐轮腿构成。在爬越楼梯过程中，机器人的前轮和后轮轮辐轮腿会发生弹性变形，分别提供拉力和推力，以帮助机器人顺利通过障碍物。此外，机器人还通过在机身

上添加一个转动副构造柔性脊柱的方式，确保四个轮与地面有效接触，从而获得最大的楼梯爬升能力。这种设计使 ASGUARD 机器人在楼梯越障任务中表现出色，展示出强大的适应性和灵活性，为未来类似任务的机器人设计提供了有益的借鉴和启发。

2009 年，弗吉尼亚理工大学机器人与机构实验室开发了一款名为 IMPASS^[39] 机器人，这是一种带有主动轮辐系统的智能移动平台。该机器人采用了一种全新的设计，配备了两个无轮缘轮辐驱动，如图 1-20 所示，每个辐条都可以通过智能运动规划单独启动，使机器人能够在不平坦的地形上行走，并具有高度机动性。这种创新的运动方式将轮式机器人的效率和腿式机器人的机动性结合在一起，展现出了广阔的应用前景。具有高度移动性的 IMPASS 机器人在地形复杂和危险的应用中可能具有重要的价值，例如搜救、侦察或反恐反应等领域。



图 1-19 ASGUARD 机器人^[38]

Fig.1-19 ASGUARD robot^[38]



图 1-20 IMPASS 机器人^[39]

Fig.1-20 IMPASS Prototype^[39]

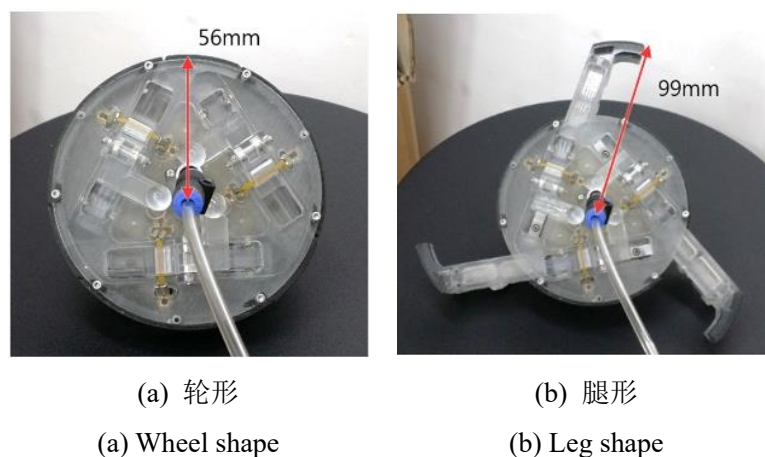
美国卡内基梅隆大学(CMU)开发了一款名为 RHex^[40] 的机器人，该机器人采用 6 个相同的半圆弧形腿部结构，左右对称安装，每条腿只有一个旋转自由度。由于腿部支架几乎集中在整体较薄的身体上，即使在倒置状态下，机器人也可以在更大的离地间隙下运行。X-RHex^[41] 是 RHex 平台的更新版本，如图 1-21 所示，其腿部结构得到了保留，主要对机身部分进行了优化，物理体积更小，占地面积和重量相当。此外，X-RHex 的大部分内部构件都使用了各种商用现货（COTS）组件，使其更易于制造和维护。



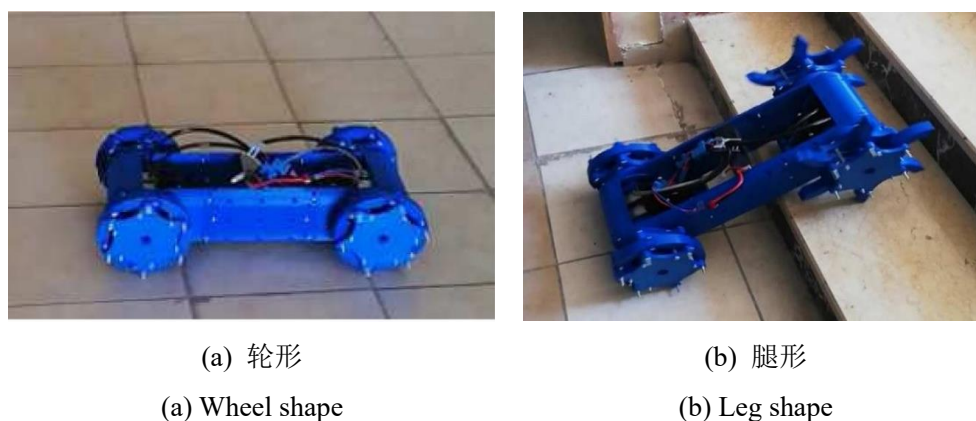
图 1-21 X-RHex 机器人^[41]

Fig.1-21 X-RHex Robot^[41]

韩国首尔国立大学提出了一种可以在高有效载荷和低操作力的情况下将其初始圆形转变为具有三条腿的变形车轮^[42]。在初始状态下，车轮是圆形的，半径为 56 毫米。在转换为腿形后，其半径为 99 毫米，如图 1-22 所示。这种轮子可以爬上 163 毫米高的障碍物，是其半径的 2.9 倍，每条腿的尖端都是弯曲的，这样它就可以挂在障碍物上。三个支腿位于轮体上，形成一个等边三角形，以使车轮从圆形模式转换为支腿模式时半径变化最大。

图 1-22 变形车轮^[42]Fig.1-22 Deformed wheels^[42]

土耳其埃拉泽菲拉特大学开发了一种新型可变形车轮腿的移动机器人 FUHAR^[43]，该机器人可用于平坦和粗糙的地形。如图 1-23，它使用带轮腿的可转换机构，将轮式机器人的稳定性和可操纵性与腿式机器人的越障能力相结合。通过基于四杆机构的转换结构，可以很容易地改变这两种模式。在崎岖不平的地形中，六片式车轮机构被打开，以确保爬楼梯和高障碍物的能力。

图 1-23 FUHAR 移动机器人^[43]Fig.1-23 FUHAR^[43]

瑞士苏黎世联邦理工大学(ETH)于 2018 年推出了 ANYmal 轮腿复合式机器人^[44,45]，如图 1-24 所示，该机器人结合了四条腿和轮子的设计，具有出色的灵活性和适应性，可

以在各种复杂环境中执行任务。ANYmal 的四条腿设计使其能够像四足动物一样穿越不规则地形，克服障碍物和登山等挑战。ANYmal 机器人配备了不可转向的、扭矩控制的轮子作为末端执行器，使其能够在平坦地面上快速移动，提高运动的效率。总的来说，ANYmal 轮腿复合式机器人的设计结合了多种优势，具有广泛的应用前景，为机器人技术在各领域的发展提供了重要的参考和启示。



图 1-24 ANYmal 轮腿复合式机器人^[45]

Fig.1-24 ANYmal Wheel-leg Composite robot^[45]

1.3.2 轮腿复合式机器人国内研究现状

轮腿复合式机器人在我国航天探测中同样扮演着重要角色。如图 1-24 所示，玉兔号月球车是中国国家航天局研制的月球车，于 2013 年 12 月 14 日成功降落在月球表面，成为世界上第三个成功软着陆并运行的月球车^[46]。为了高效完成任务，该月球车使用了六轮摇臂转向架式移动系统，使得月球车具备前进、后退、原地转向、行进间转向、20 度爬坡和 20 厘米越障的能力^[47,48]。

哈尔滨工业大学邓宗全院士团队参与了“祝融号”火星车移动系统的研究工作，如图 1-25 所示，该移动系统针对火星表面松软崎岖的地貌设计，有效解决了火星车在软沙中可能遇到的打滑自陷和牵引力不足等问题，为未来火星探测任务提供了重要的技术支持。

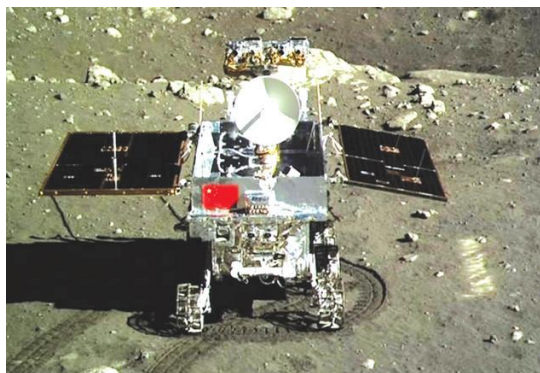


图 1-24 “玉兔号”月球车

Fig.1-24 "Jade Rabbit" lunar exploration rover

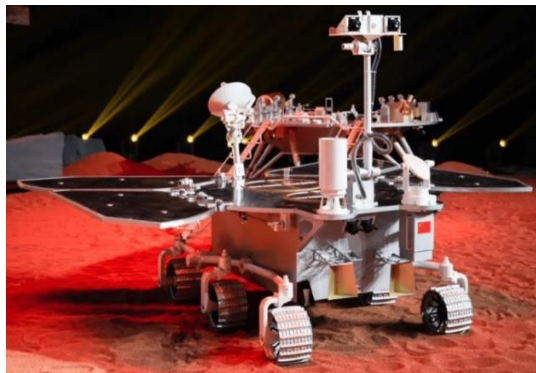


图 1-25 “祝融号”火星车

Fig.1-25 "Zhurong" Mars rover

在 2009 年，清华大学推出了一款轮腿复合式移动机器人^[49]，如图 1-26 所示。机器人采用对称式设计，主要由机体和 4 组轮腿式复合驱动机构组成，每组轮腿式复合机构由臀关节驱动轮、臀关节离合器、大腿、膝关节驱动轮、膝关节离合器、小腿、被动万向轮等部分组成，机器人系统共有 8 个主动驱动轮，4 个被动万向轮，驱动轮分别采用独立电机进行驱动，通过控制离合器，机器人可以转换到轮式、腿式、轮腿复合等多种运动模式。

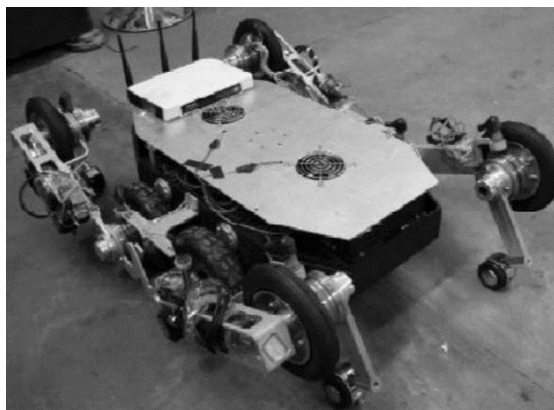


图 1-26 轮腿复合式移动机器人^[49]

Fig.1-26 Wheel-leg Composite robot^[49]

中国科学技术大学的董二宝等科研团队设计了一种新型的机械解耦腿轮混合式可变形机器人 HyTRo-I^[50]，如图 1-27 所示，该机器人由悬挂躯干、轮式移动底盘和四个模块化腿机构组成，悬挂躯干负责携带有效载荷和设备，轮式移动底盘悬挂在四足机器人的腹部，包含两个被动全向轮和两个主动驱动轮，这两个独立的驱动轮分别位于车身的左侧和右侧，而非驱动轮则分别位于车身的前后，通过腿部各个关节的收缩变形，机器人可以完成轮式和腿足式工作模式的切换。HyTRo-I 机器人的结构使其具有较强的适应性，适用于多样化的应用场景。

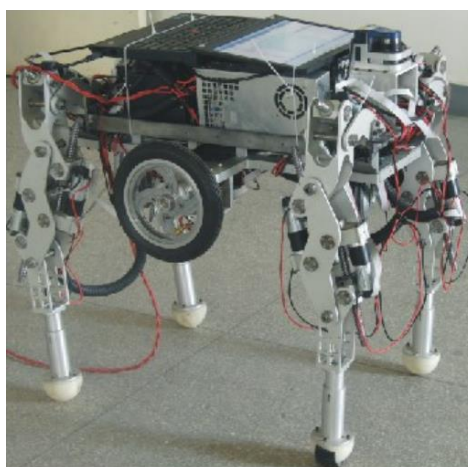


图 1-27 HyTRo-I 可变形机器人^[50]

Fig.1-27 HyTRo-I Transformable robots^[50]

2015 年, 东南大学提出了一款具有腰部关节的轮腿式机器人^[51], 如图 1-28 所示, 该机器人由前模块、活动腰关节和后模块组成, 前后两个模块中都装配有两个圆轮和两个半圆轮腿, 所有圆轮和半圆轮腿均由独立的伺服电机驱动, 机器人的腰部模块可主动调整, 使腰关节能够根据地形需求灵活旋转, 因此具备适应弯曲狭窄通道的能力。此外, 机器人采用对称结构, 使其在翻滚后仍能继续运动, 从而提供了出色的稳定性和适应性。



图 1-28 轮腿式机器人^[51]

Fig.1-28 Wheel-leg Composite robot^[51]

北京航空航天大学徐坤等人设计了一种六轮足机器人 NOROS^[52], 如图 1-29 所示, 该机器人采用六分支结构, 圆周对称排列, 每条腿包含腰关节、髋关节和膝关节三个主动关节, 足部通过球铰连接到小腿, 使得足部能适应支撑面形状, 在机器人站立步行时, 轮子不接触地面; 而在小腿折叠时, 轮子与地面接触, 实现轮式移动。这种设计使得机器人在不同情况下能够灵活适应地面形状, 具有较强的稳定性和多样化的移动能力。



图 1-29 NOROS 六轮足机器人^[52]

Fig.1-29 NOROS^[52]

2017 年, 东南大学设计了一种新型腿轮式机器人 Transleg^[53], 如图 1-30 所示。该机器人在身体的四个角落有四个可变形的腿轮机构, 因此在有腿模式下可以作为四足机器人, 在轮式模式下可以作为四轮车辆。机器人的身体由两部分组成, 通过脊柱连接。脊柱机构也由两个致动器驱动, 在偏航和俯仰方向弯曲, 这将有利于腿式和轮式模式下的转弯运动以及腿式模式下的跳跃步态。这种设计使得 Transleg 具有极大的灵活性和多

样化的运动能力，能够根据需要在不同模式下进行转换，适应不同的工作环境和任务需求。

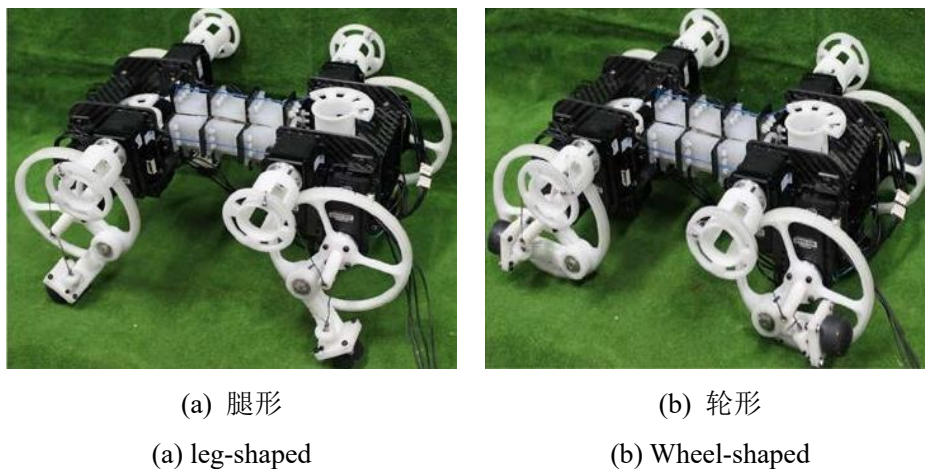


图 1-30 腿轮式机器人 Transleg^[53]

Fig.1-30 Transleg^[53]

2020 年，北京理工大学提出了一种电动并联六轮足机器人^[54]，如图 1-31 所示，该机器人采用多模式 Stewart 型腿结构，具有较大的负载能力，并集成了轮式运动和足式运动的优点，可以实现足式、轮式、以及轮足复合式运动。Stewart 平台是 6 自由度并联机构的基础平台，具有多自由度、大负载和高精度等特点，因此，研究者将 Stewart 平台倒置，在动平台上增加车轮驱动装置，形成轮足混合式单腿，而 6 条 Stewart 异型结构腿相对于机身平台呈正六边形均匀分布，这种设计使得该机器人在不同工作环境下能够灵活适应，并且具有较强的负载能力和运动灵活性。

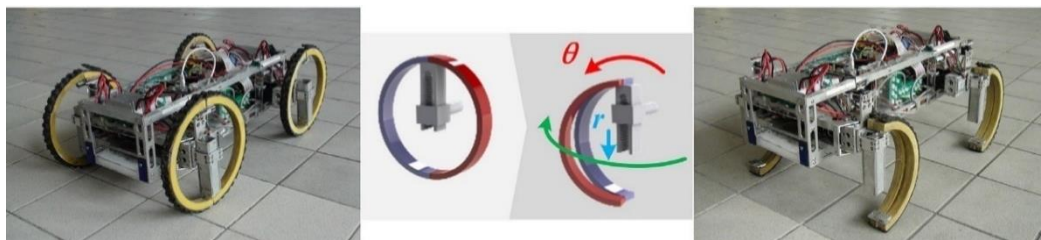


图 1-31 电动并联四轮足机器人^[54]

Fig.1-31 Parallel hexapod mobile robots^[54]

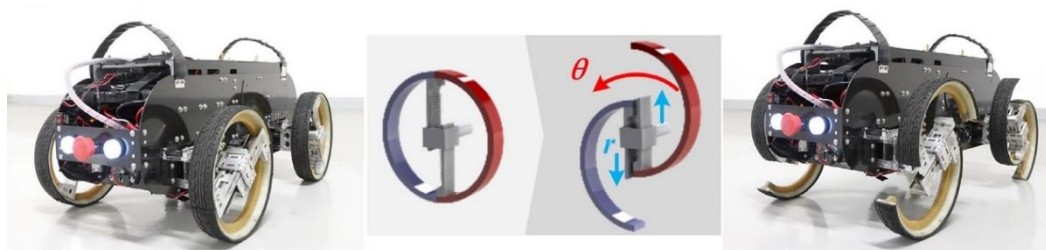
国立台湾大学的 LinPC 等人先后提出三款新型可变形轮腿^[55-57]，如图 1-32 所示。第一代用于 Quattroped 移动机器人^[55]的可变形轮腿采用转动副实现两个弧形腿的折

展，从而实现完整圆轮和 C 形腿之间的转换。由于轮腿无法在机器人行进过程中变形，第二代用于 TurboQuad 移动机器人^[56]的可变形轮腿被提出，轮腿采用移动辐实现两个弧形腿沿径向反向平移从而实现完整圆轮和 S 形腿之间的转换。由于前两代可变形轮腿越障高度有限，第三代由全转动副多环机构构成的新型轮腿机构^[57]被提出，该轮腿腿长可达圆轮轮径的 3.4 倍，且具有快速变形和弹跳能力。



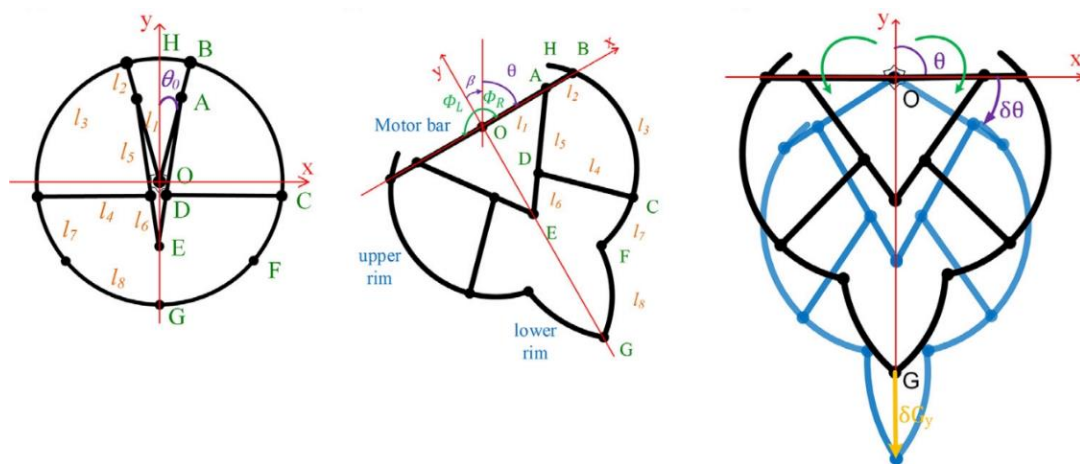
(a) 第一代可变形轮腿

(a) Quattroped



(b) 第二代可变形轮腿

(b) TurboQuad



(c) 全转动副多环轮腿机构

(c) Full rotation of the auxiliary multi-ring mechanism

图 1-32 可变形轮腿^[57]

Fig.1-32 Deformable legs^[57]

北京邮电大学张承耀等人将三自由度连杆机构和周转轮系进行结合提出一种基于周转轮系的多自由度轮腿机器人^[58]，如图 1-33，其中，1—左侧轮腿机构，2—中部平台，

3—右侧轮腿机构，该机器人采用左右两组腿部，每组包括一个周转轮系和两条三自由度轮腿；在周转轮系中，太阳轮轴与中部平台固定，而行星轮可以绕太阳轮旋转，每个周转轮系连接一条轮腿，每条轮腿由髋关节、膝关节和踝关节组成，形成一个三自由度的轮腿机构，左右两个轮腿存在一定的偏置，使得机器人能够在越障过程中实现稳定移动。这种设计结构使得机器人具备良好的适应性和稳定性，适用于复杂地面环境下的任务需求。

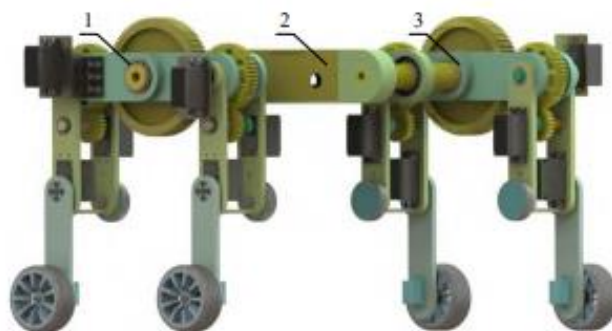


图 1-33 多自由度轮腿机器人^[58]

Fig.1-33 Multi-DOF wheel-legged robot^[58]

1.4 国内外研究现状总结

四足机器人的发展经历了从电机直驱到发动机液压驱动再到电液驱动和大扭矩关节模组驱动的演进过程，每种驱动形式都有其独特的优点和缺点。电机直驱简单可靠，但受限于功率密度和扭矩输出；发动机液压驱动具有高功率密度和扭矩输出，但噪声大、系统复杂、维护成本高且容易泄漏；电液驱动结合了电机和液压的优点，但需要更复杂的控制系统；电机驱动的大扭矩关节模组能够提供更大的扭矩输出和精准的控制，但成本较高。目前，大部分四足机器人采用电机驱动形式，电机驱动具有技术成熟、性能稳定、响应快速等优点，适用于各类任务场景。同时，随着电机技术的不断进步，如无刷电机、步进电机等的广泛应用，电机驱动在四足机器人中的应用越来越普遍。

四足机器人具有较好的稳定性和适应性，在复杂、不规则或崎岖的地形上表现出色，但在移动过程中，四足机器人的关节摩擦和足端与地面的冲击作用会导致能量消耗增加；此外，机器人腿部的周期性步态运动进一步增加了能耗；在摆动阶段，腿部的加速方向在半个周期内与机器人的移动方向相反，这一现象在支撑腿的能量消耗中尤为明显。因此，在平坦地面移动时，轮式结构的机器人在控制、速度和能效方面都优于腿足式结构，但在越障能力上存在明显不足。通过融合腿足式与轮式结构的设计，可以开发出多模式运动的轮腿复合式机器人，以显著改善上述单一结构的局限性。

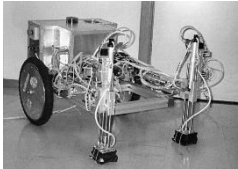
轮腿复合式机器人的发展在结构研究方面经历了多种形式的演进，其中包括简单拼

凑型、轮腿串联型、可变形轮腿和辐条轮等等，每种结构都具有各自的优缺点，为机器人在不同应用场景下提供了灵活的解决方案。最早的轮腿复合式机器人将轮子和腿部组件简单地拼凑在一起，这种设计简单易行，成本低廉，但在越障能力和适应性方面存在一定的局限性；后出现异形轮腿式，这种设计多采用非传统形状的轮腿组件，如多辐条型轮，可以提供更好的适应性，适用于复杂地形和环境，但同时也牺牲了机器人的平稳性；还有轮腿串联式和可变形式轮腿等。总的来说，轮腿复合式机器人的结构设计非常多样化，每种结构都有其独特的优点和局限性。未来的发展将继续探索创新的结构设计，以提高机器人在各种场景下的表现和应用范围。

如表 1-1 所示，轮腿复合式机器人大致可分为四种类型，各有优缺点。

表 1-1 轮腿复合式机器人分类表

Table1-1 Classification table of wheel-leg composite robots

结构类型	优点	缺点	典型范例
轮腿分离式	结构简单，控制容易，承载力强	越障能力受限，运动效率较低	
轮腿串联式	机动灵活，地形适应能力强，越障能力强	控制系统复杂	
异形轮腿式	结构简单，控制系统简单，便携	质心波动大，运动不平稳，承载力小	
轮腿变形式	结构紧凑，地形适应能力较强，控制系统简单	质心波动大，运动不平稳，越障能力有限	

综上所述，国内外对具有多运动模式的轮腿复合式移动的机器人的研究已经取得了一定的进展，但对该类机器人结构的创新设计以及在充分发挥轮腿两种模式的优点方面还存在不足。

1.5 本文主要研究内容

针对目前单一结构移动机器人的局限性，本文采用轮腿复合的形式，研究开发一种轮腿变形四足机器人，充分发挥轮腿两种模式的优点，为轮腿复合式机器人的研究提供

新的思路。文章研究的重点是对轮腿变形四足机器人进行结构设计与验证、关节模组电机的集成、运动学分析、机器人运动算法研究、进行虚拟样机仿真及分析以及对关键零部件进行样机实验。论文的主要内容如下：

第一章，讨论课题研究背景和意义、相关技术的国内外研究现状。

第二章，根据拱桥结构原理对轮腿变形机构进行设计与验证，对机器人整机进行设计。

第三章，采用 D-H 坐标变换法求解机器人正运动学方程，分析其腿部工作空间，计算雅可比矩阵。

第四章，研究电机控制算法以及机器人运动控制算法。

第五章，建立机器人虚拟样机模型，使用第四章的机器人运动控制算法进行动力学仿真。

第六章，对机器人关键零部件进行样机实验验证。

第七章，讨论本论文取得的主要研究成果，指出论文的不足之处和下一阶段要做的相关工作。

第2章 轮腿机器人总体结构设计

2.1 引言

在当前机器人技术日益发展的背景下,针对复杂工作环境需求,轮腿复合式机器人相比于单一结构的机器人综合性能更好,环境适应性更强。本章充分结合轮式和腿式机器人的优点,对轮腿变形式机器人进行结构设计。机器人满足以下设计要求:平坦地形下具备灵活的运动性和易于控制的特点,可以实现快速移动;在崎岖地形下拥有出色的越障能力和环境适应性;此外,它还要具备一定的运载能力;在面对不同地形环境时,能够自由切换轮式和腿式两种模式。本机器人使用轮腿变形式的设计,借鉴了拱桥结构能够承载重负荷的特性,提出了一个新颖的结构方案,通过对变形机构进行三维建模、运动仿真和有限元分析,验证了方案的可行性。将此结构作为四足机器人的小腿部分,设计机器人总体结构,包括机器人大腿、一体化关节、躯体部分和导电滑环等。

2.2 轮腿变形机构设计

2.2.1 拱桥结构分析

拱桥是一种古老且常见的桥梁结构形式,它能够承受大负载的原理主要基于拱形结构对力的分布和传递方式,如图 2-1 所示。

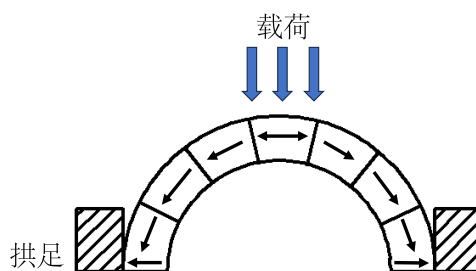


图 2-1 拱桥结构原理

Fig.2-1 Structural principle of arch bridges

拱形结构自然形成一种几何锁定效应,这种结构在受力时,每个部分都会相互支撑,形成稳定的自平衡状态。拱的形状能够将上部结构的垂直压力转化为拱脚的水平推力,这种力的转化减少了材料的弯矩,从而增强了结构的稳定性。拱桥在受到上部荷载时,荷载通过拱圈传递到拱脚。在整个过程中,荷载主要以压应力的形式存在,这使得拱桥可以使用石材、砖块或混凝土等材料建造,这些材料在抵抗压力方面表现得非常出色。拱桥的拱足通常会被桥台或其他结构所约束,这些约束防止了拱足的外移,从而确保了整个拱结构的稳定性。

在一些设计中,拱足之间还会通过拉杆或者其他结构连接起来,以进一步提高结构的稳定性。通过这些原理,拱桥能够承受自重、行车荷载、风荷载等多种负载,并且能够在不同的环境条件下提供长期稳定的服务。

2.2.2 变形原理

在对拱桥结构分析后,利用其结构特点提出了链式结构方案,可实现机构在腿形与轮形两种状态的切换,在轮形态下拥有类似拱桥结构的负载能力。变形原理如图 2-2 所示。本机构由特殊结构的扇形链节通过铰链连接而成。设单个链节对应的圆周角度为 α ,相邻链节间的角度为 θ ,通过链节间的接触面的角度设计实现限位能力,相邻链节间的相对可转动角度范围为 $[0, \alpha]$,则当所有相邻链节间转角 θ 都等于 α 时,铰链副上方的面接触起到限位作用,链节呈直线排列状态,对应腿形构型;当所有相邻链节间转角 θ 都等于 0 时,铰链副下方的面接触起到限位作用,链节呈圆周排列状态,对应轮形构型。

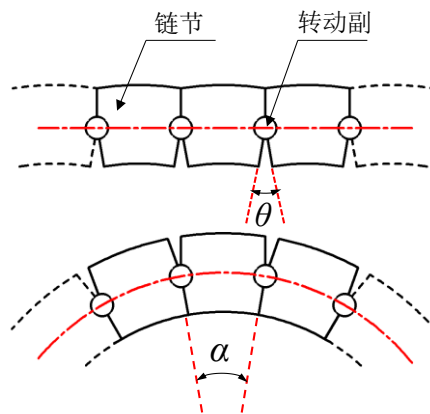


图 2-2 机构变形原理图

Fig.2-2 Schematic diagram of mechanism deformation

将拱桥型的上述两个半圆结构相连后可组合成完整的圆形,两端使用可伸缩拉杆连接,起到拱桥中的拱足的约束作用,两半圆的连接处结构原理如上图 2-3 所示,图中的末端链节起到连接两半圆结构的作用,它的转动副的位置与构成轮形的主要链节不同,可实现两种形态间的切换。

但这种结构还存在一个缺陷,即轮形态下末端链节与相邻的链节间约束不够,影响了整体结构的负载能力,所以需要在不增加驱动装置的情况下研究开发一个限位机构来增加约束,配合变形时伸缩杆的动作来实现限位机构的开合。末端限位机构原理如下图 2-4 所示。该机构使用双滑块连杆机构,当变形机构处于轮形态时,伸缩杆为收缩状态,滑块位于右端,如上图所示初始状态,当链节受到图中所示弯矩 M 时会通过连杆机构变为力 F 传递到固定在末端链节的铰链处,此时连杆构成一个直角三角形,力 F 的方向指向铰链,起到限位约束作用;当伸缩杆伸开时,变形机构向腿形态转变,如图中步骤 1,滑块向左运动,使得固定在末端链节铰链处的连杆逆时针转动,约束位置打开;随后进行步骤 2,链节逆时针转动,变形机构变为腿形态,完成变形。

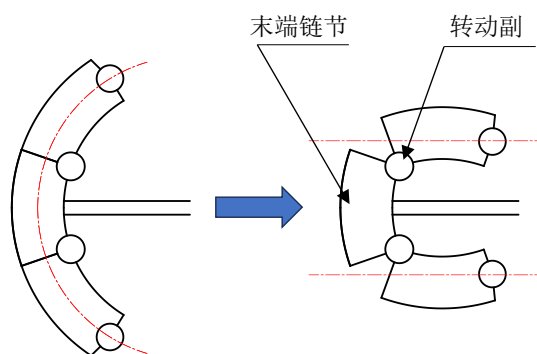


图 2-3 末端连接处结构图

Fig.2-3 Diagram of the end connection structure

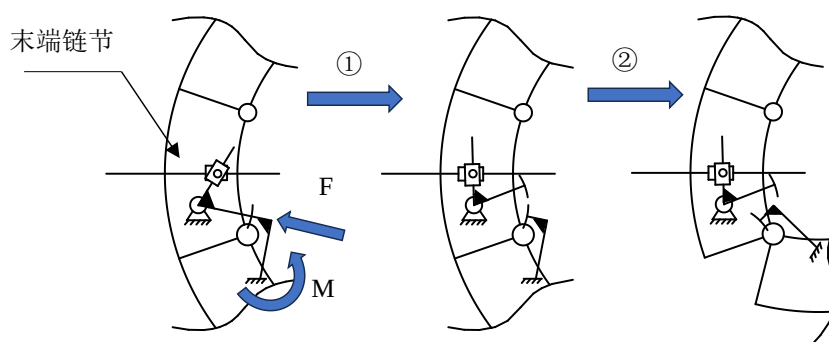


图 2-4 末端限位机构原理图

Fig.2-4 Schematic diagram of the end limiting mechanism

2.2.3 结构设计

确定变形原理与基本构型后,进行可变形轮腿结构的详细设计。综合考虑到轮式与腿式结构转变的彻底以及轮式状态下的运动平稳性,将轮腿结构分为 36 个链节,每个链节对应轮式下圆周角 10 度。如图 2-5 所示为设计的可变形轮腿机构的整体结构,其中,1—型号 b 链节;2—型号 a 链节;3—过渡链节;4—末端链节;5—电动推杆;6—末端限位机构。可以看出可变形轮腿机构由 4 种扇形链节、末端限位机构和电动推杆三部分部件组成,各链节间均采用铰链连接形式,机构整体呈轴对称形式,由型号 a 链节、型号 b 链节和过渡链节通过铰链连接为半圆形机构,可实现半圆型和直线型的变换,再将两个半圆变形机构使用两个末端链节进行连接组合,即可构成完整轮形。两个末端链节连接在推杆两端,当推杆收缩到指定的轮形直径长度时,同侧链节被动运动转角为 0 度的圆周排列状态,此时由于电动推杆的自锁,轮腿机构锁定在轮形态;而当推杆伸长到指定的腿形长度时,同侧链节被动运动转角为 10 度的直线排列状态,轮腿机构锁定在腿形态,此状态下,由推杆来承受负载。

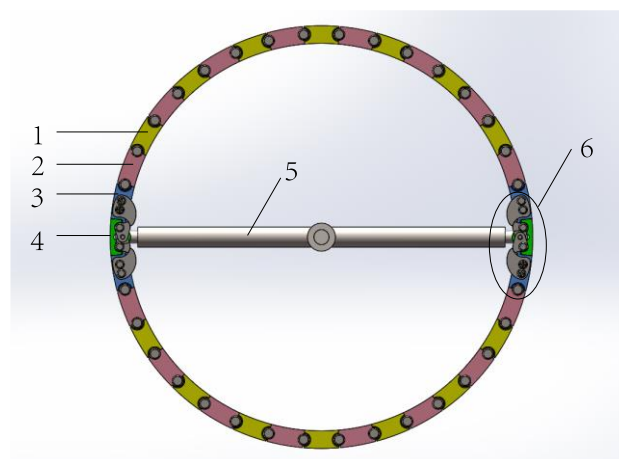


图 2-5 可变形轮腿
Fig.2-5 Deformable legs

本轮腿利用拱桥结构原理设计的部分主要由两种链节组成，如图 2-6 所示，型号 a 链节和型号 b 链节通过中间的销轴销钉连接形成铰链副，通过限位面 1 和限位面 2 的约束作用构成图 2 所示的两种状态，当限位面 1 相接触起作用时，所有链节呈直线排布，进而构成腿型态；当限位面 2 相接触起作用时，所有链节呈圆周排布，进而构成轮型态。半个变形轮使用 8 个型号 a 的链节和 7 个型号 b 的链节，整个变形轮则需要 16 个型号 a 的链节和 14 个型号 b 的链节。

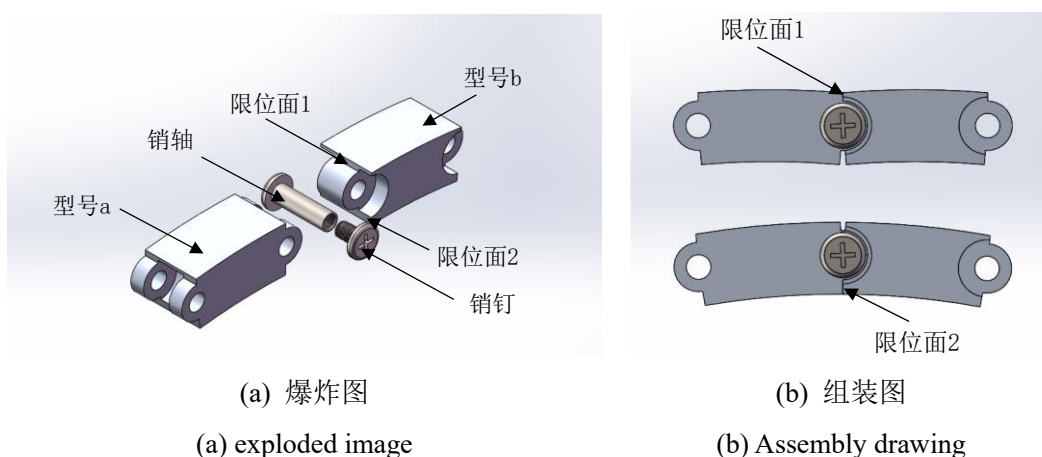


图 2-6 型号 a 链节与型号 b 链节
Fig.2-6 Type A and Type B links

随后就是将两个半圆连接起来构成完整的轮形，如下图 2-7 所示，首先将末端链节连接在电推杆的两端，末端链节上开有直径 12mm 的通孔与电推杆两端直径相等，再插入销轴连接，末端链节上的槽口与销轴进行配合，槽口长度为 6mm，使得推杆轴上的销轴能够相对于末端链节在槽口内移动，再将限位连杆通过销钉轴铰接在末端链节上，限位连杆另一端也开有槽口，于中间的销轴配合，可将中间销轴的平移运动转变为限位连杆的旋转运动，进而实现限位机构开合切换。将过渡链节与末端链接使用圆柱销进行铰

接，再将限位片固定在过渡链节上，限位片与限位连杆接触的面是以销钉轴为圆心的圆柱面，所以它并不会干涉限位连杆的旋转运动，且直到限位连杆完全旋转到位之前限位机构都能一直起到限位作用，至此限位机构安装完毕。

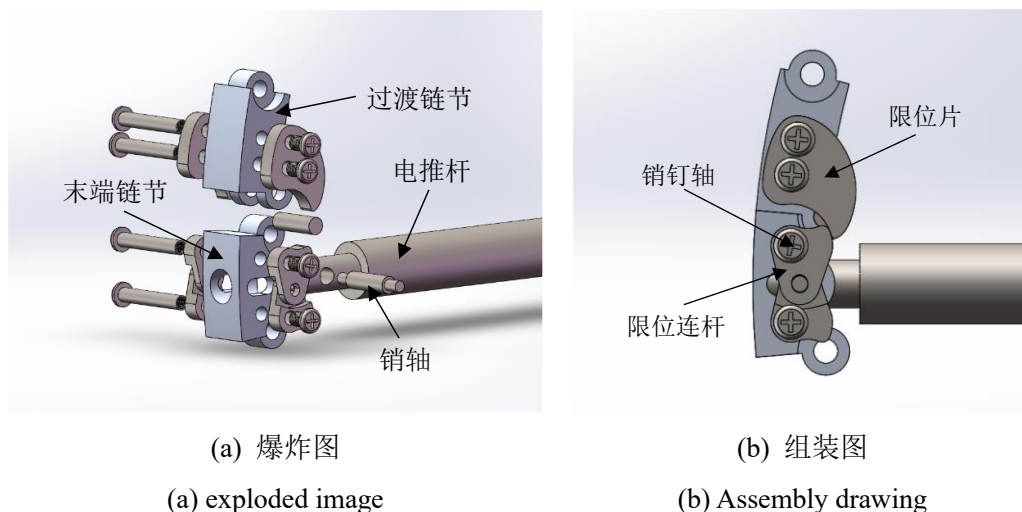


图 2-7 限位机构

Fig.2-7 Limiting mechanism

最后将由型号 a 和型号 b 构成的半圆形变形机构与过渡链节连接即可构成完整的变形轮。轮腿变形步骤如图 2-8 所示。初始状态为轮型，电推杆处于收缩状态，末端限位机构处于约束位置，变形开始后，推杆往两头伸开，销轴带着限位连杆跟随推杆一起运动，通过滑块连杆机构做旋转运动，直到离开约束位置，随后上下两个半圆结构向腿形结构变换，待推杆到达指定位置后变形结束，变形机构成功从轮形变换为腿形。上面步骤逆向操作则为腿形变换为轮形的过程。

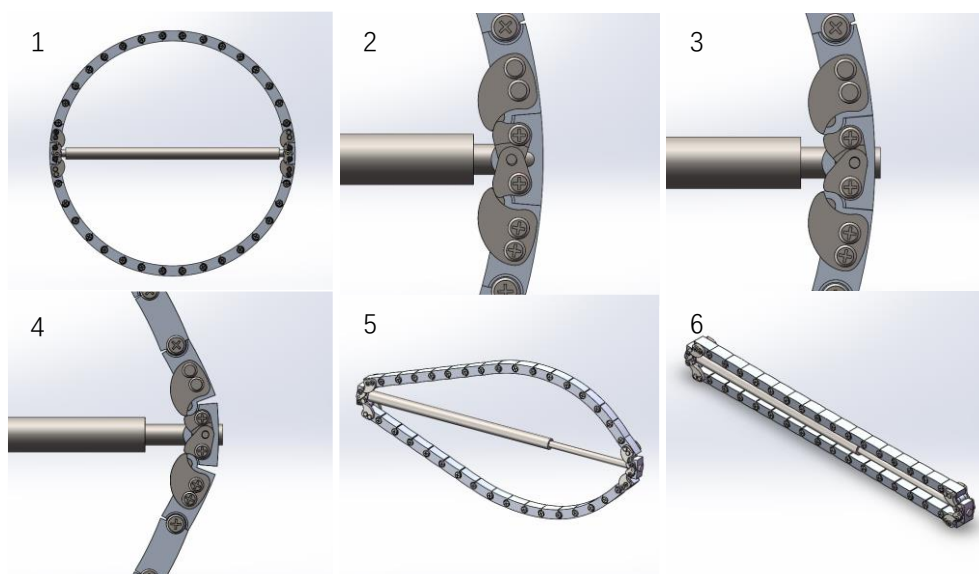


图 2-8 轮腿变形步骤

Fig.2-8 Leg deformation steps

轮腿可变形机构共有 36 个链节，单个链节对应轮式下圆周角 α 等于 10 度，如图 2-9 所示，设轮形态下的轮直径为 R ，腿形态下腿可用长度为 L 。

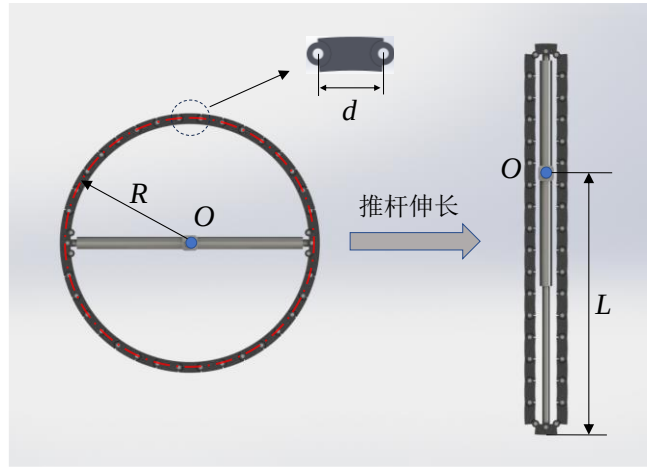


图 2-9 轮腿两形态尺寸对比图

Fig.2-9 Comparison of the dimensions of the two forms of wheel legs

单个链节长度

$$d = 2R \sin \frac{\alpha}{2} \quad (2-1)$$

推杆伸长腿的总长为单侧 17 个链节长度之和，实际可用长度需减去半径 R ，即：

$$L = 17d - R = 1.96R \quad (2-2)$$

期望的机器人尺寸大小 $R=210\text{mm}$ 。则

$$L = 412\text{mm} \quad (2-3)$$

经计算可知变形后的可用小腿长能达到约 2 倍的轮半径，再加上大腿的长度，机器人的腿部可到达的高度较轮半径更大。因此本文所研究的轮腿可变形移动机器人具有优越的越障能力。

2.2.4 有限元仿真分析

由于腿形态下主要由推杆承担负载，所以只使用 ANSYS 对轮形态进行有限元分析，考虑到机器人的强度和轻量化要求，四种链节都采用 7075T6 超硬铝合金材质加工，材料的屈服强度为 524MPa，取安全系数 $n=1.5$ ，材料的许用应力为 349MPa。末端限位机构及销钉轴采用 4140 正火低合金钢，材料的屈服强度为 652MPa，取安全系数 $n=1.5$ ，材料的许用应力为 434MPa。为提升计算速度将模型进行简化后再导入 ansys 进行仿真，分别在轮形的不同部位施加载荷，测试轮形态负载能力。

通过 ANSYS 仿真可知，负载能力最强的位置为远离推杆两端的中间位置，当在轮底部中间位置施加垂直于地面向上的载荷，大小设置为 600N 时，仿真结果如下图 2-10 所示，其中，(a) 为轮腿机构总变形云图，最大变形量为 0.758mm；(b) 为轮腿机构整体应力云图，最大应力为 235.11MPa；(c) 为铝合金材质零件的应力云图，最大应力为

235.11 MPa; (d) 为应力最大处销轴的应力云图, 最大应力为 199.33 MPa; (e) 为应力最大处限位连杆的应力云图, 最大应力为 33.624MPa; (f) 为应力最大处限位片的应力云图, 最大应力为 41.815MPa。从有限元仿真结果可以看出单轮负载 600N 情况下, 整体最大变形量为 0.757mm, 7075 铝合金材质的零件最大应力为 235.11MPa, 合金钢材质的零件最大应力为 199.33 MPa, 均小于其屈服强度, 且在许用应力范围内。

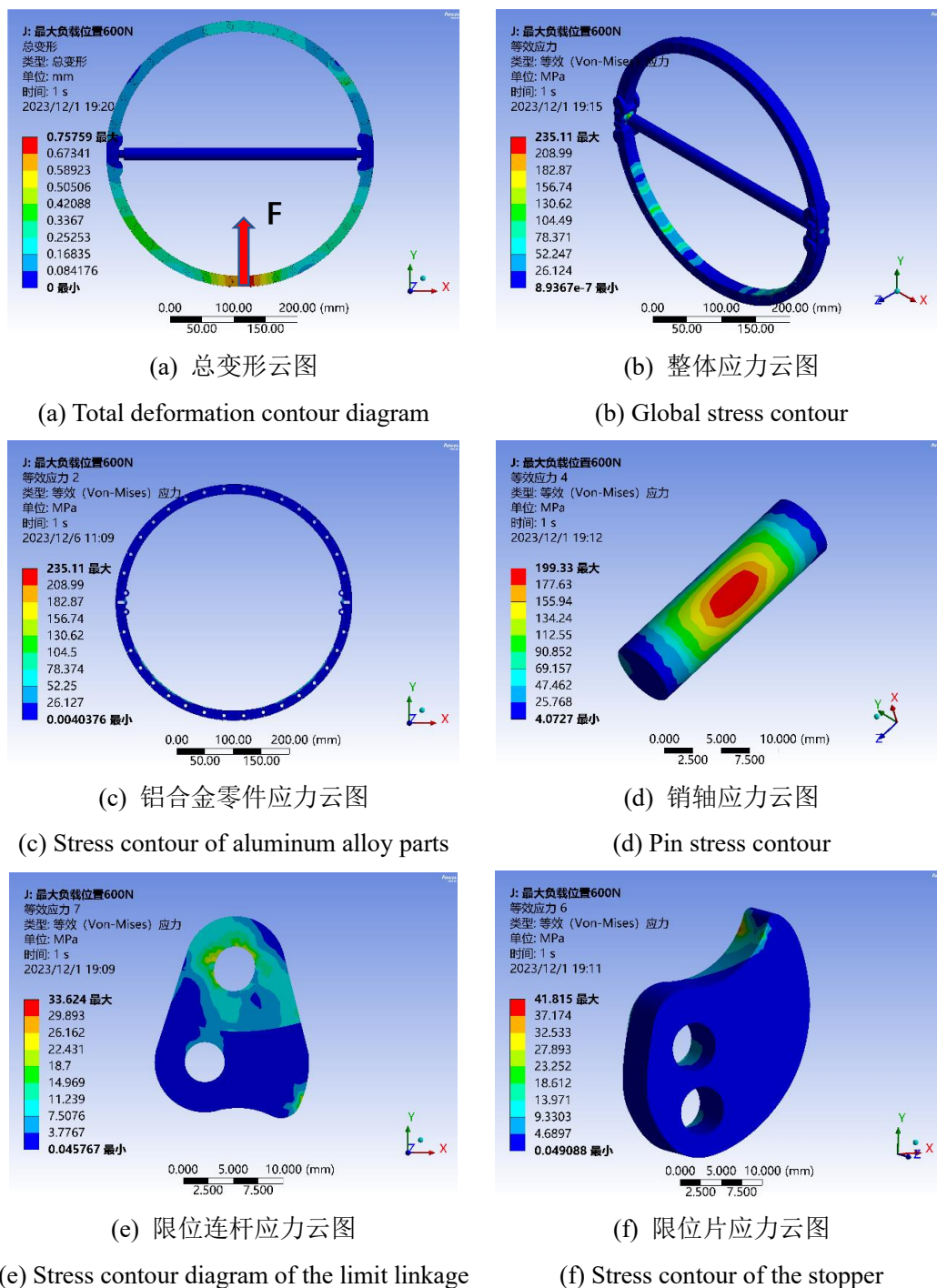
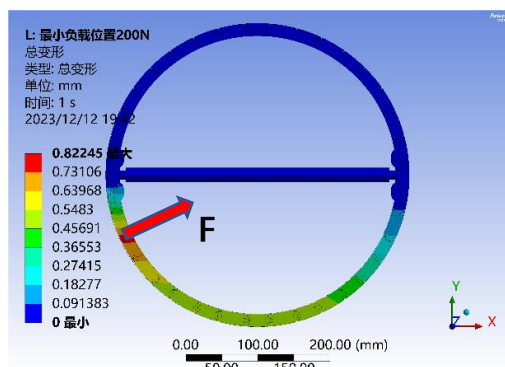


图 2-10 负载 600N 静力学有限元分析结果

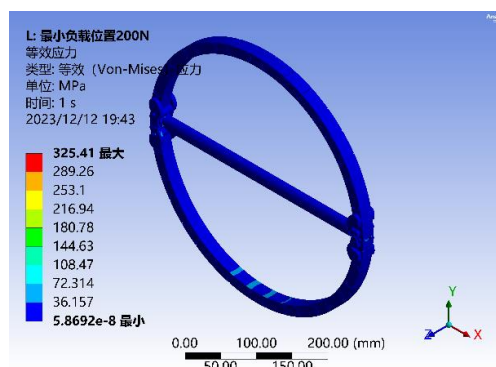
Fig.2-10 Static finite element analysis results with a load of 600N

负载能力最弱的位置为靠近推杆两端的位置，当在该位置施加指向轮子轴心方向的载荷，大小设置为 200N 时，仿真结果如下图 2-11 所示，



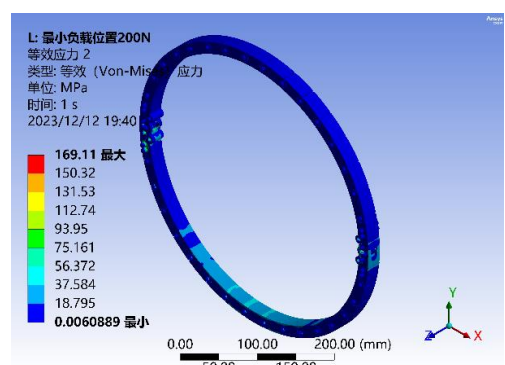
(a) 总变形云图

(a) Total deformation contour diagram



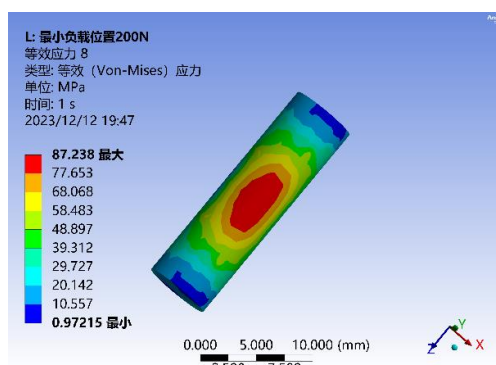
(b) 整体应力云图

(b) Global stress contour



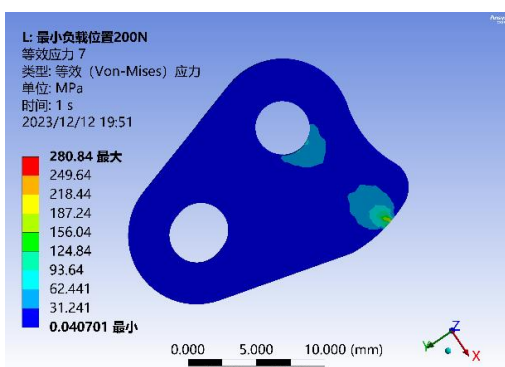
(c) 铝合金零件应力云图

(c) Stress contour of aluminum alloy parts



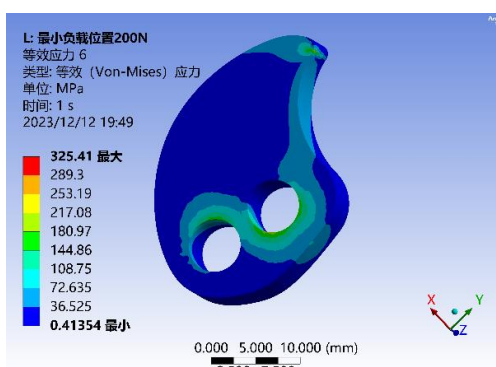
(d) 销轴应力云图

(d) Pin stress contour



(e) 限位连杆应力云图

(e) Stress contour diagram of the limit linkage



(f) 限位片应力云图

(f) Stress contour of the stopper

图 2-11 负载 200N 静力学有限元分析结果

Fig.2-11 Static finite element analysis results with a load of 200N

其中 (a) 为轮腿机构总变形云图，最大变形量为 0.823mm；(b) 为轮腿机构整体应力云图，最大应力为 325.41MPa；(c) 为铝合金材质零件的应力云图，最大应力为 169.11

MPa; (d) 为应力最大处销轴的应力云图, 最大应力为 87.238 MPa; (e) 为应力最大处限位连杆的应力云图, 最大应力为 325.41MPa; (f) 为应力最大处限位片的应力云图, 最大应力为 280.84MPa。从有限元仿真结果可以看出负载能力最弱处单轮负载 200N 情况下, 整体最大变形量为 0.822mm, 7075 铝合金材质的零件最大应力为 169.11MPa, 合金钢材质的零件最大应力为 325.41 MPa, 均小于其屈服强度, 且在许用应力范围内。

从以上分析结果可知, 单轮最大负载能力不低于 200N, 四轮总体负载能力不低于 80kg, 预估机器人整体重量为 40kg, 满足设计要求。

2.3 机器人总体结构设计

2.3.1 结构方案与驱动方式选择

随着电机和电池技术的快速发展, 电驱动方案在移动机器人领域成为首选方案。电机驱动具有许多优势, 能够为移动机器人提供高效的能量转换和传输。现代电机技术具有高功率密度和高效率, 能够将电能有效地转化为机械能, 从而提供足够的动力驱动机器人的运动; 此外, 电机驱动还具有可编程性, 可以通过调整控制算法和参数来实现精确的运动控制和路径规划, 使机器人能够适应不同的工作环境和任务需求。

在结构方案的选择上, 四足结构在移动机器人中具有独特的优势。相较于双足结构, 四足结构具有更好的稳定性和平衡性, 能够在不平坦的地面上保持稳定的姿态; 四足结构的多点支撑使机器人能够分布载荷, 减少对单个腿的负荷, 提高机器人的承载能力和运动灵活性; 此外, 四足结构还具备良好的越障能力和适应性, 能够应对各种地形和障碍物, 使机器人能够在复杂环境中自由移动。

电机驱动和四足结构的组合为足式机器人提供了高效的动力和稳定的运动能力, 这种结构方案在工业自动化、救援任务和探索领域等多个应用场景中具有潜在的优势。基于以上考虑, 选择了电机驱动和四足结构的组合作为足式机器人的结构方案。

2.3.2 机器人总体结构及参数

在机器人领域中, 单腿尺寸对整个四足机器人大小有显著影响, 不同尺寸的四足机器人在步态设计、运动控制以及驱动需求等方面都会存在差异; 研究发现, 当大腿长度与小腿长度的比值越接近 1:1 时, 四足机器人的工作空间范围越大^[59]。更大的工作空间表示机器人具有更强的越障能力和地形适应性。因此, 在相同条件下, 扩展工作空间可以使机器人在复杂环境中更加灵活地移动和操作, 提升其面对各种挑战的能力, 从而增强机器人在实际任务中的表现和效率。机器人尺寸参数如表 2-1 所示。

表 2-1 机器人参数

Table2-1 Robot parameters

名称	参数	单位
前后髋关节距离	900	mm
左右髋关节距离	200	mm
轮直径	420	mm
小腿长度	420	mm
大腿长度	420	mm
质量	40	kg

如图 2-12 所示为机器人腿形态，图 2-13 所示为机器人轮形态，机器人总体采用具有 12 主动自由度的四足移动机器人结构设计，串联关节式腿部结构，单腿由 3 个旋转关节驱动（一个横滚自由度，两个俯仰自由度），小腿为电动推杆驱动的可变形机构。通过电动推杆伸长和收缩控制小腿形态的变换，可实现机器人在轮式和足式两种运动模式间的切换。



图 2-12 腿形态

Fig.2-12 Leg morphology



图 2-13 轮形态

Fig.2-13 Wheel morphology

2.3.3 一体化关节模组设计

关节模组作为机器人的动力输出部件，通常由力矩电机、减速器、电机驱动器和反馈编码器等多个零部件集成而成。

在关节模组中，电机起着至关重要的作用，它作为力矩致动器的核心，需要提供可靠的动力源。为了满足高功率密度的要求，通常选择体积小、力矩大且无需电刷换向的直流无刷电机作为驱动电机；另一方面，为了实现关节输出大扭矩的需求，关节模组中还需要搭配适当的减速器。常见的减速器类型包括行星减速器、谐波减速器、摆线针轮减速器和 RV 减速器等，这些减速器各自具有特定的优势和适用场景，例如，行星减速器具有低成本、高传动效率和强承载能力；谐波减速器体积小、减速比范围大；RV 减速器承载能力强、减速比范围大，但制造工艺复杂、成本较高；摆线针轮减速器相比行

星减速器体积小、减速比范围大，能输出更大扭矩且承载能力强，综合考虑后选用摆线针轮减速器。编码器用于电机运行时转子位置和转角信息的收集，常用的编码器有光电编码器、磁性编码器和电容式编码器等。本文选用的霍尔编码器属于磁性编码器，是一种非接触式传感器，它通过测量磁场的变化来确定位置，因此不需要物理接触，这使得霍尔编码器在长时间使用中不易磨损，具有较高的寿命和可靠性，且霍尔编码器的测量过程响应速度较快，具有较高的实时性。

(1) 无刷电机选型

最大扭矩输出是机器人驱动关节的一个重要参数。第四章虚拟样机仿真中，按机器人自重 40kg，负载 10kg 粗略估算，当机器人以 1.5m/s 的速度以 trot 步态行走时，机器人膝关节关节扭矩的峰值约为 60N·m，此时转速为 20r/min 左右，关节转速的峰值约为 80 r/min，摆线针轮减速器传动效率为 80%，取安全系数为 1.5，可以计算出所需电机功率为：

$$P > \frac{2\pi \times n \times T}{60 \times \eta} \times 1.5 \quad (2-4)$$

解得：

$$P > 235.62W \quad (2-5)$$

选用 9910 外转子直流无刷电机，额定输出功率可达 300W。图 2-15 所示为 9910 无刷电机在 36V 直流电源下的性能参数。



图 2-15 直流 36V 下电机性能图

Fig.2-15 Motor performance at DC 36V

(2) 关节模组结构设计

图 2-14 为 9910 外转子直流无刷电机外观图，如图 2-14 (b) 所示，电机背面装有

霍尔编码器，可有效提高关节的集成度，节约空间。

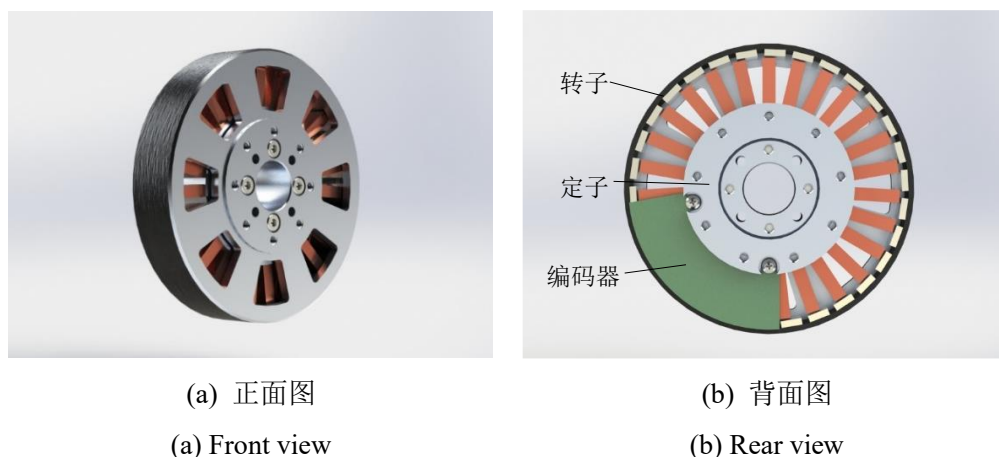
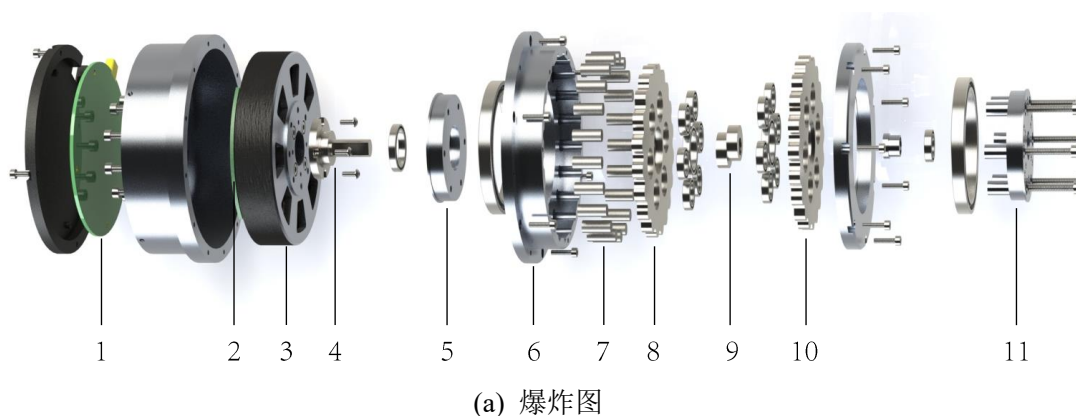


图 2-14 9910 外转子直流无刷电机

Fig.2-14 9910 External Rotor Brushless DC Motor

结合 9910 无刷电机尺寸设计关节模组，使用 SolidWorks 软件建模后，将带编码器的无刷电机、摆线针轮减速器、驱动器等部件通过轴承、螺栓连接，组合后模组厚度为 72.5mm，直径为 130mm，如图 2-16 所示。无刷电机与摆线减速器相连，增大关节模组输出扭矩，编码器与驱动器放置在模组尾部。根据扭矩需求选取减速比为 20:1 的摆线针轮减速器，组合后的关节模组理论上的额定输出扭矩为 40 Nm，最大瞬时扭矩超 100Nm，最大转速为 12.57 rad/s，满足性能要求。



- 1.电机驱动器 2.编码器 3.无刷电机 4.输入轴 5.内端盖
6.针齿壳 7.针齿销 8.摆线轮 9.偏心轴套 10.摆线轮 11.输出端盖

(a) Exploding Drawing

- 1.Motor driver 2.Encoder 3.Brushless motor 4.Input shaft 5.Inner end cover
6. Needle tooth shell 7. Needle tooth pin 8. Cycloidal wheel 9.Eccentric bushing
10. Cycloidal wheel 11. Output end cover



(b) 组装图

(b) Assembly Drawing

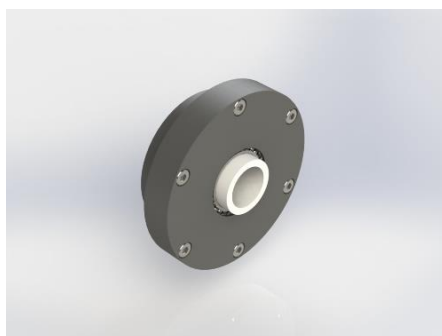
图 2-16 关节模组

Fig.2-16 Joint Module

2.3.4 导电滑环设计

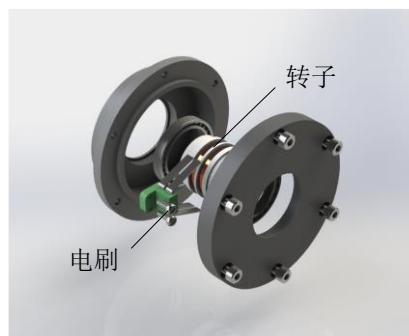
当机器人处于轮模式下时，轮子需要做连续旋转的运动，此时电推杆出现绕线的问题，所以需要连接一个导电滑环来传递旋转连接的电力和信号。导电滑环是一种用于在旋转部件上传输电力或信号的装置。它通常由导电材料制成，用于连接固定部分和旋转部分，以便在旋转过程中保持电流或信号的传输。导电滑环通常由两部分组成：固定部分和旋转部分。固定部分连接在固定不动的结构上，而旋转部分连接在需要旋转的部件上，例如电机的转子或者旋转的机械臂。

设计的导电滑环结构如图 2-17 所示，它由导电刷子和导电环组成，电刷为弹性金属夹片，通过弹性力夹在导电环上，导电环为铜环，固定在转子上。导电刷子连接在固定部分上，而导电环连接在旋转部分上，外壳为固定部分，转子为旋转部分，转子两端通过两个轴承与外壳连接。当转子转动时，导电刷子与导电环之间会保持接触，从而形成一个持续的电气连接。



(a) 组装图

(a) Assembly Drawing



(b) 爆炸图

(b) Exploding Drawing

图 2-17 导电滑环

Fig.2-17 Conductive slip rings

2.3.5 腿部连接件设计

为减小腿部惯量，将三个关节均布置在机器人腿部髋关节处，如图 2-18 所示。

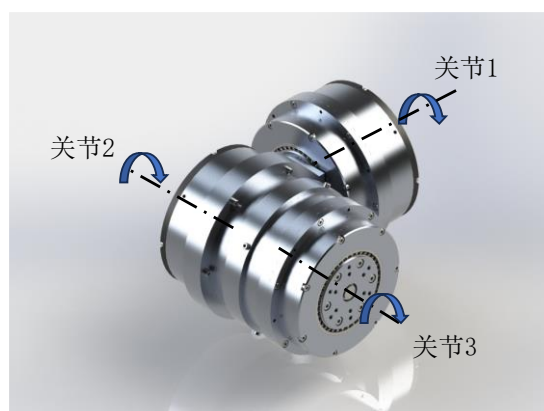


图 2-18 单腿三个关节的组装图

Fig.2-18 Diagram of the assembly of three joints in a single leg

更小的腿部惯量可以带来多方面的好处，可以使机器人更容易改变运动方向和速度，提高其灵活性和敏捷性；减少机器人运动时所需的能量，从而延长电池续航时间或减少对外部能源的依赖；减少机器人在运动和转向时所受到的惯性力，有助于降低机器人结构的应力，延长机器人的使用寿命；提高稳定性，使机器人更容易保持平衡，对于在不平整或动态环境中运动的机器人尤为重要；改善运动控制，提高运动控制的响应速度，使机器人更容易实现精确的运动和动作。

关节 1 为胯关节，关节 2 为髋关节，关节 3 为膝关节，各关节间连接法兰如下图所示，图 2-19 为法兰 1，作为关节 1 和关节 2 的连接法兰；图 2-20 为法兰 2，作为关节 2 和关节 3 的连接法兰。



图 2-19 法兰 1

Fig.2-19 Flange 1



图 2-20 法兰 2

Fig.2-20 Flange 2

小腿的运动通过同步带传动方式从关节 3 的输出轴传动到膝关节转轴处，大腿连杆使用碳纤维材料整体加工，以达到轻量化的目的，小腿推杆通过导电滑环进行电气连接，保证推杆能够正常运行。具体结构如图 2-21、图 2-22 所示。



图 2-21 大腿结构爆炸图

Fig.2-21 Exploded view of the leg structure



图 2-22 单腿组装图

Fig.2-22 Single-leg assembly drawing

2.4 本章小结

本章节通过分析拱桥结构的负载特性提出了一种新型可变形轮腿结构，并对其进行了优化改进，利用 Solidworks 对其进行三维建模及装配，使用 motion 模块进行运动仿真，再利用 Ansys 对其进行静力学有限元分析，验证了轮腿结构的可行性。而后通过分析确定了机器人采用四足结构作为基本构型，关节采用电驱动方式，进而设计了机器人关节模组、大腿连杆、导电滑环等部件，完成了机器人的整体结构设计，并进行了三维建模装配，为后续仿真做准备。

第3章 机器人运动学分析

3.1 引言

四足机器人以其出色的机动性而闻名，特别是在未知且复杂的非结构化地形中表现突出。基于第二章设计的机器人整体结构尺寸，本章通过建立机器人腿部运动学模型，对机器人四肢进行正逆运动学分析、工作空间的求解及雅可比矩阵的计算。

3.2 单腿运动学及工作空间分析

通过正运动学分析，可以推导出关于机器人各关节运动的方程，从而确定腿部末端的具体位置；而逆运动学分析则是从预期的末端位置出发，逆推出所需的关节运动角度。通过综合正逆运动学分析，将计算机器人末端的可达空间，以验证其是否能够满足复杂地形的越障需求。

3.2.1 正运动学分析

使用 D-H 参数法对机器人腿部进行正运动学分析，根据机器人各部件的尺寸及运动关系建立基于 D-H 参数的机器人坐标系，按照图 3-1 中所示的机器人单腿关节与自由度的配置关系进行分析，根据关节位置及自由度的不同，机器人腿部的关节被区分为髋关节、髌关节以及膝关节。



图 3-1 单腿关节配置图

Fig.3-1 Single-leg joint configuration diagram

为了求解轮腿四足机器人的整体腿部运动空间，在机器人中央位置建立机体坐标系 $\{O\}$ 。同时，为了方便计算，引入过渡坐标系 $\{O_k\}$ ($k=1, 2, 3, 4$)，分别对应轮腿四足

机器人每条腿的腿部基坐标系，整体机器人 D-H 坐标系的分布如图 3-2 所示。

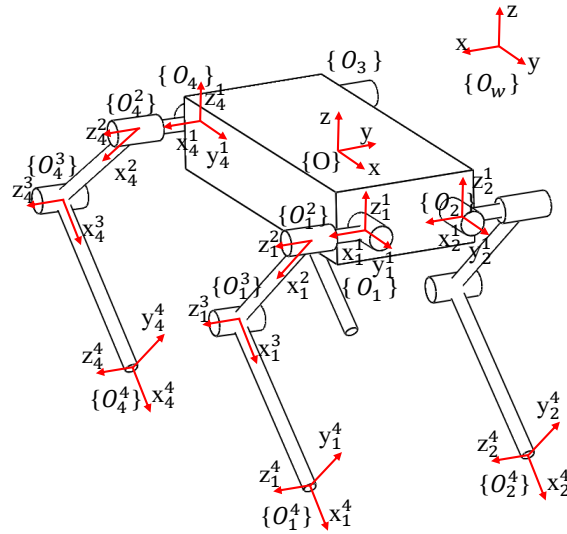


图 3-2 机器人 D-H 坐标分布

Fig.3-2 Robot D-H coordinate distribution

机体坐标系 $\{O\}$ 的 x 轴方向为机体前进方向， y 轴的方向由右手定则确定， z 轴方向与地面垂直。每条腿部支链的基关节坐标为 $\{O_k^1\}$ ($k=1, 2, 3, 4$)，以右前腿坐标系为例， $\{O_1^1\}$ 为髋关节坐标系， $\{O_1^2\}$ 为髌关节坐标系， $\{O_1^3\}$ 为膝关节坐标系， $\{O_1^4\}$ 为足末端与地面接触点的惯性坐标系。对于图 3-2 所示的机器人 D-H 坐标系，腿部的杆件参数可以分为四个独立的部分，具体定义如下：

关节转角 θ_i ：表示从 x_{i-1} 到 x_i 绕 z_i 旋转的角度；

杆件扭角 α_{i-1} ：表示从 z_{i-1} 到 z_i 绕 x_{i-1} 旋转的角度；

杆件长度 a_{i-1} ：表示从 z_{i-1} 到 z_i 沿 x_{i-1} 测量的距离；

关节距离 d_i ：表示从 x_{i-1} 到 x_i 沿 z_i 测量的距离。

机器人腿部相邻连杆的位置关系只有转动副， θ_i 是变量， d_i 为常数值。根据以上定义，得到机器人四条腿的 D-H 参数，如表 3-1 至表 3-4 所示。其中， d_2 的值为 134mm， a_2 和 a_3 都为 420mm。

表 3-1 右前腿 D-H 参数

Table3-1 Right front leg D-H parameters

连杆 i	扭转角 α_{i-1}	连杆长度 a_{i-1}	连杆偏置 d_i	关节变量 θ_i	变量范围
1	-90°	0	0	θ_1	$45^\circ \sim 135^\circ$
2	90°	0	d_2	θ_2	$-90^\circ \sim 90^\circ$
3	0	a_2	0	θ_3	$-180^\circ \sim 180^\circ$
4	0	a_3	0	0	

表 3-2 左前腿 D-H 参数

Table3-2 Left front leg D-H parameters

连杆 i	扭转角 α_{i-1}	连杆长度 a_{i-1}	连杆偏置 d_i	关节变量 θ_i	变量范围
1	-90°	0	0	θ_1	$45^\circ \sim 135^\circ$
2	90°	0	$-d_2$	θ_2	$-90^\circ \sim 90^\circ$
3	0	a_2	0	θ_3	$-180^\circ \sim 180^\circ$
4	0	a_3	0	0	

表 3-3 左后腿 D-H 参数

Table3-3 Left hind leg D-H parameters

连杆 i	扭转角 α_{i-1}	连杆长度 a_{i-1}	连杆偏置 d_i	关节变量 θ_i	变量范围
1	-90°	0	0	θ_1	$45^\circ \sim 135^\circ$
2	90°	0	$-d_2$	θ_2	$-90^\circ \sim 90^\circ$
3	0	a_2	0	θ_3	$-180^\circ \sim 180^\circ$
4	0	a_3	0	0	

表 3-4 右后腿 D-H 参数

Table3-4 Right hind leg D-H parameters

连杆 i	扭转角 α_{i-1}	连杆长度 a_{i-1}	连杆偏置 d_i	关节变量 θ_i	变量范围
1	-90°	0	0	θ_1	$45^\circ \sim 135^\circ$
2	90°	0	d_2	θ_2	$-90^\circ \sim 90^\circ$
3	0	a_2	0	θ_3	$-180^\circ \sim 180^\circ$
4	0	a_3	0	0	

分析上表可知，同侧腿的 DH 参数一样，所以只需分析不同侧的两条腿的运动学，这里选择右前腿和左前腿进行分析。忽略腿部编号简化后右前腿 D-H 坐标系分布如图 3-3 所示。根据 D-H 参数表可得矩阵表达式：

$${}^{i-1}_iT = Rot(x, \alpha_{i-1}) \cdot Trans(x, a_{i-1}) \cdot Rot(z, \theta_i) \cdot Trans(z, d_i)$$

$$Rot(x, \alpha_{i-1}) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c\alpha_{i-1} & -s\alpha_{i-1} & 0 \\ 0 & -s\alpha_{i-1} & c\alpha_{i-1} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$Trans(x, a_{i-1}) = \begin{bmatrix} 1 & 0 & 0 & a_{i-1} \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

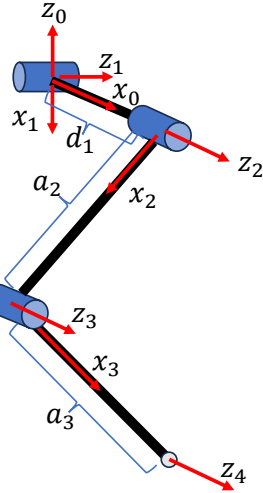


图 3-3 右前腿 D-H 坐标分布

Fig.3-3 Right front D-H coordinate distribution

$$\begin{aligned}
 Rot(z, \theta_i) &= \begin{bmatrix} c\theta_i & -s\theta_i & 0 & 0 \\ -s\theta_i & c\theta_i & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 Trans(z, d_i) &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}
 \end{aligned} \tag{3-1}$$

将上述各项带入式 (3-1)，得到通用变换矩阵为：

$${}^{i-1}_iT = \begin{bmatrix} c\theta_i & -s\theta_i & 0 & a_{i-1} \\ s\theta_i c\alpha_{i-1} & c\theta_i c\alpha_{i-1} & -s\alpha_{i-1} & -d_i s\alpha_{i-1} \\ s\theta_i s\alpha_{i-1} & c\theta_i s\alpha_{i-1} & c\alpha_{i-1} & d_i c\alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \tag{3-2}$$

其中： $c\theta_i = \cos\theta_i$ ， $s\theta_i = \sin\theta_i$ ， $c\alpha_{i-1} = \cos\alpha_{i-1}$ ， $s\alpha_{i-1} = \sin\alpha_{i-1}$ 。

将表 3-1 中右前腿数据带入式 (3-2) 可得变换矩阵：

$$\begin{aligned}
 {}^0_1T &= \begin{bmatrix} c\theta_1 & -s\theta_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -s\theta_1 & -c\theta_1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 {}^1_2T &= \begin{bmatrix} c\theta_2 & -s\theta_2 & 0 & 0 \\ 0 & 0 & -1 & -d_2 \\ s\theta_2 & c\theta_2 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
 \end{aligned}$$

$$\begin{aligned}
{}^2_3T &= \begin{bmatrix} c\theta_3 & -s\theta_3 & 0 & a_2 \\ s\theta_3 & c\theta_3 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
{}^3_4T &= \begin{bmatrix} 1 & 0 & 0 & a_3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
\end{aligned} \tag{3-3}$$

将上述变换矩阵带入式(3-3)可得足端相对腿部基关节坐标系的总变换矩阵:

$$\begin{aligned}
{}^0_4T &= {}^0_1T \cdot {}^1_2T \cdot {}^2_3T \cdot {}^3_4T \\
{}^0_4T &= \begin{bmatrix} n_x & o_x & a_x & x \\ n_y & o_y & a_y & y \\ n_z & o_z & a_z & z \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} & R & P \\ 0 & 0 & 0 & 1 \end{bmatrix}
\end{aligned} \tag{3-4}$$

其中:

$$n_x = c_1 c_{23}$$

$$n_y = s_{23}$$

$$n_z = -s_1 c_{23}$$

$$o_x = -c_1 s_{23}$$

$$o_y = c_{23}$$

$$o_z = s_1 s_{23}$$

$$a_x = s_1$$

$$a_y = 0$$

$$a_z = c_1$$

$$x = a_3 c_1 c_{23} + a_2 c_1 c_2 + d_2 s_1$$

$$y = a_3 s_{23} + a_2 s_2$$

$$z = -a_3 s_1 c_{23} - a_2 s_1 c_2 + d_2 c_1$$

式中: $s_i = \sin \theta_i$, $c_i = \cos \theta_i$, $s_{ij} = \sin(\theta_i + \theta_j)$, $c_{ij} = \cos(\theta_i + \theta_j)$ 。

根据式(3-4)可得到右前腿足端的姿态矩阵 R 和位置矩阵 P :

$$\begin{aligned}
R &= \begin{bmatrix} c_1 c_{23} & -c_1 s_{23} & s_1 \\ s_{23} & c_{23} & 0 \\ -s_1 c_{23} & s_1 s_{23} & c_1 \end{bmatrix} \\
P &= \begin{bmatrix} a_3 c_1 c_{23} + a_2 c_1 c_2 + d_2 s_1 \\ a_3 s_{23} + a_2 s_2 \\ -a_3 s_1 c_{23} - a_2 s_1 c_2 + d_2 c_1 \end{bmatrix}
\end{aligned} \tag{3-5}$$

在已知三个关节旋转角度 θ_1 、 θ_2 、 θ_3 的情况下，可以通过足端到腿部基关节坐标系的变换矩阵来确定足端的位姿。为了验证推导出的总变换矩阵的准确性，本文采用了 Matlab 仿真进行检验。首先使用 Matlab 的机器人工具箱构建了腿部机构的数学模型，并为关节角度选定了一组适当的值。接下来，分别运用 Matlab 和之前导出的总变换矩阵解算足端位姿，并将两种方法的结果进行对比以确定它们是否吻合。在仿真中，各关节的初始状态角度设置为 $\theta=(\pi/2, 0, 0)$ ，而验证的目标角度为 $\theta=(\pi/2, -\pi/6, \pi/3)$ ，如下图 3-4 所示。仿真结果显示足端位置坐标为(0.134, -0.727)，这与利用总变换矩阵计算得到的结果一致，从而证实了正运动学推导的准确性。

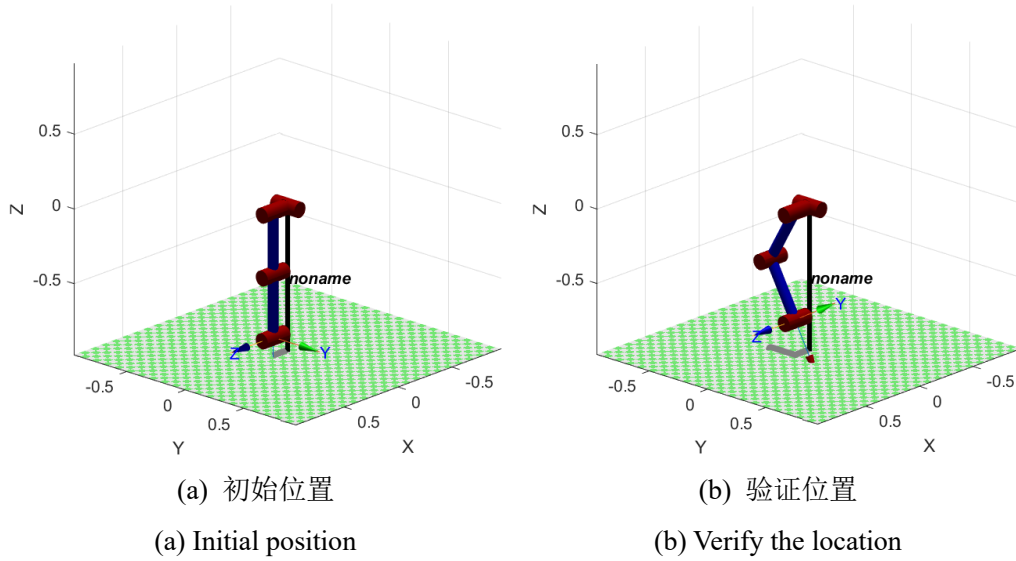


图 3-4 正运动学仿真

Fig.3-4 Positive kinematic simulation

同理可求得左前腿的正运动学总变换矩阵：

$${}^0_4T = \begin{bmatrix} n_x & o_x & a_x & x \\ n_y & o_y & a_y & y \\ n_z & o_z & a_z & z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3-6)$$

其中：

$$n_x = c_1 c_{23}$$

$$n_y = s_{23}$$

$$n_z = -s_1 c_{23}$$

$$o_x = -c_1 s_{23}$$

$$o_y = c_{23}$$

$$o_z = s_1 s_{23}$$

$$a_x = s_1$$

$$a_y = 0$$
$$a_z = c_1$$
$$x = a_3c_1c_{23} + a_2c_1c_2 - d_2s_1$$
$$y = a_3s_{23} + a_2s_2$$
$$z = -a_3s_1c_{23} - a_2s_1c_2 - d_2c_1$$

3.2.2 工作空间分析

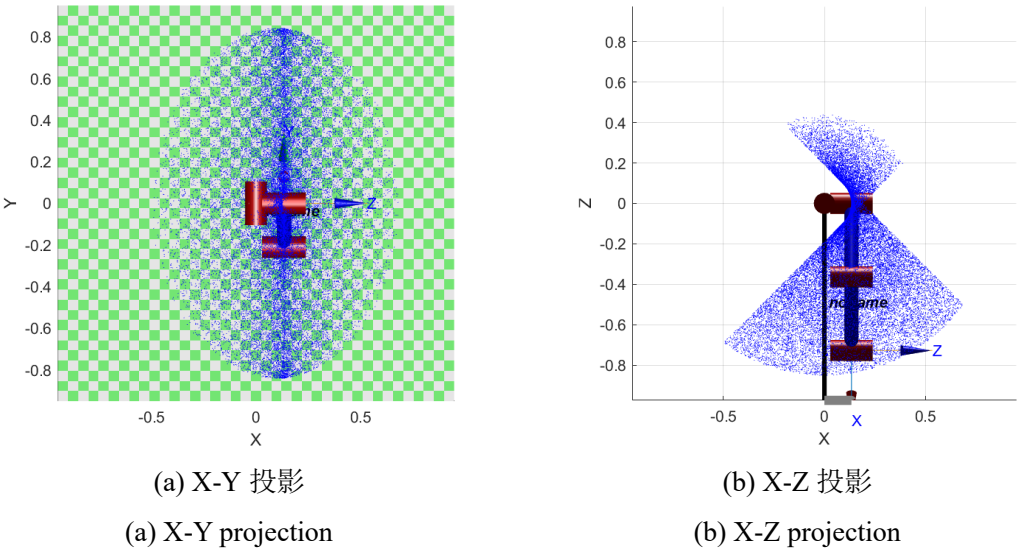
根据第 2 章机器人整体结构设计可知右前腿结构参数如下表 3-5 所示：

表 3-5 右前腿结构参数

Table3-1 Right front leg Structural parameters

参数名称	符号	数值	单位
胯关节偏置长度	d_2	134	mm
大腿长度	a_2	420	mm
小腿长度	a_3	420	mm
关节 1 角度范围	θ_1	[45, 135]	度($^{\circ}$)
关节 2 角度范围	θ_2	[-90, 90]	度($^{\circ}$)
关节 3 角度范围	θ_3	[-180, 180]	度($^{\circ}$)

将上表中的参数带入上面求得的正运动学方程，选取各关节转动范围内的任意角度可得到腿部足端所对应的坐标位置与姿态。在所有关节转角范围内的值随机取样，随机样本数为 10000 组，可得到如图所示的单腿工作空间。机器人移动速度的快慢和越障能力的强弱就取决于机器人腿部的工作空间范围的大小。分别在 XY 、 XZ 、 YZ 平面和三维空间内观察机器人腿部的运动范围，如图 3-5 所示。



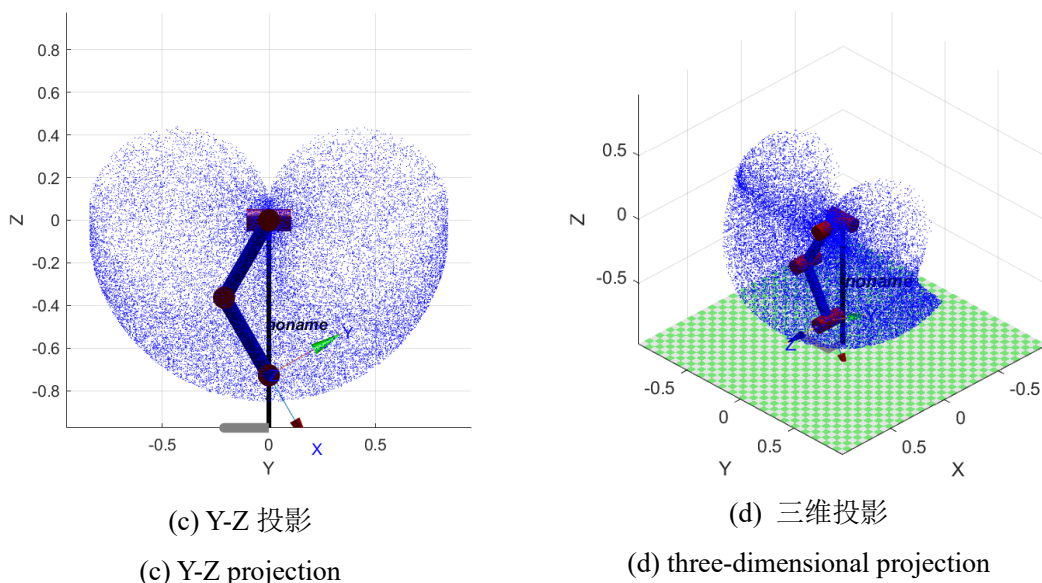


图 3-5 单腿工作空间

Fig.3-5 Single-leg workspace

其中机器人腿部的工作空间在 XY 平面的投影反映了机器人腿部的平面运动范围；机器人腿部的工作空间在 XZ 平面的投影反映了机器人腿部运动的上下限范围；机器人腿部的工作空间在 YZ 平面投影的 z 轴参数表示机器人可跨越障碍的最大高度， y 轴参数表示机器人可跨越的距离。

3.2.3 逆运动学分析

逆运动学解析是机器人运动学研究的一个重要分支，其核心任务是根据机器人末端执行器的预定位置及连杆的固定尺寸，反推出各个关节所需达到的角度。这一过程对于后续的轨迹规划和步态设计至关重要。

本文采用几何算法来求解其逆解，这里以右前腿为例求其运动学逆解，如下图 3-6 所示：

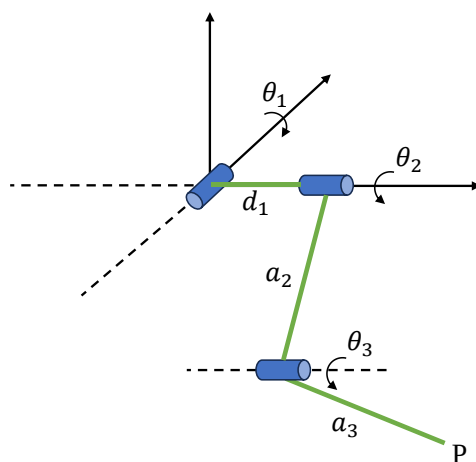


图 3-6 右前腿连杆结构

Fig.3-6 Right front leg linkage structure

(1) 求解胯关节角度, 如图 3-7 所示:

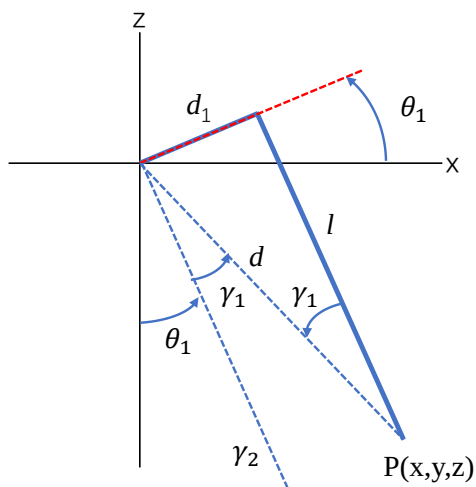


图 3-7 XZ 平面投影图

Fig.3-7 XZ planar projection map

$$\begin{aligned} d &= \sqrt{x^2 + z^2} \\ l &= \sqrt{d^2 - d_1^2} \\ \gamma_1 &= \arctan \frac{d_1}{l} \\ \gamma_2 &= -\arctan \frac{x}{z} \\ \theta_1 &= \gamma_2 - \gamma_1 \end{aligned} \quad (3-7)$$

(2) 求解膝关节角度, 如图 3-8 所示:

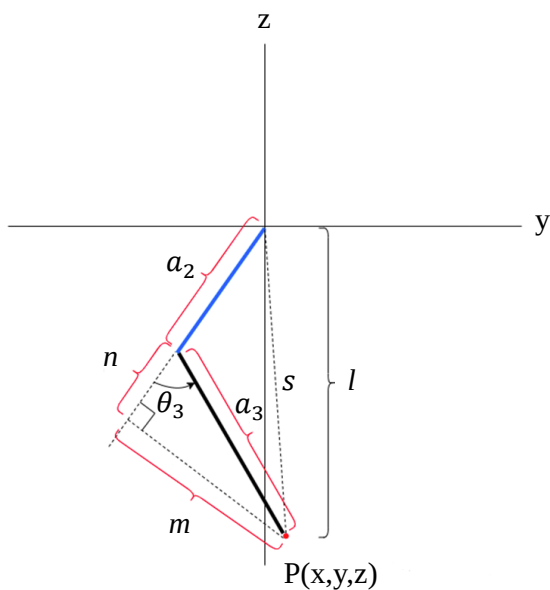


图 3-8 腿部连杆平面投影图

Fig.3-8 Planar projection of the leg linkage

$$s = \sqrt{l^2 + y^2}$$

$$(a_2 + n)^2 + m^2 = s^2 \quad (3-8)$$

$$n^2 + m^2 = a_3^2 \quad (3-9)$$

式 (3-8) 减去式 (3-9) 得:

$$\begin{aligned} [(a_2 + n)^2 + m^2] - (n^2 + m^2) &= s^2 - a_3^2 \\ 2 \cdot a_2 \cdot n + a_2^2 &= s^2 - a_3^2 \\ n &= \frac{s^2 - a_3^2 - a_2^2}{2a_2} \\ \theta_3 &= -\arccos \frac{n}{a_3} \end{aligned} \quad (3-10)$$

(3) 求解髋关节角度, 如图 3-9 所示:

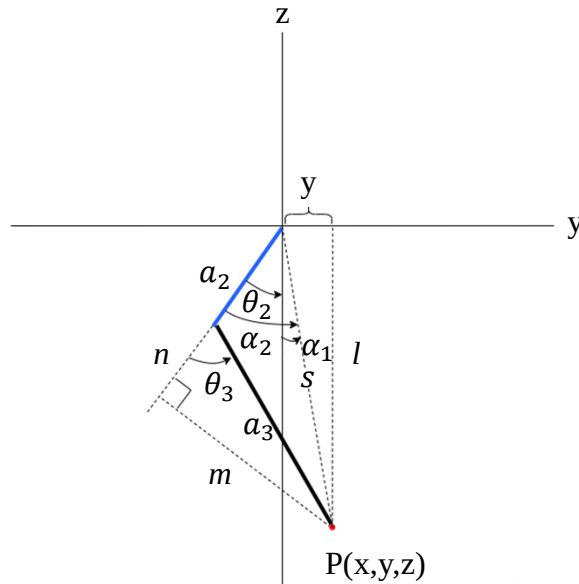


图 3-9 腿部连杆平面投影图

Fig.3-9 Planar projection of the leg linkage

$$\begin{aligned} \alpha_1 &= -\arctan \frac{y}{l} \\ \alpha_2 &= \arccos \frac{a_2 + n}{s} \\ \theta_2 &= \alpha_1 + \alpha_2 \end{aligned} \quad (3-11)$$

3.3 单腿雅可比矩阵分析

雅可比矩阵由一个向量的各分量对另一向量各分量的偏导数构成。在机器人技术领域, 雅可比矩阵描述的是机器人末端执行器的位姿 (包括位置和姿态) 相对于各关节位

置参数的偏导数关系。

对于右前腿总变换矩阵可得：

$$\begin{aligned}x &= a_3 c_1 c_{23} + a_2 c_1 c_2 + d_2 s_1 \\y &= a_3 s_{23} + a_2 s_2 \\z &= -a_3 s_1 c_{23} - a_2 s_1 c_2 + d_2 c_1\end{aligned}\quad (3-12)$$

即：

$$\begin{aligned}x &= x(\theta_1, \theta_2, \theta_3) \\y &= y(\theta_1, \theta_2, \theta_3) \\z &= z(\theta_1, \theta_2, \theta_3)\end{aligned}\quad (3-13)$$

对上式进行微分得：

$$\begin{aligned}dx &= \frac{\partial x}{\partial \theta_1} d\theta_1 + \frac{\partial x}{\partial \theta_2} d\theta_2 + \frac{\partial x}{\partial \theta_3} d\theta_3 \\dy &= \frac{\partial y}{\partial \theta_1} d\theta_1 + \frac{\partial y}{\partial \theta_2} d\theta_2 + \frac{\partial y}{\partial \theta_3} d\theta_3 \\dz &= \frac{\partial z}{\partial \theta_1} d\theta_1 + \frac{\partial z}{\partial \theta_2} d\theta_2 + \frac{\partial z}{\partial \theta_3} d\theta_3\end{aligned}\quad (3-14)$$

即：

$$\begin{bmatrix} dx \\ dy \\ dz \end{bmatrix} = \begin{bmatrix} \frac{\partial x}{\partial \theta_1} & \frac{\partial x}{\partial \theta_2} & \frac{\partial x}{\partial \theta_3} \\ \frac{\partial y}{\partial \theta_1} & \frac{\partial y}{\partial \theta_2} & \frac{\partial y}{\partial \theta_3} \\ \frac{\partial z}{\partial \theta_1} & \frac{\partial z}{\partial \theta_2} & \frac{\partial z}{\partial \theta_3} \end{bmatrix} \begin{bmatrix} d\theta_1 \\ d\theta_2 \\ d\theta_3 \end{bmatrix}\quad (3-15)$$

可写成：

$$dX = Jd\theta\quad (3-16)$$

其中： J 为右前腿的雅可比矩阵。通过计算可得：

$$J = \begin{bmatrix} -a_3 s_1 c_{23} - a_2 s_1 c_2 + d_2 c_1 & -a_3 c_1 s_{23} - a_2 c_1 s_2 & -a_3 c_1 s_{23} \\ 0 & a_3 c_{23} + a_2 c_2 & a_3 c_{23} \\ -a_3 c_1 c_{23} - a_2 c_1 c_2 - d_2 s_1 & a_3 s_1 s_{23} + a_2 s_1 s_2 & a_3 s_1 s_{23} \end{bmatrix}\quad (3-17)$$

静态条件下，关节扭矩与足端承受的力达到平衡状态。当位移极微小时，可以应用虚功原理来表征这一静态平衡，即在笛卡尔坐标空间中力所做的功应与在关节空间中由力矩所做的功相等，有：

$$F \cdot \delta_x = \tau \cdot \delta_\theta\quad (3-18)$$

式中 F 为作用在足端的作用力矢量, δ_x 为足端无穷小的位移矢量, τ 为各关节力矩矢量, δ_θ 为各关节位移矢量。

上式也可以写成:

$$F^T \delta_x = \tau^T \delta_\theta \quad (3-19)$$

雅可比矩阵的定义为:

$$\delta_x = J \delta_\theta \quad (3-20)$$

因此, 可得:

$$F^T J \delta_\theta = \tau^T \delta_\theta \quad (3-21)$$

简化后:

$$F^T J = \tau^T \quad (3-22)$$

对上式两边转置, 可得:

$$\tau = J^T F \quad (3-23)$$

分析上式可知: 足端作用力可以通过雅可比矩阵的转置转化为等效的关节扭矩, 这一原理为本文后续章节中的运动控制算法提供了理论基础。

3.4 本章小结

基于第二章设计的机器人整体结构尺寸, 本章利用 D-H 参数法建立机器人腿部运动学模型, 对机器人四肢进行正逆运动学分析、工作空间的求解及雅可比矩阵的计算, 为后续的运动控制算法研究提供了理论基础。

第4章 运动控制系统及控制方法

4.1 引言

机器人控制系统是整个机器人系统的核心，它承担着对机器人各个组件的协调和控制任务。设计一套高效、稳定的机器人控制系统对于实现机器人在各种环境中的复杂任务至关重要。控制系统的优越性直接影响到机器人的性能，包括运动的精准度、响应速度以及对外部环境的适应能力。本章主要对机器人上层控制系统、关节电机控制、机器人整体运动控制三个方面进行设计。

4.2 控制系统

机器人传感器包括 12 路关节位置传感器、4 路电推杆位置传感器、4 路足端力传感器、IMU 和 GPS 等，底层采用 CAN 总线通信技术。图 4-1 所示为机器人运动控制系统框图。

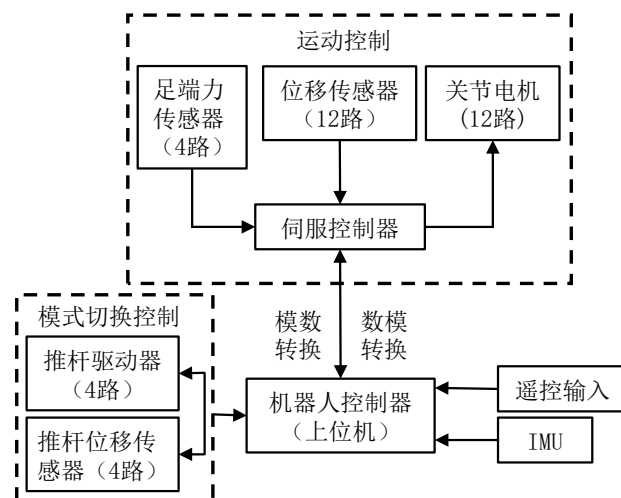


图 4-1 运动控制系统框图

Fig.4-1 Block diagram of a motion control system

4.2.1 上位机系统

上位机使用 ROS 机器人控制系统。ROS 是一款开源、灵活的机器人软件平台，旨在为机器人开发提供通用的操作系统功能。其设计理念通过提供一系列工具、库和约定，使机器人软件开发变得更加简便、模块化和可重用。ROS 最初由加利福尼亚大学圣迭戈分校（UCSD）的机器人实验室（WillowGarage）于 2007 年发起，并目前由 OpenRobotics 维护和支持。

ROS 具有以下几个显著特点：

(1) 模块化架构: ROS 采用分布式、模块化的设计, 允许开发者使用独立的软件包 (packages) 来实现不同的机器人功能。这种模块化的方式使得各个组件能够独立开发、测试和部署, 提高了软件的可维护性和可重用性。

(2) 通用性: ROS 是一个通用性的平台, 适用于各种类型的机器人, 包括移动机器人、飞行器和工业机器人等。这使得开发者能够在不同类型的机器人项目之间共享代码和经验。

(3) 通信机制: ROS 提供了强大的进程间通信机制, 允许不同模块之间进行数据交换和信息共享。这是通过 ROS 的发布-订阅 (Publish-Subscribe) 模型和服务调用 (ServiceCall) 机制实现的, 使得不同组件能够在运行时动态连接。

(4) 工具支持: ROS 提供了一系列的开发和调试工具, 包括可视化工具 (如 Rviz)、数据记录工具 (如 rosbag), 以及调试工具 (如 rqt)。这些工具帮助开发者更轻松地监控和调试机器人系统。

(5) 开放源代码: ROS 的源代码完全开放, 并遵循开源协议, 这使得任何人都能够查看、修改和分享 ROS 的代码。这种开放性促进了全球机器人社区的合作和创新。

ROS 还提供了各种传感器的驱动程序和接口, 包括摄像头、激光雷达、惯性测量单元 (IMU) 等, 使得与这些传感器的集成变得更加简单。通过 ROS, 可以轻松地控制和管理机器人的执行器, 例如电机、执行臂和夹持器等。ROS 与 Gazebo 集成紧密, Gazebo 是一个功能强大的机器人仿真器, 允许开发者在虚拟环境中测试和验证机器人的行为和算法, 从而减少实际机器人测试所需的时间和成本。此外, ROS 提供了用于机器人导航的导航栈, 包括路径规划、障碍物避障和定位等功能, 使得机器人能够在不同环境中进行自主移动。

ROS 已经成为机器人领域中最流行的开发平台之一, 被广泛用于学术研究、工业应用和个人项目。其生态系统不断壮大, 为机器人开发者提供了丰富的资源和支持, 推动了机器人技术的不断进步。

4.2.2 关节电机

常见的电机种类包括直流电机、交流电机、步进电机, 其中, 直流电机又分为有刷直流电机, 通过刷子与转子上的换向器接触来实现换向, 结构简单但容易产生摩擦和磨损, 以及无刷直流电机, 通过电子换向器实现换向, 无刷结构降低了摩擦和磨损, 提高了效率和使用寿命; 交流电机分为同步电机和异步电机, 同步电机通常用于需要恒速运转的场合, 异步电机常见于家用电器和工业设备, 结构简单、可靠但启动和控制相对复杂; 步进电机分为单相、双相和多相步进电机, 具有精确的位置控制能力, 适合需要准确定位的应用场景。

对于四足机器人而言, 需要考虑到控制精度、功率密度、响应速度和可靠性等因素。无刷直流电机在这些方面表现优秀, 具有高效性能、精准控制、高转矩密度和低维护成

本等优点，更适合作为四足机器人的关节电机。

无刷直流电机（BLDC）是一种同步电机，采用直流电源进行驱动。无刷直流电机的工作原理基于逆变器将直流电源转换为三相交流电源，并配备位置反馈的永磁同步电机。这种电机结合了直流电机和交流电机的优点，如精准调节性能、结构简单、无需换向电刷、运行可靠且易于维护等特点。根据定子位置的不同，无刷直流电机可分为外转子和内转子两种类型，如下图 4-2 所示。

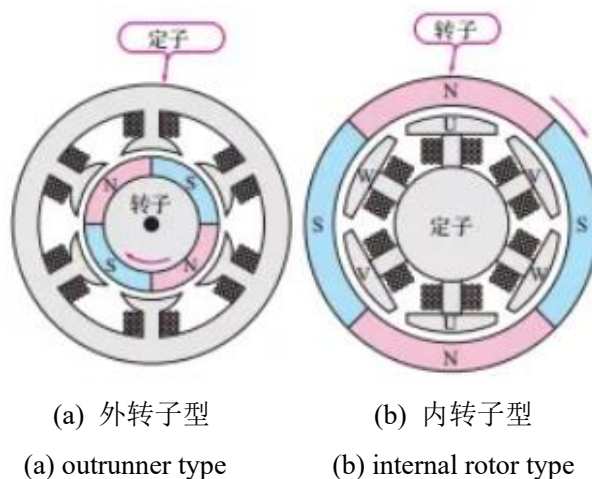


图 4-2 无刷直流电机

Fig.4-2 BLDC

外转子电机的磁钢片被制成片状并贴在外壳上，转子为永磁磁钢，与外壳一同连接到输出轴上。定子则是线圈绕组，在电机运行时，整个外壳转动，而中间的线圈定子则保持静止。

4.2.3 电机控制方法

Field-Oriented Control（FOC），也称为矢量控制，是一种利用变频器（VFD）技术来精确控制三相电机的方法^[60]。FOC 的核心在于调整电机的电磁场方向，使定子磁场向量与转子磁场向量垂直，从而产生旋转力矩。通过调整三相输入电流的大小和方向，维持定子磁场与转子磁场的垂直关系，最大化电机转矩。FOC 引入了空间电流矢量的概念来控制输入电流方向，将三相电流矢量分解为垂直和平行于转子磁轴的两个分量，垂直方向的电流分量会形成与转子磁场正交的磁场，产生所需的旋转力矩；而平行于转子磁轴的电流分量则只会引起额外热量并增加轴承磨损，而不产生力矩。FOC 算法将平行于转子磁轴的电流分量最小化，通过调节线圈电流使垂直于转子磁轴的电流分量最大化，从而获得与电流分量成正比的电机力矩。此外，实时获取转子位置信息对于实现有效控制至关重要。确定转子位置的方法分为有传感器和无传感器两种情形。当存在位置传感器时，可以直接反馈转子位置信息，从而简化控制过程。而当没有位置传感器时，需要使用位置估算算法通过采集电机相电流来计算转子位置。这种方法不仅保证了系统稳定

性，同时也降低了电机系统的复杂度。

如图 4-3 所示，以转子磁铁为参考，建立 d-q 坐标系，该坐标系随转子旋转。在这个坐标系下，当设定电流 I_d 为 0， I_q 为 1 时，转子将逆时针旋转，且转速随 I_q 的增大而增大。另一方面，设置电流 I_d 为 1， I_q 为 0 时，转子将保持静止，但发热量会随着 I_d 的增大而增大。为了最小化定子电流矢量 I_d ，需要实时调整定子线圈的弦波电流，使其随着转子转动角度的相位变化，确保电流在 d-q 坐标系下能够按照所需方式进行调节，从而实现精准的矢量控制。

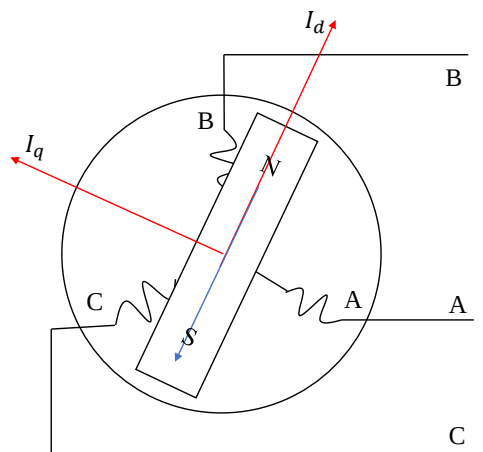


图 4-3 FOC 控制原理图

Fig.4-3 FOC control schematic

电机电流转换为 d-q 结构需要用到两个工具，Clark 变换和 Park 变换。

(1) Clark 变换

如图 4-4 所示，对电机的三相电流进行采样，得到三个相位差为 120° 的正弦波。将三相电流抽象到二维平面内，得到三个基向量 I_a 、 I_b 、 I_c ，这三个基向量是线性相关的，可以使用两个正交的基向量表示，因此，将三个基向量 I_a 、 I_b 、 I_c 投影到二维平面坐标轴上后可得到两个正交基向量 I_α 、 I_β 。这样就将三个控制量变为一个。

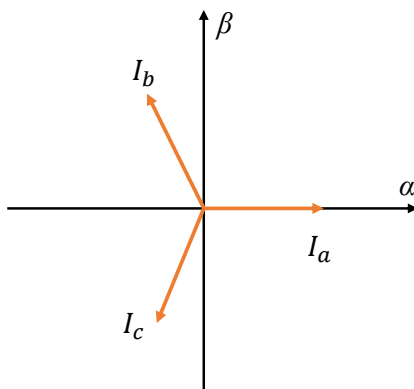


图 4-4 Clark 变换

Fig.4-4 Clark transform

变换矩阵为:

$$\begin{bmatrix} I_\alpha \\ I_\beta \end{bmatrix} = \begin{bmatrix} 1 & -\frac{1}{2} & -\frac{1}{2} \\ 0 & \frac{\sqrt{3}}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} I_a \\ I_b \\ I_c \end{bmatrix} \quad (4-1)$$

(2) Park 变换

Park 变换是将静止坐标系 α - β 变换到同步旋转坐标系 d-q, 如图 4-5 所示。

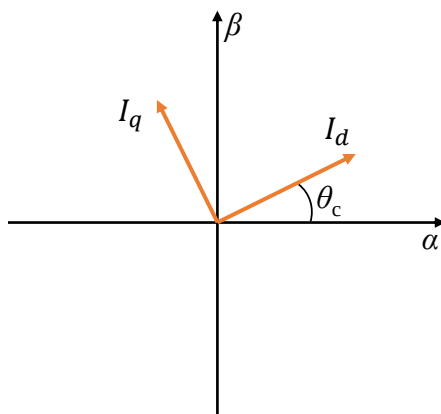


图 4-5 Park 变换

Fig.4-5 Park transform

变换矩阵为:

$$\begin{bmatrix} I_d \\ I_q \end{bmatrix} = \begin{bmatrix} \cos\theta_c & \sin\theta_c \\ -\sin\theta_c & \cos\theta_c \end{bmatrix} \begin{bmatrix} I_\alpha \\ I_\beta \end{bmatrix} \quad (4-2)$$

一旦电机电流转换为 d-q 结构, 控制过程变得更为简单且灵活。其中, 两个 P-I 控制器分别用于平行于转子磁场的电流和垂直电流的精确控制。通过使平行转子磁场电流控制信号为零, 可以实现电机电流矢量完全转化为垂直电流, 最大化电机效率。另一个 P-I 控制器则专注于调控垂直电流, 以实现所需的力矩输出。为了让绕组产生的磁场方向等效为 I_d 、 I_q , 需要引入 Park 反变换和 Clark 反变换。

将 Park 变换矩阵求逆即可得到 Park 反变换矩阵, 通过 Park 反变换矩阵可将 d-q 坐标系变换到 α - β 坐标系。变换矩阵为:

$$\begin{bmatrix} I_\alpha \\ I_\beta \end{bmatrix} = \begin{bmatrix} \cos\theta_c & -\sin\theta_c \\ \sin\theta_c & \cos\theta_c \end{bmatrix} \begin{bmatrix} I_d \\ I_q \end{bmatrix} \quad (4-3)$$

将 Clark 变换矩阵求逆即可得到 Clark 反变换矩阵, Clark 反变换的目的是把两个垂直的坐标系转变为三相绕组坐标系。变换矩阵为:

$$\begin{bmatrix} I_a \\ I_b \\ I_c \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -\frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{1}{2} & -\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} I_\alpha \\ I_\beta \end{bmatrix} \quad (4-4)$$

图 4-6 所示为 FOC 控制流程图：

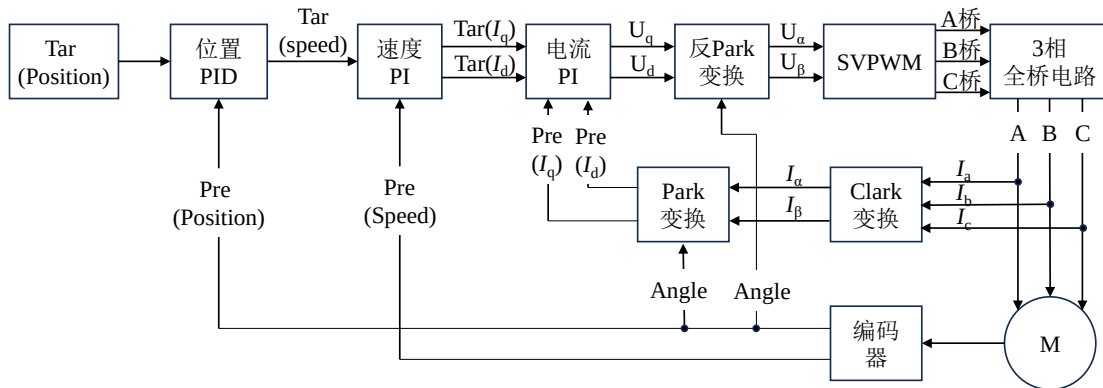


图 4-6 FOC 控制流程图

Fig.4-6 FOC control flow chart

FOC 控制的流程如下：

- 1) 采集三相电流 I_a 、 I_b 、 I_c 。
- 2) 进行 Clark 变换，得到正交轴电流 I_α 、 I_β 。
- 3) 进行 Park 变换，获得旋转正交电流 I_d 和 I_q 。将 I_d 设为 0，使得这两个量成为恒定值。
- 4) 对 I_q 和 I_d 分别经过 PI 调节器，得到 U_q 和 U_d 输出。
- 5) 利用传感器获取电机转动角度。
- 6) 进行逆 Park 变换，得到二轴电压。
- 7) 对逆 Park 变换得到的 U_α 和 U_β 进行逆 Clark 变换，得到最终需要的三相电压信号，以驱动逆变器桥，完成电机控制。

4.3 机器人整机运动控制方案

4.3.1 四足模式控制方案

腿足构型下采用 VMC 虚拟模型控制方法^[61,62]、trot 步态，实现了四足机器人的全方向稳定运动，虽然 VMC 方法并没有很高的控制精度，但已经可以应对大多数任务要求，具有简单、高效的优点，以及基于 4 个足底力传感器和 IMU 传感器实现地形感知和自适应功能，可感知机器人所处地形的坡度自动调整自身姿态，增强机器人运动平稳性。

机器人在运动中具有6个自由度,沿自身坐标系的3个坐标轴的平动和绕三个轴的转动。为对这6个自由度的控制,需在机器人质心处对每个自由度都施加虚拟构件。如图4-7所示, $\{B\}$ 为机器人质心坐标系, $\{O\}$ 为世界坐标系, $\{H_1\}$ 、 $\{H_2\}$ 、 $\{H_3\}$ 和 $\{H_4\}$ 为机器人四条腿对应的腿部基坐标系。对于3个平动自由度,沿 x 轴和 y 轴需要控制的量为速度值, $\dot{x}$ 和 $\dot{y}$, 所以只需施加虚拟阻尼构件, 而沿 z 轴需要控制位移量 z , 需要施加弹簧-阻尼构件。对于3个转动自由度, 都需要控制位置量, 绕 x 轴的俯仰角 α , 绕 y 轴的横滚角 β , 绕 z 轴的偏航角 γ , 所以绕每个轴都需要施加弹簧-阻尼构件。为简化控制, 将3个平动自由度分解到机器人支撑腿上, 单独控制每条腿。

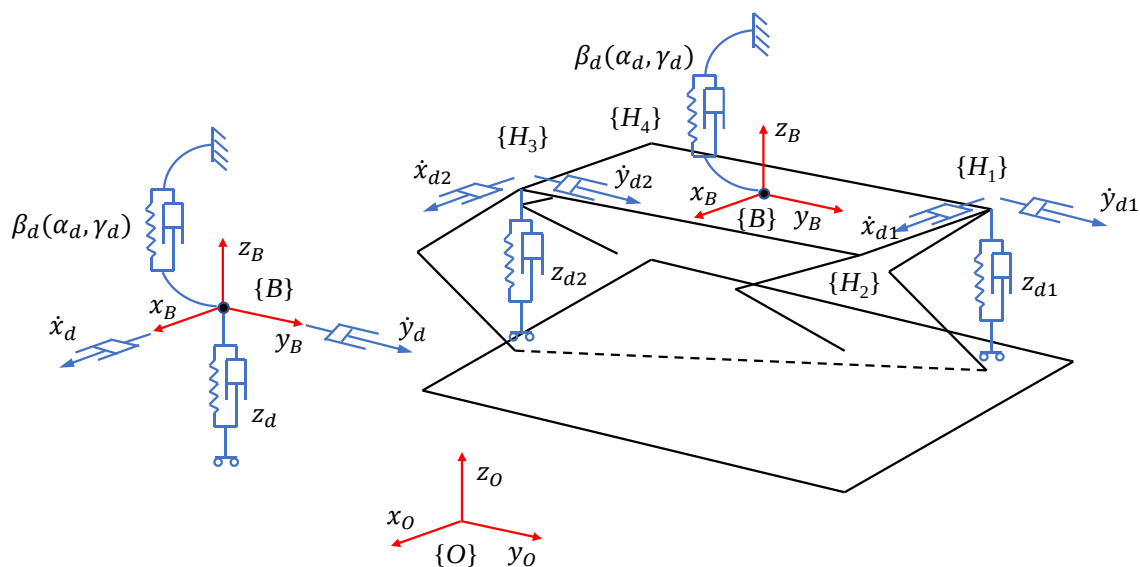


图 4-7 机器人虚拟构件

Fig.4-7 Robot virtual building blocks

虚拟构建弹簧-阻尼器的本构方程为:

$$F_u = K_u (u_d - u) + B_u (\dot{u}_d - \dot{u}) \quad (4-5)$$

其中, F_u 为达到期望值虚拟构件所产生的虚拟力, K_u 和 B_u 为弹性和阻尼系数, u_d 和 $\dot{u}_d$ 为期望位移和速度, u 和 $\dot{u}$ 为实时位移和速度。

当机器人运动时, 支撑腿对应的膝关节坐标系的期望速度与质心的期望速度相等, 支撑腿 z 方向期望位移与质心期望高度相等, 期望速度为 0。即:

$$\begin{bmatrix} \dot{x}_{di} \\ \dot{y}_{di} \\ z_{di} \end{bmatrix} = \begin{bmatrix} \dot{x}_d \\ \dot{y}_d \\ z_d \end{bmatrix} \quad (4-6)$$

(式中下标 i 为腿代号)

则, 为实现机体平动控制, 支撑腿需要产生的虚拟力为:

$$\begin{bmatrix} f_{xi} \\ f_{yi} \\ f_{zi} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & K_z \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ z_{di} - z_i \end{bmatrix} + \begin{bmatrix} B_x & 0 & 0 \\ 0 & B_y & 0 \\ 0 & 0 & B_z \end{bmatrix} \begin{bmatrix} \dot{x}_{di} - \dot{x}_i \\ \dot{y}_{di} - \dot{y}_i \\ 0 - \dot{z}_i \end{bmatrix} \quad (4-7)$$

设机器人期望姿态角为 α_d 、 β_d 和 γ_d ，期望角速度为 $\dot{\alpha}_d$ 、 $\dot{\beta}_d$ 和 $\dot{\gamma}_d$ ，为控制机器人姿态所需的虚拟转矩为：

$$\begin{bmatrix} M_\alpha \\ M_\beta \\ M_\gamma \end{bmatrix} = \begin{bmatrix} K_\alpha & 0 & 0 \\ 0 & K_\beta & 0 \\ 0 & 0 & K_\gamma \end{bmatrix} \begin{bmatrix} \alpha_d - \alpha \\ \beta_d - \beta \\ \gamma_d - \gamma \end{bmatrix} + \begin{bmatrix} B_\alpha & 0 & 0 \\ 0 & B_\beta & 0 \\ 0 & 0 & B_\gamma \end{bmatrix} \begin{bmatrix} \dot{\alpha}_d - \dot{\alpha} \\ \dot{\beta}_d - \dot{\beta} \\ \dot{\gamma}_d - \dot{\gamma} \end{bmatrix} \quad (4-8)$$

控制机器人俯仰角 α 的虚拟转矩 M_α 可由前后支撑腿 z 轴虚拟力产生，设机器人前后胯关节间的距离为 L ：

$$f_{z,\alpha} = \frac{M_\alpha}{L} \quad (4-9)$$

控制机器人横滚角 β 的虚拟转矩 M_β 可由左右支撑腿 z 轴虚拟力产生，设机器人左右胯关节间的距离为 W ：

$$f_{z,\beta} = \frac{M_\beta}{W} \quad (4-10)$$

控制机器人偏航角 γ 的虚拟转矩 M_γ 可由前后支撑腿 x 轴虚拟力产生：

$$f_{x,\gamma} = \frac{M_\gamma}{L} \quad (4-11)$$

最终支撑腿所需产生的虚拟力为：

$$\begin{bmatrix} F_{xi} \\ F_{yi} \\ F_{zi} \end{bmatrix} = \begin{bmatrix} f_{xi} \pm f_{x,\gamma} \\ f_{yi} \\ f_{zi} \pm f_{z,\alpha} \pm f_{z,\beta} \end{bmatrix} \quad (4-12)$$

上式中，若支撑腿 i 为前腿，则 $+f_{z,\alpha}$ 、 $-f_{x,\gamma}$ ，后腿反之；若支撑腿 i 为左腿，则 $+f_{z,\beta}$ ，右腿反之。

再利用对应腿的雅可比矩阵 J_i 将虚拟力 $[F_{xi} \ F_{yi} \ F_{zi}]^T$ 映射为关节期望力矩 $[\tau_{1,i} \ \tau_{2,i} \ \tau_{3,i}]^T$ ，得：

$$\begin{bmatrix} \tau_{1,i} \\ \tau_{2,i} \\ \tau_{3,i} \end{bmatrix} = J_i^T \begin{bmatrix} F_{xi} \\ F_{yi} \\ F_{zi} \end{bmatrix} \quad (4-13)$$

由于机器人俯仰和横滚姿态实际由机器人支撑腿的高度控制，上述单纯的力的叠加对参数的要求较高，所以下面给出力位混合控制方法，降低调参的难度。

图 4-8 为机器人坡度感知原理简图, $\{H_2\}$ 和 $\{H_3\}$ 为前后腿膝关节坐标系, P_2 、 P_3 分别为前后腿足端相对自身膝关节坐标系的位置坐标, α 为 IMU 传感器测得的相对世界坐标系的俯仰角, θ_1 为当前斜坡相对于机器人俯仰方向的坡度, φ 为前后腿足端所在面与机器人质心坐标系的角速度。

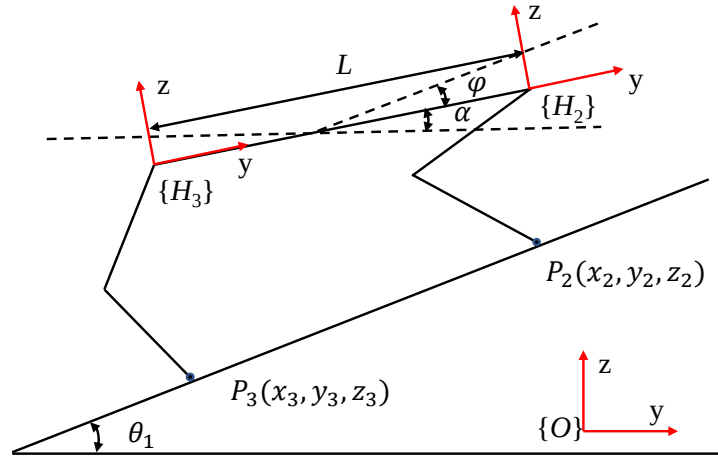


图 4-8 机器人坡度感知原理

Fig.4-8 Principle of Robot Slope Perception

P_2 、 P_3 可由机器人单腿正运动学求出, φ 可由 P_2 、 P_3 点坐标求出:

$$\varphi = \tan^{-1} \left(\frac{z_2 - z_3}{y_2 + L - y_3} \right) \quad (4-14)$$

则:

$$\theta_1 = \alpha + \varphi \quad (4-15)$$

将上式中前后腿足端坐标替换为左右腿足端坐标, L 替换为左右髋关节距离 W 后, 同理可得当前斜坡相对于机器人横滚方向的坡度 θ_2 。

再根据期望姿态角 α_d 和 β_d 以及计算所得的坡度 θ_1 和 θ_2 计算出每条腿的期望高度。如图 4-9 所示, 为增加机器人在斜坡上运动的稳定性, 可让机器人起始状态的足端位置在其对应的膝关节正下方, 使得机器人质心向下的投影始终处于落足区域的中心位置。支撑腿只调整 Z 方向的位置补偿, X 与 Y 方向的位置补偿由摆动腿调整。

机器人俯仰方向的坡度补偿值

$$dh_1 = \tan \varphi \frac{L}{2} = \tan(\theta_1 - \alpha_d) \frac{L}{2} \quad (4-16)$$

同理可得机器人横滚方向的坡度补偿值

$$dh_2 = \tan \varphi \frac{W}{2} = \tan(\theta_2 - \beta_d) \frac{W}{2} \quad (4-17)$$

在实际任务中一般取期望姿态角 $\alpha_d = 0$ 和 $\beta_d = 0$, 则:

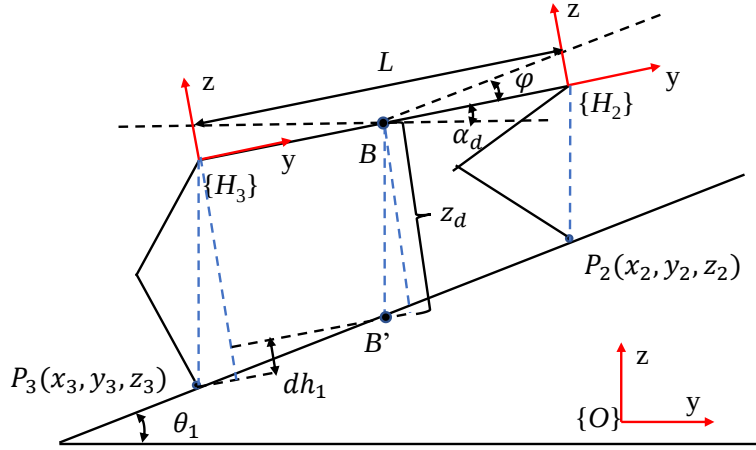


图 4-9 机器人坡度自适应原理

Fig.4-9 Robot slope adaptation principle

$$\begin{aligned} dh_1 &= \frac{L}{2} \tan \theta_1 \\ dh_2 &= \frac{W}{2} \tan \theta_2 \end{aligned} \quad (4-18)$$

或 $\alpha_d = \theta_1$ 和 $\beta_d = \theta_2$ ，此时 $dh_1 = dh_2 = 0$ 。

将式(4-6)改为如下形式：

$$\begin{bmatrix} \dot{x}_{di} \\ \dot{y}_{di} \\ z_{di} \end{bmatrix} = \begin{bmatrix} \dot{x}_d \\ \dot{y}_d \\ z_d \pm dh_1 \pm dh_2 \end{bmatrix} \quad (4-19)$$

上式中，因 z_{di} 在负轴上，所以若 i 为前腿，则 $-dh_1$ ，反之则 $+dh_1$ ；同理若 i 为后腿，则 $-dh_2$ ，反之则 $+dh_2$ ，后面计算过程不变。如此，相当于先使用位控方式控制机器人的姿态，再使用 VMC 方法进行微调，大大降低了参数调节的难度。

对于摆动腿，先对足端进行轨迹规划，再使用 VMC 方法进行轨迹跟踪，这里加入几个控制策略来增加运动稳定性。

步幅调整策略，根据给定的期望速度实时调整步幅大小。

$$d_k = \frac{\dot{y}_d}{p} \quad (4-20)$$

上式中 $\dot{y}_d$ 为期望速度， p 为步频， d_k 为步幅大小。一般设置步频 p 为固定值，则当期望速度 $\dot{y}_d$ 变化时，步幅 d_k 也会随之变化，保证落地转变为支撑腿后的运动轨迹在合适的范围内。

足端轨迹延伸策略，在有高度落差的地形上继续延伸摆动腿轨迹寻找地面。

触地保持策略，在复杂地形上，两条摆动腿不一定同时落地，若再继续按设计的轨迹运动将破坏机器人的平衡，所以此时先记录落地腿触地时的足端位置，停止后继的摆

动轨迹跟踪,随支撑腿一起运动,等到另一条腿也落地后再进行摆动腿与支撑腿的切换。以及与支撑腿共同作用的斜坡补偿策略。

摆动腿足端轨迹如图 4-10 所示,最后形成的摆动腿轨迹方程为:

$$\begin{cases} y_i = -\frac{d_k}{2} + \dot{y}_d t - z_{di} \cdot \tan \alpha \\ z_i = z_{di} + h - \frac{4h}{d_k^2} y_i^2 \end{cases} \quad (4-21)$$

上式中 i 为腿代号, t 是以切换为摆动腿那一刻为开始的时间, z_{di} 为转换为摆动腿之前的 z 轴期望坐标, α 为 IMU 测得的俯仰角, h 为抬腿高度。根据运动特性同理可求得横移与原地转向的摆动腿轨迹方程。

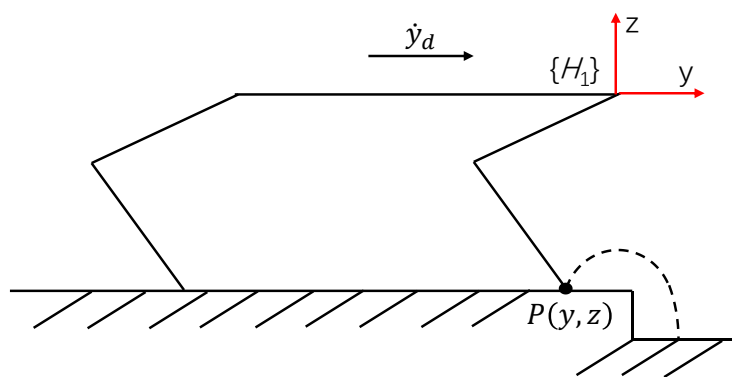


图 4-10 摆动腿轨迹

Fig.4-10 Swing the leg trajectory

4.3.2 机器人步态规划

四足机器人作为一种模仿生物四肢运动的高级机器人,其设计和运动控制的灵活性赋予了它们在多种地形中行走的能力。这些机器人可以通过模拟自然界中四足动物的步态来实现高效的移动,其中包括步行、小跑(Trot)、疾跑(Gallop)等多种步态。每种步态都有其独特的运动特征,适用于不同的速度范围和地形条件,因此选择合适的步态对于四足机器人的性能至关重要。

Trot 小跑步态是一种对角线腿对成对移动的步态,如图 4-11 所示,即机器人的左前腿和右后腿同时抬起并向前移动,然后是右前腿和左后腿的组合。这种步态的一个显著特点是其高效的速度与能耗平衡^[63]。这些优点使得 Trot 步态成为最常用的四足步态^[64]。

优势一:平衡性与稳定性的提升

Trot 步态在四足机器人中具有独特的平衡性和稳定性优势。其核心机制是通过左前腿和右后腿、右前腿和左后腿的交替运动,使机器人在行进过程中始终保持相对平衡的状态。这种交替步态有助于分散机器人重量,减小单一支撑腿的负担,从而减缓了机器人在运动中可能出现的倾斜和不稳定情况。通过对 Trot 步态的数学建模和力学分析,研

究发现其支撑面积相对均匀，有效减轻了机器人在不同地形上的平衡难题。这一特性对于需要在不同地势条件下执行任务的四足机器人尤为重要，例如在救援任务中，Trot 步态使机器人能够在复杂的地形中保持相对平稳，提高了其在紧急情况下的应对能力。

除了平衡性的提升，Trot 步态还通过合理的腿部协调实现了优越的稳定性。相对于一些其他步态，Trot 步态的运动规律更加规整，腿部之间的协调性更强。这有助于降低机器人在快速移动过程中可能出现的晃动和震动，提高了其整体稳定性。通过先进的运动控制算法，Trot 步态能够实现高频率的腿部运动，进一步维持机器人的平衡状态。这种高度的平衡性和稳定性使得 Trot 步态在工业生产线上执行复杂操作、搬运重物等任务时表现出色，确保了高效而稳定的工作执行。

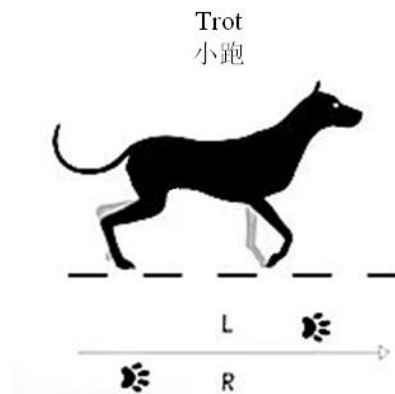


图 4-11 Trot 步态

Fig.4-11 Trot gait

优势二：高效的速度与能耗平衡

在四足机器人步态中，Trot 步态以其相对高效的速度和能耗平衡而脱颖而出。由于 Trot 步态的设计理念注重左前腿和右后腿、右前腿和左后腿的协同工作，机器人能够以相对较高的速度移动而不失稳定性。这为四足机器人在执行快速搜索、巡逻和监测等任务时提供了理想的运动模式。

同时，Trot 步态在提供高速移动的同时，相对较为节约能量。与一些高频率、高振幅的步态相比，Trot 步态更注重在高速移动中的能耗平衡。这得益于其腿部运动的相对简洁性和规则性，减小了机器人在运动中产生的能量消耗。通过合理的电池管理系统和功率控制策略，四足机器人能够更长时间地维持 Trot 步态，延长了其在实际应用中的工作时间。因此，在需要快速响应和长时间运动的任务中，Trot 步态具备了较高的适用性，为四足机器人的实际应用提供了更为可靠的解决方案。

如图 4-12 所示，在 Trot 步态中，整个运动周期内，四条腿的抬起时刻分别为 $\varphi_1=0$ ， $\varphi_2=0.5$ ， $\varphi_3=0$ ， $\varphi_4=0.5$ 。另外，步态中的参数 $1-p$ 代表了摆动相的时间，但要注意 $1-p$ 必须小于 φ ，以确保步态的稳定性和正确性。而当 $p=\varphi=0.5$ 时，任何时刻都只有两条腿处于摆动相或支撑相，这便是 Trot 步态中最为理想的状态。

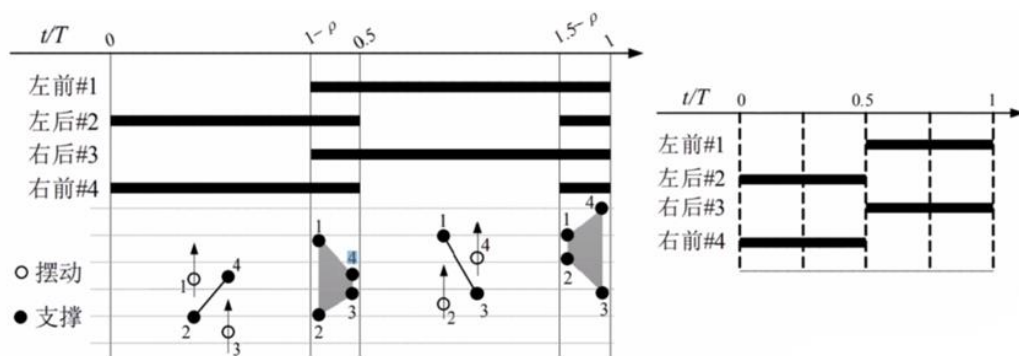


图 4-12 Trot 步态

Fig.4-12 Trot gait

4.3.3 四轮模式控制方案

四轮模式使用四轮差速控制方法。四轮差速控制是一种广泛应用于各种移动机器人和智能车辆的运动控制方法。该方法通过灵活调整四个车轮的转速，使车辆能够在不同的环境中实现平稳、灵活的运动。差速控制的核心思想是通过左右两侧的车轮差异速度来实现车体的旋转和转向。在这个过程中，控制系统需要实时监测车辆的状态，计算适当的差速值，以确保车辆能够按照预定轨迹运动。四轮差速控制方法的优越性在于其简单性和适应性，使其成为移动机器人和智能车辆领域中的热门选择。

四轮差速控制的基本原理涉及到车轮差速和转向的协同控制，如图 4-13 所示。

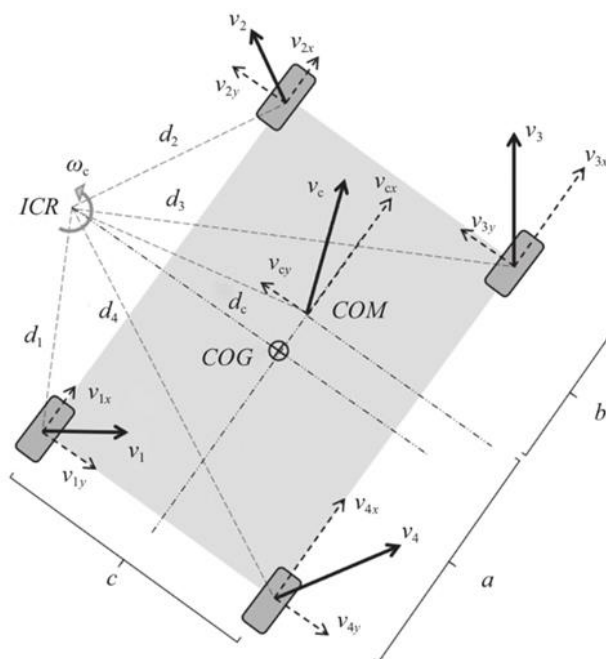


图 4-13 四轮差速原理图

Fig.4-13 Schematic diagram of a four-wheel differential

首先，通过控制左右两侧车轮的差速，车辆可以实现旋转运动。当左侧车轮的转速大于右侧车轮时，车体顺时针旋转；反之，则逆时针旋转。这种方式使得车辆能够在有

限的空间内实现精准的转向。通过对车轮速度的差异进行精准控制，车辆能够在不同的工作环境中实现平稳移动。这一基本原理的实现离不开先进的传感器系统，如编码器、陀螺仪等，用于实时监测车辆的状态，为控制系统提供准确的反馈信息。

图中： GOG 为底盘几何中心， COM 为底盘质心

d_i : 四个轮子到 ICR 的距离

v_i : 轮子的实际速度，侧向滑移速度 v_{ix} 和预设目标速度 v_{iy} 的合成速度

v_c : 底盘运动的线速度

w_c : 底盘运动的角速度

d_c : COM 到 ICR 的距离

a: 点 COM 与底盘后端之间的距离

b: 点 COM 与底盘前端之间的距离

c: 底盘左右轮的轴距

运动学分析:

绕圆心做圆周运动的物体，其线速度 v 、角速度 w 和圆周半径 d 满足 $w=v/d$ 。由此可建立底盘运动的约束关系:

$$w_c = \frac{v_c}{d_c} \quad (4-22)$$

假设 d_c 与 y 轴的夹角为 α_c ，上式可写成:

$$\begin{aligned} w_c &= \frac{v_c}{d_c} = \frac{v_c \cos \alpha_c}{d_c \cos \alpha_c} = \frac{v_{cx}}{d_{cy}} \\ w_c &= \frac{v_c}{d_c} = \frac{v_c \sin \alpha_c}{d_c \sin \alpha_c} = \frac{v_{cy}}{d_{cx}} \end{aligned} \quad (4-23)$$

刚体旋转时，刚体各个位置的角速度相等，即:

$$\begin{aligned} w_c &= \frac{v_i}{d_i} = \frac{v_i \cos \alpha_i}{d_i \cos \alpha_i} = \frac{v_{ix}}{d_{iy}} \\ w_c &= \frac{v_i}{d_i} = \frac{v_i \sin \alpha_i}{d_i \sin \alpha_i} = \frac{v_{iy}}{d_{ix}} \end{aligned} \quad (4-24)$$

并且， d_i 与 d_c 的 x 轴和 y 轴分量满足下式关系:

$$\begin{aligned} d_{1y} &= d_{2y} = d_{cy} - \frac{c}{2} \\ d_{3y} &= d_{4y} = d_{cy} + \frac{c}{2} \end{aligned} \quad (4-25)$$

设机器人底盘左右轮速度分别为 v_l 和 v_r ，且前后轮速度相同，则:

$$v_l = v_{1x} = v_{2x}$$

$$v_r = v_{3x} = v_{4x} \quad (4-26)$$

联立上式可得：

$$\begin{aligned} v_l = v_{1x} = v_{2x} &= w_c \left(d_{cy} - \frac{c}{2} \right) = v_{cx} - w_c \cdot \frac{c}{2} \\ v_r = v_{3x} = v_{4x} &= w_c \left(d_{cy} + \frac{c}{2} \right) = v_{cx} + w_c \cdot \frac{c}{2} \end{aligned} \quad (4-27)$$

将上式写为矩阵格式可得到四轮差速运动学关系式：

$$\begin{bmatrix} v_{cx} \\ w_c \end{bmatrix} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ -\frac{1}{c} & \frac{1}{c} \end{bmatrix} \begin{bmatrix} v_l \\ v_r \end{bmatrix} \quad (4-28)$$

4.3.4 机器人模式切换运动规划

在当前的机器人设计中，其独特之处在于能够在四足模式和轮模式之间进行切换，以适应不同的地形和任务需求。然而，由于结构和控制策略的限制，这种模式变换不能在机器人移动过程中直接进行。换句话说，机器人必须先停止当前的运动，然后才能开始形态的转换过程。下面详细描述了从四足模式到轮模式，以及从轮模式回到四足模式的具体变换步骤。

(1) 从四足模式转换为轮模式步骤如下图 4-14 所示。



图 4-14 腿型变轮型

Fig4-14 Leg type to Wheel type

1) 停止运动：机器人首先需要在原地完全停止所有运动，确保在变换形态时的稳定性和安全性。

2) 调整小腿姿态：接下来，机器人将其小腿反转，使用较短的一端作为接触地面的足端。这一步骤是为了准备小腿变形为轮形态的过程。

3) 变形为轮形态：随后，机器人将反转后的小腿进一步变形，形成轮状结构。

至此，机器人成功从四足模式切换到轮模式，可以开始利用轮式行动的优势，在平坦地面上高效移动。

(2) 从轮模式转换为四足模式步骤如下图 4-15 所示。

1) 降低身体：当机器人需要从轮模式切换回四足模式时，首先需要降低其身体，以减小转换过程中的不稳定性。

2) 抬起大腿并调整轮子：机器人抬起大腿，同时将轮子旋转至目标位置。这一步骤是为了恢复轮腿的初始位置，为接下来的变形做准备。

3) 轮腿变形：机器人进行轮腿变形操作，将轮状结构转变回腿状结构。这需要精确的控制，以确保变形过程的平稳和准确。

4) 站立为四足模式：完成轮腿变形后，机器人站立起来，恢复为四足模式。在这个模式下，机器人可以在更加复杂多变的地形中进行探索和移动。

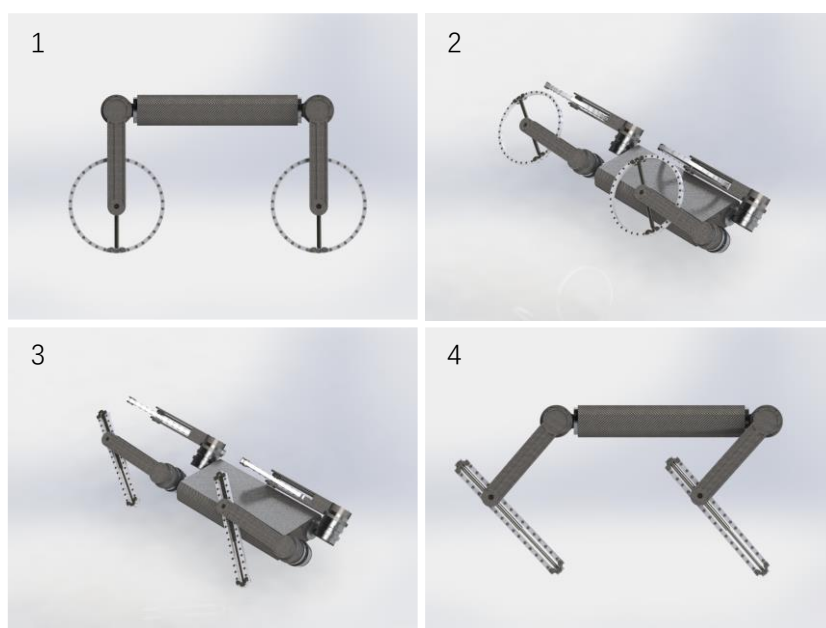


图 4-15 轮型变腿型

Fig.4-15 Wheel type to Leg type

4.4 本章小结

本章对四足机器人控制系统进行了研究，包括控制系统框架、电机驱动系统及机器人轮腿双模式控制算法；利用 FOC 算法实现无刷直流电机驱动的机器人关节的扭矩、位置、速度精确控制；使用力位混合控制 VMC 算法以及 Trot 步态规划实现机器人四足模式下平稳的运动控制；使用四轮差速算法实现机器人四轮模式下的运动控制；最后进行了机器人轮腿双模式切换运动规划。

第5章 运动控制仿真实验

5.1 引言

机器人仿真实验是在虚拟环境中对机器人系统进行模拟测试和评估的过程。机器人仿真实验在提高效率、降低成本、提供安全性保障等方面具有显著的好处和必要性。首先,仿真实验能够有效降低实验成本,避免对昂贵硬件的过度依赖,为研究者提供了经济、高效的测试手段;其次,通过仿真,可以模拟各种复杂环境和场景,从而全面评估机器人在多样化任务中的性能;最重要的是,仿真实验能够加速机器人算法和控制系统的优化过程,为实际应用提供更为可靠的技术基础。本章通过在 Webots 平台搭建仿真环境,对机器人两种运动模式进行运动控制仿真实验。

5.2 仿真环境搭建

5.2.1 软件介绍

Webots 是一款强大的机器人仿真软件,广泛应用于机器人领域的算法开发和性能评估。该软件提供了先进的物理引擎和仿真环境,使用户能够模拟各种机器人硬件、传感器和环境条件。Webots 支持多种编程语言,包括 C, C++, Python 等,方便开发者使用其熟悉的语言进行仿真实验。其直观的用户界面和丰富的模型库使用户能够快速构建和定制机器人模型,进行多样性的实验。Webots 还支持 ROS (机器人操作系统) 的集成,使得仿真实验与实际硬件开发更加衔接。无论是用于教学、研究还是工程应用,Webots 都是一个高度灵活且功能丰富的仿真平台,为机器人开发者提供了便捷而强大的工具。

5.2.2 环境搭建

本章基于 Webots 环境构建了四足轮腿变形机器人仿真与操作训练平台,如图 5-1 所示。在该平台中,利用控制器和 PyCharm 联合进行虚拟样机的仿真控制和传感器信息采集,采用 VMC 控制方法实现机器人的控制。通过键盘输入期望速度和位姿等指令,实现对虚拟样机的操作操控,并记录其运动状态参数并进行图形化输出,以方便进行状态分析和调试。

通过在仿真平台上进行控制方法有效性验证实验,可以发现控制方法的潜在问题,从而减少实物样机实验失败造成的损失。此外,可以通过反复训练,提升对机器人遥控器的熟练程度,增强应对紧急状况的能力,而无需承担实物样机训练的成本和代价。

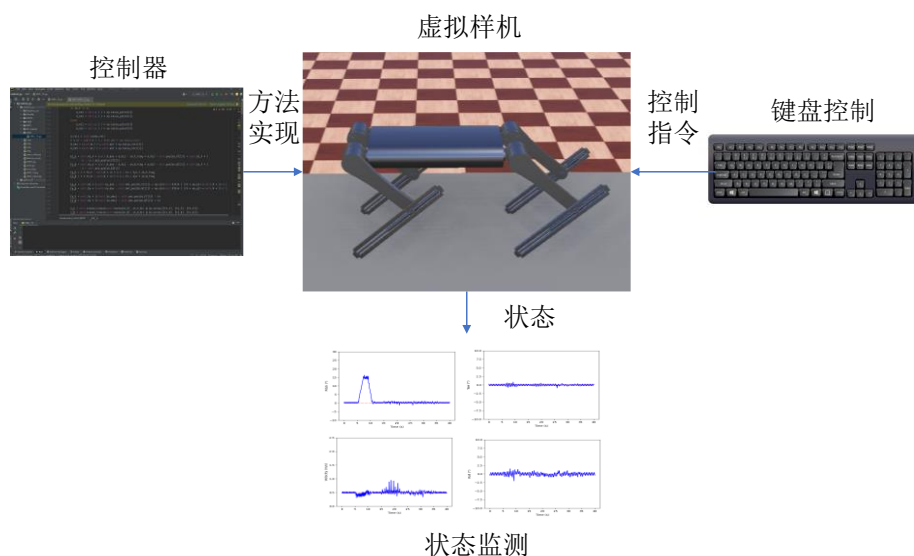


图 5-1 仿真环境

Fig.5-1 Simulation environment

5.3 四足模式仿真

将 SolidWorks 中设计的机器人三维模型导出为 URDF 格式是实现仿真和控制的关键一步。URDF 是一种用于描述机器人结构和运动学的 XML 文件格式,它包含了关节、连杆等信息。首先,通过 SolidWorks 导出 URDF 文件,确保模型的关节和连杆信息准确无误。

接下来,使用 Python 中的 urdf2webots 工具包,将 URDF 格式转换为 Webots 可识别的 Proto 格式。Webots 是一款强大的机器人仿真软件,支持多种机器人模型和传感器,使得开发者能够在虚拟环境中进行快速而准确的仿真实验。

在 Webots 软件内,配置机器人的各个关节,包括电机和传感器。通过添加电机,定义关节的运动范围和速度、扭矩控制范围,为机器人建立运动学模型。同时,配置传感器,如足底力传感器、IMU 惯性传感器、电机位置编码器等,以模拟真实环境中的感知能力。

为了进一步完善仿真,建立仿真地形环境是不可或缺的一步。Webots 支持用户创建各种地形,从简单的平面到复杂的三维地形,为机器人提供多样的运动和感知挑战。这有助于验证机器人在不同环境下的性能和适应能力。

在仿真环境搭建完成后,根据第四章设计的运动控制算法,编写运动控制器,以实现机器人在仿真环境中的智能运动。编写控制器的过程中,需要考虑与 Webots 仿真引擎的接口,确保控制指令能够正确传递给机器人模型。具体代码实现详见附录 A。

四足模式测试其地形通过性和运动稳定性,如图 5-2 所示,机器人从右向左,以 0.5m/s 的速度先后通过 20 度斜坡、台阶、单边桥以及减速带区域。

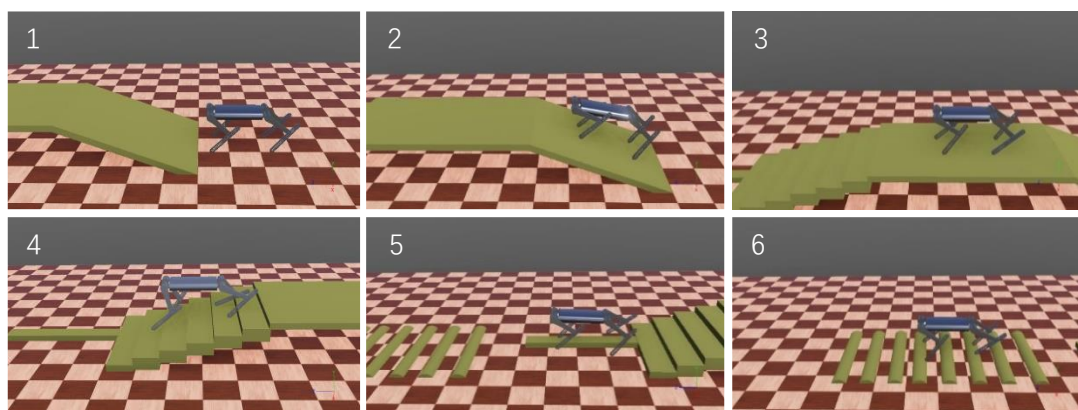


图 5-2 四足多地形测试

Fig.5-2 Quadruped multi-terrain test

图 5-3 为仿真测试过程中机器人躯干姿态数据，上斜坡时机器人采取坡度自适应策略，改变姿态以适应斜坡地形，之后采取保持机身水平姿态通过楼梯、单边桥以及减速带区域，从图中数据可以看出，机器人运动平稳，姿态稳定性满足一般任务要求。

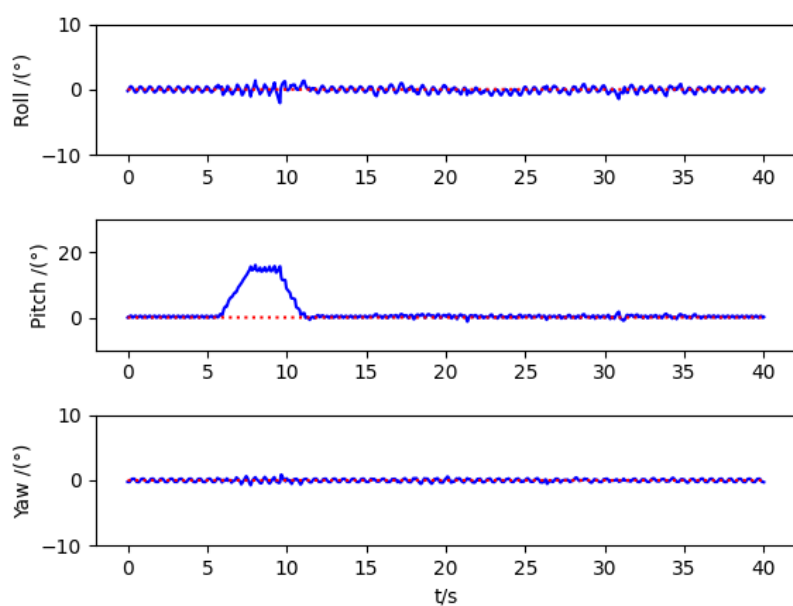


图 5-3 四足躯干姿态

Fig.5-3 Four-legged torso pose

机器人有两种自适应地形模式，分别为保持机身水平和斜坡自适应两种，如下图 5-4 所示。两种模式都具有其各自的优点，适用于不同场景，第一种保持机身水平模式在不平整的坑洼路面以及载有敏感货物时较为适用，可保证运载货物的平稳性；第二种斜坡自适应模式则适合在通过较长的倾斜路面以及上下楼梯时较为适用，通过倾斜机身来控制重心提高机器人运动的稳定性。

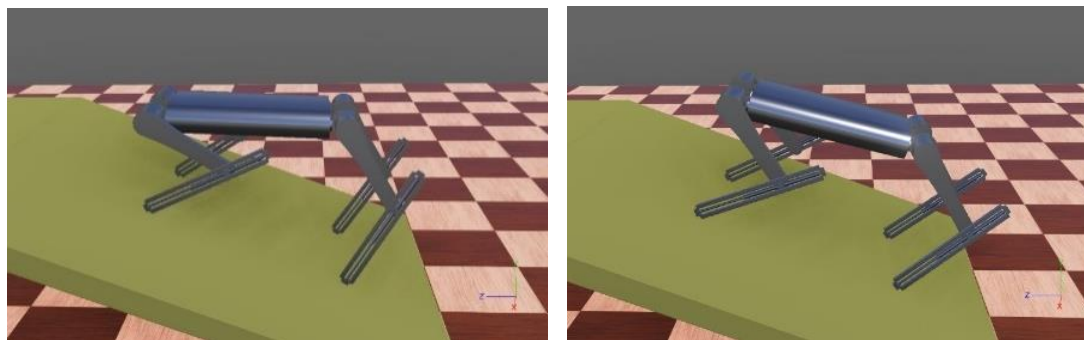


图 5-4 自适应地形模式

Fig.5-4 Adaptive terrain mode

通过仿真进行四足模式极速测试，下图 5-5 为测试数据，可以看出，当速度达到 1.8m/s 时，机器人速度及姿态角发生剧烈变化，说明此时机器人已经失控，所以 1.8m/s 即为当前控制算法和步态下的机器人极限速度。

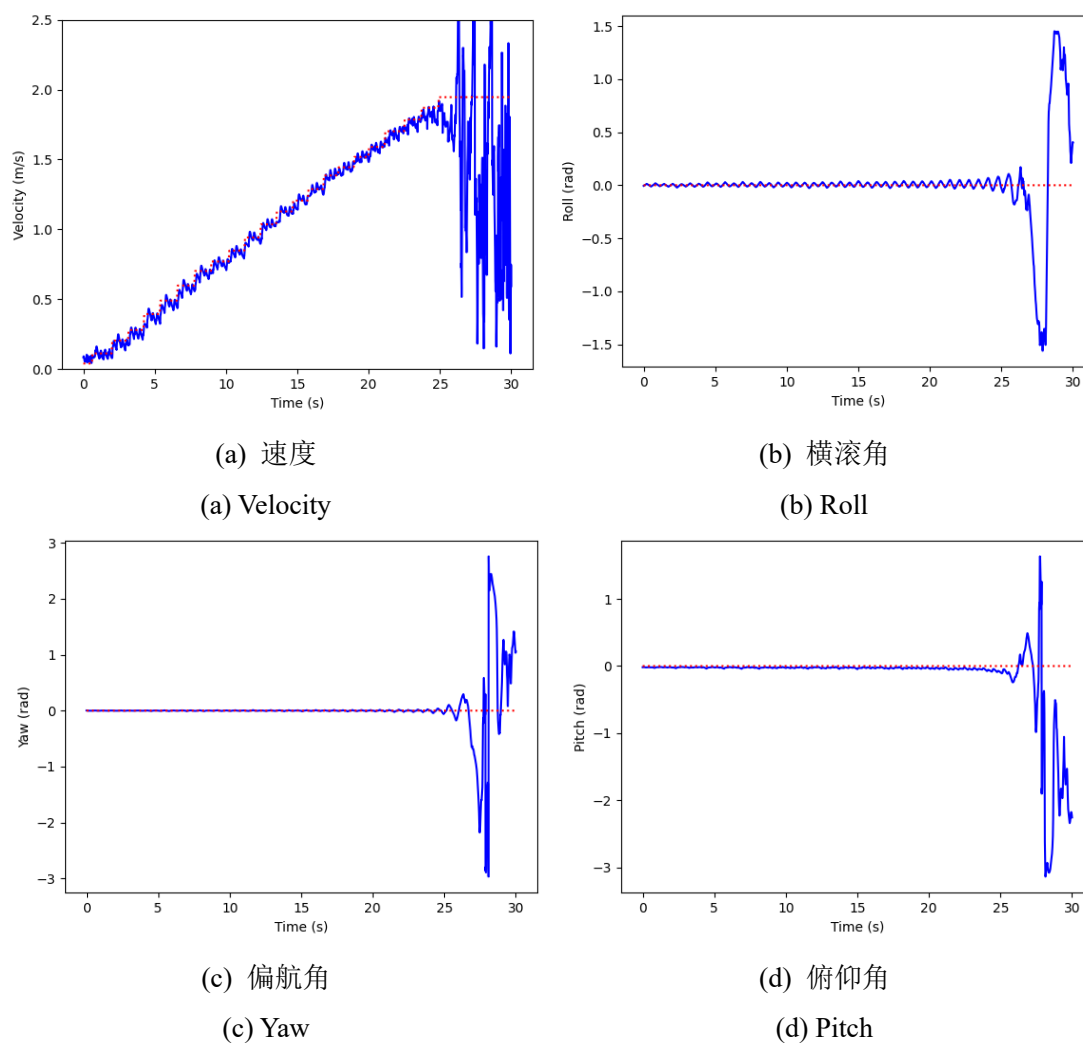
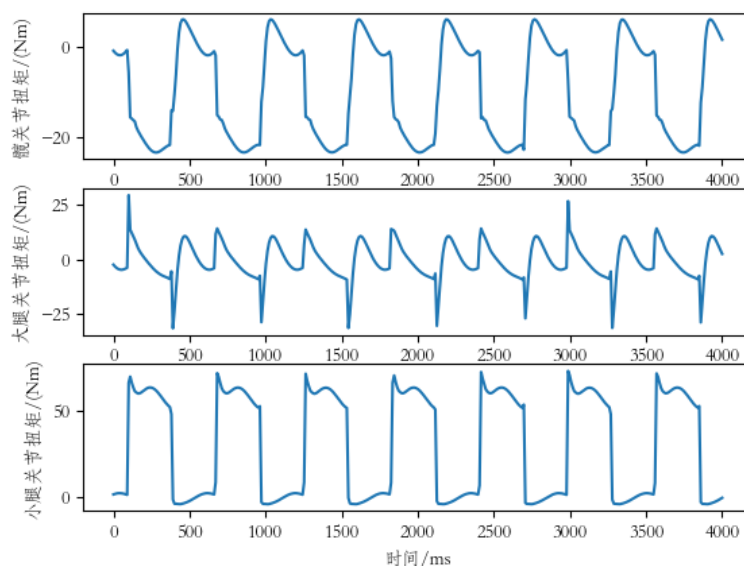


图 5-5 四足极速测试仿真数据

Fig.5-5 Quadruped speed test simulation data

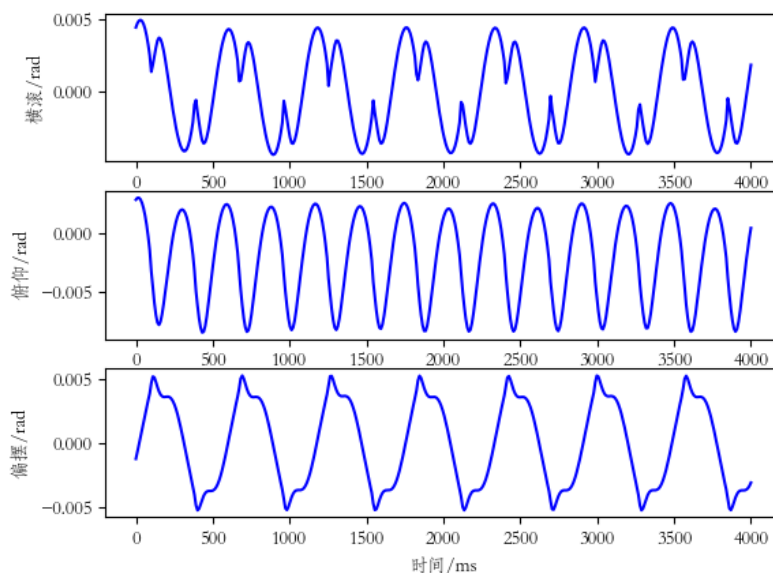
在进行机器人设计和性能测试时，平地行走的稳定性测试是一个关键的评估指标。这项测试旨在模拟机器人在实际应用场景中的运动状态，以确保其在执行任务时的稳定性和可靠性。本次测试中，特别关注了机器人在平地上以 1.5 米/秒的速度行走时的表现。

测试过程详细记录了机器人各主要关节的输出扭矩以及机器人整体姿态角的变化情况，具体数据如下图 5-6 所示。



(a) 关节扭矩

(a) joint torque



(b) 姿态角

(b) attitude angle

图 5-6 稳定性测试数据

Fig.5-6 Stability test data

在这些数据中,可以看到机器人髋关节、大腿关节和小腿关节的最大扭矩值分别为 24Nm、30 Nm 和 65 Nm。这些扭矩值反映了机器人各关节在行走过程中所承受的力矩,是评估机器人结构强度和驱动系统性能的重要参数。除了关节扭矩之外,机器人的姿态稳定性也是测试的重要内容。在本次测试中,观察到三个主要姿态角(可能包括俯仰角、滚转角和偏航角)的变化幅度均控制在 2 度以内。这一结果表明,机器人在平地行走过程中能够保持较高的姿态稳定性。

在进行机器人控制算法的研究与开发过程中,算法的优化对于提高机器人的性能至关重要。本研究中,重点关注了机器人在具有挑战性的环境下的表现,特别是在倾斜坡面上的稳定性问题。为了更准确地评估算法优化的效果,设计了一项仿真实验,通过对比优化前后的结果来展示改进的成效。

实验设置中,构建了一个倾斜坡面环境,如图 5-7 所示,该环境的坡度在 x 方向和 y 方向均为 20 度。这种设置模拟了一种常见但具挑战性的场景,要求机器人能够在不同方向的倾斜面上保持稳定。

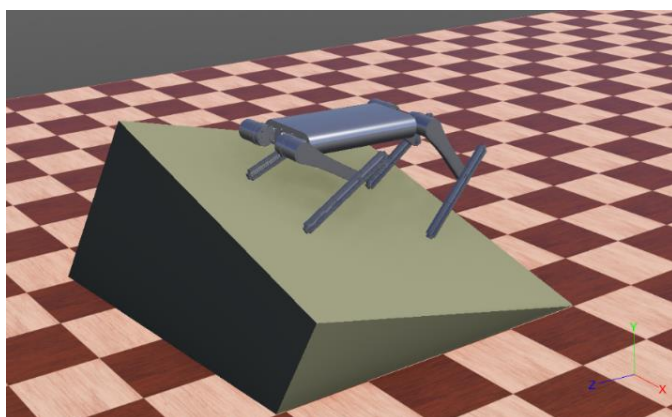


图 5-7 斜面稳定性测试

Fig.5-7 Slope stability test

在这个环境下,首先使用了未经优化的 VMC(虚拟模型控制)算法作为基准。VMC 算法是一种常用的控制策略,它通过模拟虚拟力来实现对机器人的控制,但在处理复杂环境时可能表现不佳。随后,引入了优化后的力位混合控制算法。这种算法结合了力控制和位置控制的优点,旨在提高机器人在复杂环境中的稳定性和适应性。通过在仿真环境中应用这两种算法,收集了机器人三个姿态角(横滚角、俯仰角和偏摆角)的实时数值,以便进行详细的分析和对比。

对比结果如图 5-8 所示,通过分析图 5-8 中的(a)优化前的俯仰角、(b)优化后的俯仰角、(c)优化前的横滚角、(d)优化后的横滚角、(e)优化前的偏摆角以及(f)优化后的偏摆角的数据,可以清晰地看到优化后的效果。优化后的力位混合控制算法显著减少了机器人在三个姿态角上的波动,从而使机器人的运动更加平稳。优化前的数据显示,在倾斜坡面上,机器人的姿态角波动较大,这意味着机器人在尝试保持平衡时会

经历较大的姿态调整，这不仅消耗了更多的能量，也增加了机器人失稳的风险。相反，优化后的数据展示了在相同条件下，机器人的姿态角波动明显减少，说明机器人能够更有效地维持稳定状态，即使面对复杂的倾斜坡面环境。

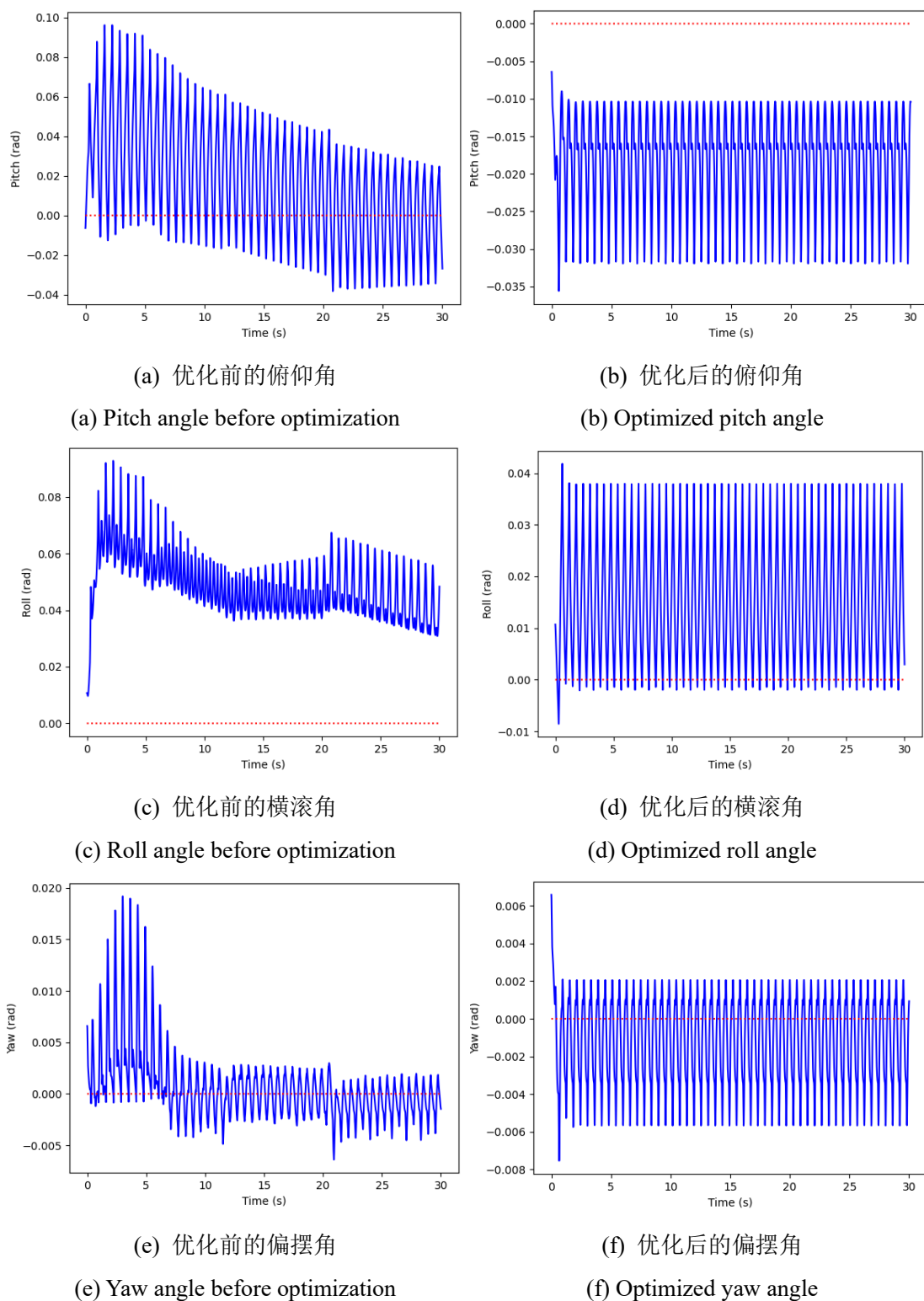


图 5-8 算法优化前后数据对比

Fig.5-8 Comparison of data before and after algorithm optimization

通过引入优化后的力位混合控制算法，成功地提高了机器人在倾斜坡面上的稳定性和性能。

5.4 四轮模式仿真

本节进行机器人四轮差速控制仿真实验的主要目的是验证四轮差速控制在运动过程中的灵活性和效果，实验过程如图 5-9 所示。在实验中，设置了 4 个锥桶，每两个之间间隔 2 米，机器人需要绕过这些桩进行移动。通过这样的布置，测试机器人在狭窄环境中的灵活性和控制能力。

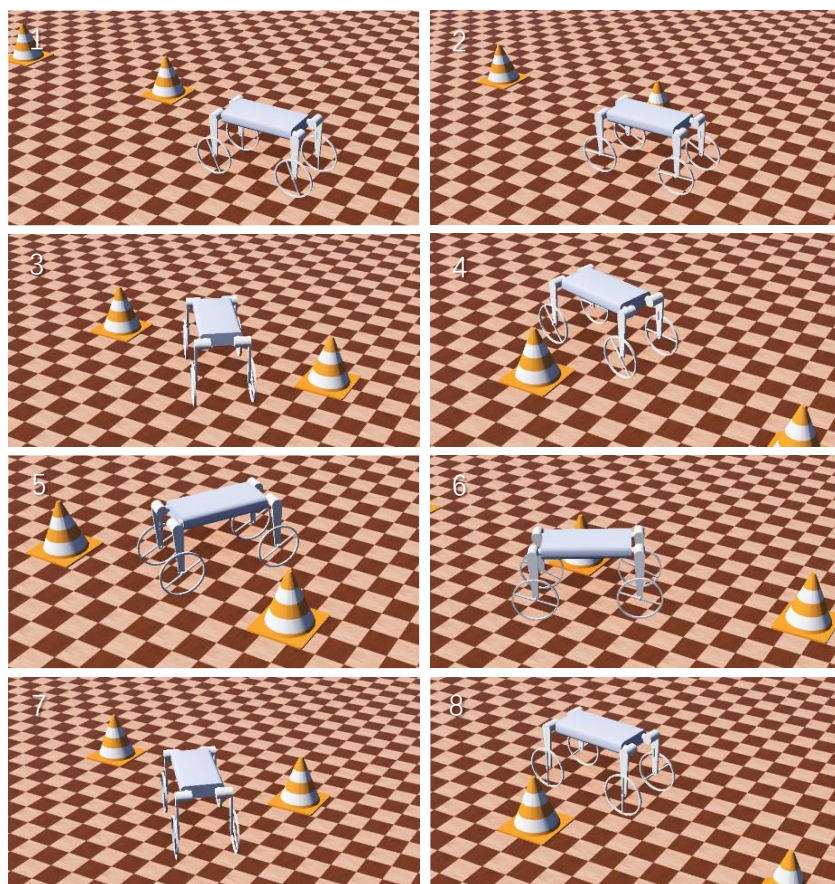


图 5-9 四轮模式仿真

Fig.5-9 Four-wheel mode simulation

实验过程中，机器人需要根据预设的控制算法，调整四个车轮的差速来实现绕桩运动。通过控制不同车轮的转速和方向，机器人可以实现精确的转弯和定位，从而顺利通过设置的障碍物。在实验结果中，观察到机器人成功绕过每个锥桶，完成了整个移动任务。这表明四轮差速控制系统具有良好的运动灵活性和精准性，能够在复杂环境中有效地执行任务。

5.5 本章小结

本章综合运用机械设计、URDF 文件处理、仿真环境搭建和控制算法编写等多个领域的知识，基于 Webots 软件搭建了机器人仿真平台；对机器人两种运动模式分别进行了仿真控制实验，对机器人的性能进行了测试和优化，仿真结果表明机器人在两种运动模式下的性能均满足设计要求。

第6章 样机实验

6.1 引言

在本章节中，利用 3D 打印技术成功制作了可变形轮腿和关节模组的实物，并进行了一系列实验验证，旨在探究这些创新设计的实际效果。包括可变形轮腿的变形可行性实验以及关节模组电机控制实验。这些实验结果将有助于深入理解可变形轮腿和关节模组的设计特点，探索其在机器人领域的实际应用潜力，并为未来的研发工作提供重要参考。

6.2 变形机构实验测试

下图所示为机器人整体结构的渲染图，其中图 6-1 展示了其腿形态，图 6-2 展示了其轮形态。由于时间和经济等条件的限制，无法完成对整机的制造，因此只对关键部件进行了样机实验，包括可变形轮腿和一体化关节模组。



图 6-1 腿形态

Fig.6-1 Leg morphology



图 6-2 轮形态

Fig.6-2 Wheel morphology

图 6-3 所示为 3D 打印末端限位装置，图 6-4 所示为组装好的可变形轮腿，连接电源进行了变形实验验证。通过实验结果与之前的仿真结果进行对比，发现它们高度一致，从而验证了变形轮腿结构的可行性和有效性。在实验中，观察到可变形轮腿在连接电源后成功进行了变形，其变形情况与预期一致。这表明本文研究开发的可变形轮腿结构具有良好的实用性和稳定性，在实际应用中可以有效完成指定的功能和任务。这一实验结果的验证提供了关于本可变形轮腿结构性能的重要证据，加强了对其设计优势和适用性的理解。通过将仿真结果与实际实验相结合，进一步确认了本设计在机器人领域的潜在应用，为未来机器人结构设计和功能拓展提供了有力支持。



(a) 轮型状态

(b) 腿形状态

(a) wheel state

(b) leg state

图 6-3 3D 打印末端限位装置

Fig.6-3 3D printing End limiting device



图 6-4 3D 打印可变形轮腿变形实验

Fig.6-4 3D printing deformable wheel leg deformation experiment

6.3 关节模组控制实验

在这次实验中,对关节模组电机进行了速度、位置和力矩三种不同控制方式的实验,并通过调节 PID 控制器的参数,寻找最佳的控制响应速度。下图 6-5 所示为 3D 打印关节模组,图 6-6 所示为关节模组实验装置,其中包括 24V 直流电源、电机驱动器、CH340、关节模组,连接电脑后可通过上位机观察其动态数据。



(a) 零部件图

(a) drawings of components



(b) 组装图

(b) assembly drawings

图 6-5 3D 打印关节模组

Fig.6-5 3D Printed Joint Module

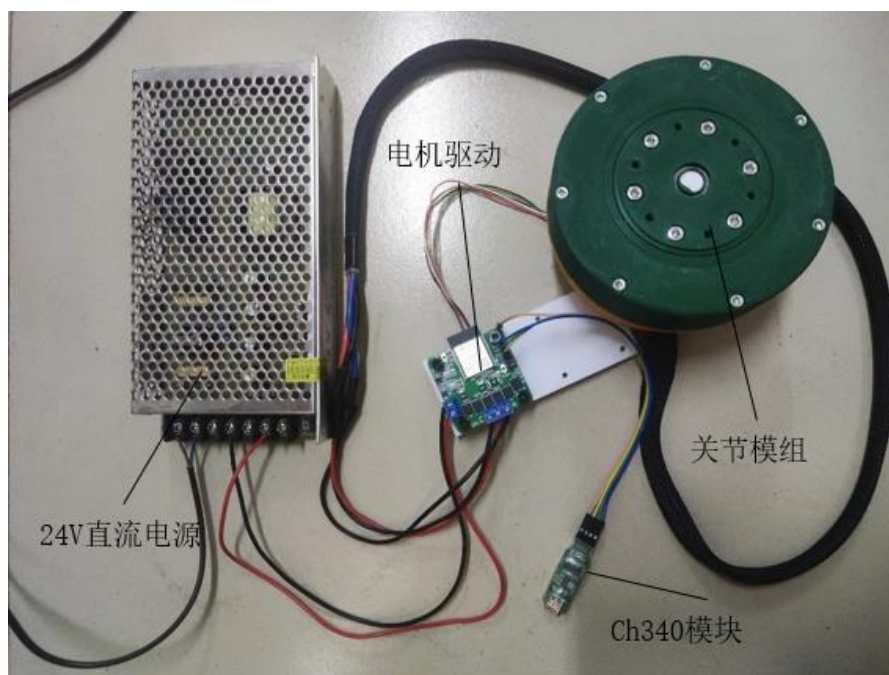


图 6-6 关节模组实验装置

Fig.6-6 Joint Module Experimental Apparatus

首先，对电机进行了速度控制实验。如图 6-7 所示，记录下了速度控制的响应图，通过调节速度环的 PID 系数，尝试使电机的速度响应达到最优状态。

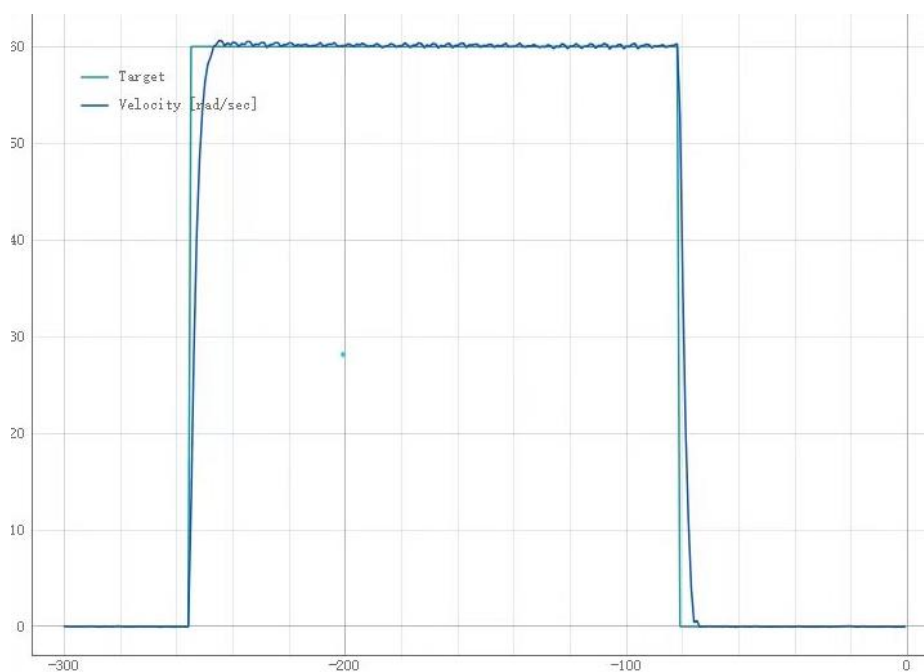


图 6-7 关节模组电机速度控制实验

Fig.6-6 Experiment on motor speed control of joint module

随后进行了位置控制实验，得到了图 6-8 中的位置控制响应图。在这个实验中，调节了位置环的 PID 系数，以期望获得最佳的位置控制响应速度。

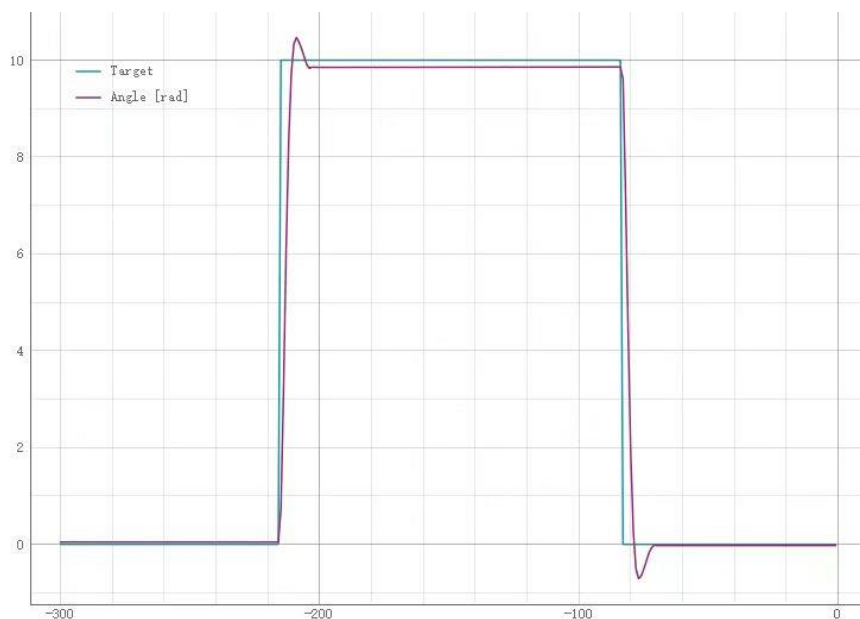


图 6-8 关节模组电机位置控制实验

Fig.6-8 Experiment on motor position control of joint module

最后，进行了力矩控制实验，并记录下了图 6-9 中的力矩控制响应图。需要说明的是，由于力矩大小与电流大小成比例关系，因此实际控制的量为经过 Clark 变换和 Park 变换后的 I_q 、 I_d 电流值。在这个实验中，调节了电流环的 PID 系数，力求达到最佳的力矩控制响应速度。

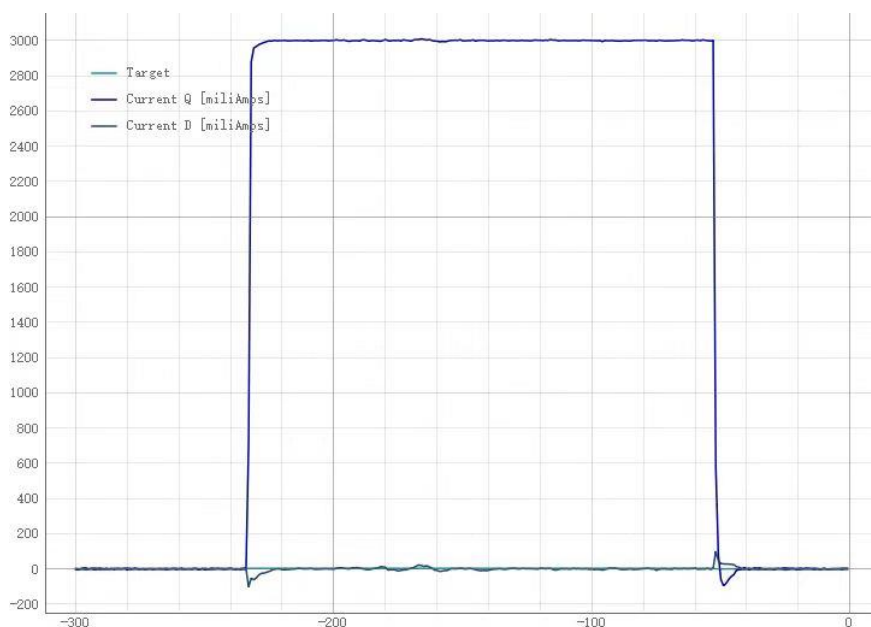


图 6-9 关节模组电机力矩控制实验

Fig.6-9 Experiment on torque control of joint module motor

通过这一系列实验，希望能够深入了解电机在不同控制方式下的响应特性，并通过调节 PID 控制器的参数，寻找到最适合电机的控制策略，以实现最优的控制效果。

为了测试关节模组的负载能力对其进行扭矩测试实验,如下图 6-10 所示,关节模组输出端装有长度 0.4 米的连杆,连杆末端使用绳子连接负载物体,对关节模组进行控制,使连杆旋转至水平位置拉起负载,逐渐增加负载进行实验。负载重量从 2kg 逐渐增加至 10kg,由于 3D 打印材料强度有限,负载增加到 10kg 后结束了实验,此时正好达到关节模组设计的额定扭矩,实验过程中关节模组没有出现震荡的情况,都非常平稳的拉起了负载。此次实验结果表明本关节模组具有出色的负载能力,满足性能要求。

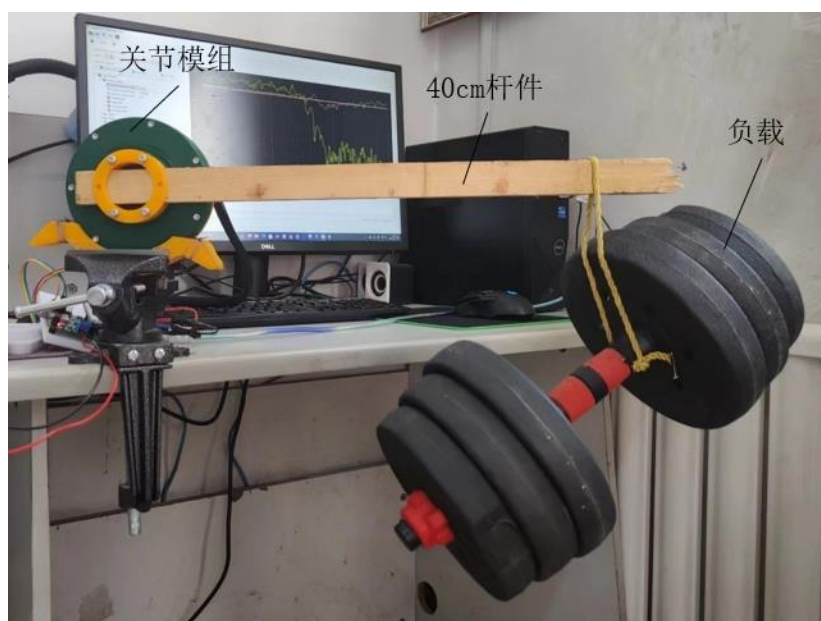


图 6-10 关节模组扭矩测试实验

Fig.6-10 torque test experiment of joint module

6.4 本章小结

本章节进行了可变形轮腿的变形可行性实验,通过对实物进行测试和观察,验证了其在实际应用中的可行性和稳定性。其次,进行了关节模组电机控制实验,通过搭建电机控制系统,并结合关节模组的实物,验证了关节模组电机控制在关节角度、速度以及扭矩控制中的效果和精度,最后进行了关节模组的扭矩测试,测试结果表明本关节模组满足设计的性能要求。

第7章 结论与展望

7.1 全文总结

针对目前单一结构移动机器人的局限性,本文采用轮腿复合的形式,提出了一种轮腿变形四足机器人,本轮腿变形四足机器人采用完整的仿生四足机器人结构作为基础结构,机器人小腿部分采用轮腿变形式设计,可充分发挥轮、腿两种结构的优点。

本论文主要开展了如下工作:

通过分析拱桥结构的负载特性提出了一种新型可变形轮腿结构,并对其进行了优化改进,利用 Solidworks 对其进行三维建模及装配,使用 motion 模块进行运动仿真,利用 Ansys 对其进行静力学有限元分析,验证了本轮腿结构的可行性;而后设计了机器人关节模组、大腿连杆、导电滑环等部件,完成了机器人的整体结构设计,并进行了三维建模装配;

采用 D-H 参数法对机器人进行正逆运动学建模分析,求解腿部工作空间,同时,计算了单腿的雅可比矩阵,并进行了机器人行走步态的规划;对四足机器人控制系统进行了研究,包括控制系统框架、电机驱动系统及机器人轮腿双模式控制算法;利用 FOC 算法实现无刷直流电机驱动的机器人关节的扭矩、位置、速度精确控制;

在仿真软件中建立了虚拟样机模型,使用力位混合控制 VMC 算法以及 Trot 步态规划实现机器人四足模式下平稳的运动控制,使用四轮差速算法实现机器人四轮模式下的运动控制,并对机器人足式和轮式在各类地形下的运动进行了仿真实验,验证了机器人在平地、非结构地形和坡面地形下的通过性;而后进行了机器人轮腿双模式切换运动规划;

最后采用 PLA 材料 3D 打印制作了轮腿变形机构和一体化关节模组,对轮腿变形机构样机进行了变形实验,验证了其可行性;而后进行了关节模组电机控制实验,通过搭建电机控制系统,并结合关节模组的实物,验证了关节模组电机控制在关节角度、速度以及扭矩控制中的效果和精度,最后进行了关节模组的扭矩测试,测试结果表明本关节模组满足设计的性能要求。

通过以上研究分析和实验测试等工作验证了本文研究的轮腿变形四足机器人的可行性,为轮腿复合式机器人的研究提供新的思路。

7.2 展望

本文对轮腿可变形四足机器人的结构与控制方法进行了初步探讨,鉴于技术的发展和应用场景的多样性,目前的设计方案仍有许多改进的空间,以提升机器人的整体性能

和适应能力。未来还需要做的工作主要如下：

- （1）进一步优化可变形轮腿结构，增强轮腿的承载能力和耐用性。
- （2）增加视觉传感器，如激光雷达、深度摄像头等，增强机器人感知环境的能力。
- （3）使用强化学习算法开发机器人控制器，提高机器人智能水平和自主决策能力。
- （4）机器人整体样机的制作。

参考文献

- [1] 周济. 智能制造——“中国制造 2025”主攻方向[J]. 中国机械工程, 2015, 26(17): 2273-2284.
- [2] 陶永, 王田苗, 刘辉, 等. 智能机器人研究现状及发展趋势的思考与建议[J]. 高技术通讯, 2019, 29(02): 149-163.
- [3] Kaur T, Kumar D. Wireless Multifunctional Robot for Military Applications[C]// Proceedings of the International Conference on Recent Advances in Engineering and Computational Sciences. Chandigarh, India: IEEE, 2015: 1-5.
- [4] 侯康, 孙汉旭, 贾庆轩. 球一轮复合可变形机器人的结构设计与分析[J]. 机械工程学报, 2012, 48(15): 25-31.
- [5] 姜楠, 杨志岳, 孙涛. 轮腿复合式移动机器人的结构与步态规划研究[J]. 机械设计, 2022, 39(06): 79-85.
- [6] 朱磊磊, 陈军. 轮式移动机器人研究综述[J]. 机床与液压, 2009, 37(08): 242-247.
- [7] 丁希仑, 徐坤. 星球探测机器人[J]. 航空制造技术, 2013(18): 34-39.
- [8] Siegwart R, Nourbakhsh I. R. Introduction to Autonomous Mobile Robots[J]. 2004.
- [9] 朱岩, 王明辉, 李斌, 等. 履带可变形机器人越障性能研究[J]. 机器人, 2015, 37(06): 693-701.
- [10] 苏波, 闫瞳, 许威, 等. 四足机器人高机动越野技术研究[J]. 中国科学: 技术科学, 2023, 53(09): 1574-1588.
- [11] Takahashi M, Yoneda K, Hirose S. Rough Terrain Locomotion of a Leg-Wheel Hybrid Quadruped Robot[C]// Proceeding of the IEEE International Conference on Robotics and Automation ICRA. Orlando, FL, USA: IEEE, 2006: 1090-1095.
- [12] 刘静, 赵晓光, 谭民. 腿式机器人的研究综述[J]. 机器人, 2006(01): 81-88.
- [13] 张程煜, 郭盛, 赵福群. 新型轮腿复合机器人的运动分析及步态研究[J]. 机械工程学报, 2019, 55(15): 145-153.
- [14] Mosher R. Test and evaluation of a versatile walking truck[J]. Off-Road Mobility Research Symposium, 1968: 359-379.
- [15] Rebula J R, Neuhaus P D, Bonnlander B V, et al. A controller for the littledog quadruped walking on rough terrain[C]//Proceedings 2007 IEEE International Conference on Robotics and Automation. IEEE, 2007: 1467-1473.
- [16] Raibert M, Blankespoor K, Nelson G, et al. Bigdog, the rough-terrain quadruped robot[J]. IFAC Proceedings Volumes, 2008, 41(2): 10822-10825.
- [17] Sapaty P. Military robotics: latest trends and spatial grasp solutions[J]. International Journal of Advanced Research in Artificial Intelligence, 2015, 4(4): 9-18.
- [18] Bouman A, Ginting M F, Alatur N, et al. Autonomous Spot: Long-Range Autonomous Exploration of Extreme Environments with Legged Locomotion[C]//2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2020.
- [19] Hutter M, Gehring C, Jud D, et al. Anymal-a highly mobile and dynamic quadrupedal robot[C]//2016 IEEE/RSJ international conference on intelligent robots and systems (IROS). IEEE, 2016: 38-44.
- [20] Hyun D J, Seok S, Lee J, et al. High speed trot-running: Implementation of a hierarchical controller using proprioceptive impedance control on the MIT Cheetah[J]. The International Journal of Robotics Research, 2014, 33(11): 1417-1445.

- [21] Park H W, Park S, Kim S. Variable-speed quadrupedal bounding using impulse planning: Untethered high-speed 3d running of mit cheetah 2[C]//2015 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2015: 5163-5170.
- [22] Bledt G, Powell M J, Katz B, et al. MIT Cheetah 3: Design and control of a robust, dynamic quadruped robot[C]//2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2018: 2245-2252.
- [23] Katz B, Di Carlo J, Kim S. Mini cheetah: A platform for pushing the limits of dynamic quadruped control[C]//2019 International Conference on Robotics and Automation (ICRA). IEEE, 2019: 6295-6301.
- [24] 马培荪, 陆顺寿, 何奇伟. 四足步行机器人的运动和结构设计[J]. 机械设计与研究, 1991(04): 34-39.
- [25] 吴立卫, 刘明宝, 林润华. JTUWM- II 四足步行机器人控制系统的计算机结构[J]. 机器人, 1992(03): 28-32.
- [26] 马培荪, 窦小红, 刘臻. 全方位四足步行机器人的运动学研究[J]. 上海交通大学学报, 1994(02): 39-47.
- [27] 孙磊. 四足机器人仿生控制方法及行为进化研究[D]. 合肥: 中国科学技术大学, 2009.
- [28] 柴汇, 孟健, 荣学文, 等. 高性能液压驱动四足机器人 SCalf 的设计与实现[J]. 机器人, 2014, 36(04): 385-391.
- [29] 蒋振宇. 基于 SLIP 模型的四足机器人对角小跑步态控制研究[D]. 哈尔滨: 哈尔滨工业大学, 2015.
- [30] 邓黎明. 四足小象机器人实时控制系统的设计与研究[D]. 上海: 上海交通大学, 2015.
- [31] Chuanyu Y, Kai Y, Qiuguo Z, et al. Multi-expert learning of adaptive legged locomotion[J]. Science robotics, 2020, 5(49).
- [32] 房晓楠. 云深处: “不止要做中国版波士顿动力”[J]. 机器人产业, 2020(03): 104-108.
- [33] 金红. 中国团队推出四足机器人, 对标波士顿动力[J]. 智能机器人, 2017(5)1.
- [34] Truong J, Yarats D, Li T, et al. Learning Navigation Skills for Legged Robots with Learned Robot Embeddings[J]. 2020.
- [35] Kassel S. Lunokhod-1 Soviet lunar surface vehicle[M]. Rand, 1971.
- [36] Bogue, Robert. Robots for space exploration[J]. The Industrial Robot, 2012, 39(4): 323-328.
- [37] Guccione S, Muscato G. The wheeleg robot[J]. IEEE Robotics & Automation Magazine, 2003, 10(4): 33-43.
- [38] Eich M, Grimminger F, Kirchner F. A versatile stair-climbing robot for search and rescue applications[C]//2008 IEEE international workshop on safety, security and rescue robotics. IEEE, 2008: 35-40.
- [39] Jeans J B, Hong D. IMPASS: Intelligent mobility platform with active spoke system[C]//2009 IEEE International Conference on Robotics and Automation. IEEE, 2009: 1605-1606.
- [40] Campbell D, Buehler M. Stair descent in the simple hexapod'RHex'[C]//2003 IEEE International Conference on Robotics and Automation (Cat. No. 03CH37422). IEEE, 2003, 1: 1380-1385.
- [41] Galloway K C, Haynes G C, Ilhan B D, et al. X-RHex: A highly mobile hexapedal robot for sensorimotor tasks[J]. 2010.
- [42] Yun S S, Lee J Y, Jung G P, et al. Development of a transformable wheel actuated by soft pneumatic actuators[J]. International Journal of Control, Automation and Systems, 2017, 15: 36-44.
- [43] Merṫẏüz İ, Tanyıldızı A K, Taşar B, et al. FUHAR: A transformable wheel-legged hybrid mobile robot[J].

- Robotics and Autonomous Systems, 2020, 133: 103627.
- [44] Bjelonic M, Bellicoso C D, de Viragh Y, et al. Keep rollin'—whole-body motion control and planning for wheeled quadrupedal robots[J]. IEEE Robotics and Automation Letters, 2019, 4(2): 2116-2123.
- [45] Marko B, K. P S, Dario C B, et al. Rolling in the Deep – Hybrid Locomotion for Wheeled-Legged Robots Using Online Trajectory Optimization[J]. IEEE Robotics and Automation Letters, 2020, 5(2): 3626-3633.
- [46] Zhou H, Li H, Xu D, et al. Ground-based real-time tracking and traverse recovery of China's first lunar rover[J]. Advances in Space Research, 2016, 57(3): 880-888.
- [47] 胡明, 邓宗全, 王少纯, 等. 月球探测车移动系统的关键技术分析[J]. 哈尔滨工业大学学报, 2003, (07): 795-798.
- [48] 张狄, 欧阳求保. “玉兔号”: 行走在月球[J]. 科技纵览, 2016(09): 52-53.
- [49] 郭丽峰, 陈恳, 赵旦谱, 等. 一种轮腿式变结构移动机器人研究[J]. 制造业自动化, 2009, 31(10): 1-6.
- [50] Lu D, Dong E, Liu C, et al. Design and development of a leg-wheel hybrid robot “HyTRo-I”[C]//2013 IEEE/RSJ International Conference on Intelligent Robots and Systems. IEEE, 2013: 6031-6036.
- [51] Qiao G, Song G, Zhang Y, et al. A Wheel-legged Robot with Active Waist Joint: Design, Analysis, and Experimental Results[J]. Journal of Intelligent Robotic Systems, 2016, 83(3-4): 485-502.
- [52] 徐坤, 郑羿, 丁希仑. 六轮腿式机器人结构设计与运动模式分析[J]. 北京航空航天大学学报, 2016, 42(01): 59-71.
- [53] Wei Z, Song G, Qiao G, et al. Design and implementation of a leg-wheel robot: transleg[J]. Journal of Mechanisms and Robotics, 2017, 9(5): 051001.
- [54] 陈志华, 汪首坤, 王军政, 等. 电动并联六轮足机器人的运动驱动与多模态控制方法[J]. 机器人, 2020, 42(05): 534-549.
- [55] Chen S C, Huang K J, Chen W H, et al. Quattroped: a leg--wheel transformable robot[J]. IEEE/ASME Transactions On Mechatronics, 2013, 19(2): 730-742.
- [56] Chen W H, Lin H S, Lin Y M, et al. TurboQuad: A novel leg-wheel transformable robot with smooth and fast behavioral transitions[J]. IEEE Transactions on Robotics, 2017, 33(5): 1025-1040.
- [57] Chen H Y, Wang T H, Ho K C, et al. Development of a novel leg-wheel module with fast transformation and lea** capability[J]. Mechanism and Machine Theory, 2021, 163: 104348.
- [58] 张承耀, 王刚, 车洪磊, 等. 新型周转式轮腿机器人运动分析与步态规划研究[J/OL]. 北京航空航天大学学报, 1-15[2024-03-22].
- [59] 谭跃刚, 董新宇, 胡天麒. 面向 Trot 步态的四足机器人单腿结构及其尺寸优化[J]. 机械传动, 2021, 45(11): 99-108.
- [60] Blaschke F. A new method for the structural decoupling of AC induction machines[C]//IFAC Symposium, Duesseldorf, Germany. 1971, 6: 1.
- [61] Pratt J. Virtual Model Control: An Intuitive Approach for Bipedal Locomotion[J]. The International Journal of Robotics Research, 2001, 20(2): 129-143.
- [62] Xie H, Ahmadi M, Shang J, et al. An intuitive approach for quadruped robot trotting based on virtual model control[J]. Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering, 2015, 229(4): 342-355.
- [63] Waldron K J, Nanua P. Energy comparison between trot, bound, and gallop using a simple model [J]. Journal of Biomechanical Engineering, 1995, 117(4): 466-473.
- [64] Alexander R M. The gaits of bipedal and quadrupedal animals [J]. The International Journal of Robotics

Research, 1984, 3(2): 49-59.

附录 A 运动控制算法程序实现

```
from controller import Robot
from controller import Supervisor
import math
import numpy as np
import copy
from matplotlib import pyplot as plt
robot = Supervisor()# 创建机器人实例
timestep = int(robot.getBasicTimeStep())
robot_motor = [] # LF RF RB LB
for i in range(4):
    robot_motor.append([])
robot_motor[0].append(robot.getMotor("FL_joint1"))# 左前足马达
robot_motor[0].append(robot.getMotor("FL_joint2"))
robot_motor[0].append(robot.getMotor("FL_joint3"))
robot_motor[1].append(robot.getMotor("FR_joint1"))# 右前足马达
robot_motor[1].append(robot.getMotor("FR_joint2"))
robot_motor[1].append(robot.getMotor("FR_joint3"))
robot_motor[2].append(robot.getMotor("BR_joint1"))# 右后足马达
robot_motor[2].append(robot.getMotor("BR_joint2"))
robot_motor[2].append(robot.getMotor("BR_joint3"))
robot_motor[3].append(robot.getMotor("BL_joint1"))# 左后足马达
robot_motor[3].append(robot.getMotor("BL_joint2"))
robot_motor[3].append(robot.getMotor("BL_joint3"))
for leg in range(4):
    for i in range(3):
        robot_motor[leg][i].setAvailableTorque(60.0)
# 设置机器人所有位置传感器
robot_pos_sensor = [] # LF RF RB LB
for i in range(4):
    robot_pos_sensor.append([])
# 左前足位置传感器
```

```

robot_pos_sensor[0].append(robot.getPositionSensor("FL_joint1_sensor"))
robot_pos_sensor[0].append(robot.getPositionSensor("FL_joint2_sensor"))
robot_pos_sensor[0].append(robot.getPositionSensor("FL_joint3_sensor"))
# 右前足位置传感器
robot_pos_sensor[1].append(robot.getPositionSensor("FR_joint1_sensor"))
robot_pos_sensor[1].append(robot.getPositionSensor("FR_joint2_sensor"))
robot_pos_sensor[1].append(robot.getPositionSensor("FR_joint3_sensor"))
# 右后足位置传感器
robot_pos_sensor[2].append(robot.getPositionSensor("BR_joint1_sensor"))
robot_pos_sensor[2].append(robot.getPositionSensor("BR_joint2_sensor"))
robot_pos_sensor[2].append(robot.getPositionSensor("BR_joint3_sensor"))
# 左后足位置传感器
robot_pos_sensor[3].append(robot.getPositionSensor("BL_joint1_sensor"))
robot_pos_sensor[3].append(robot.getPositionSensor("BL_joint2_sensor"))
robot_pos_sensor[3].append(robot.getPositionSensor("BL_joint3_sensor"))
# 触地传感器
robot_touch_sensor = []
robot_touch_sensor.append(robot.createTouchSensor("LF_touch sensor"))
robot_touch_sensor.append(robot.createTouchSensor("RF_touch sensor"))
robot_touch_sensor.append(robot.createTouchSensor("RB_touch sensor"))
robot_touch_sensor.append(robot.createTouchSensor("LB_touch sensor"))
IMU = robot.getInertialUnit("inertial unit")# 惯性传感器
ACC = robot.getAccelerometer("accelerometer")# 加速度传感器
GPS = robot.getGPS("gps")# GPS 位置传感器
KeyBoard = robot.getKeyboard()# 设置键盘控制
def get_IMU_Angle():# 获取姿态角的函数
    data = IMU.getRollPitchYaw()
    return [data[1] * 180.0 / math.pi, data[0] * 180.0 / math.pi, data[2] * 180.0 / math.pi]
def get_GPS_speed():# 获取 GPS 速度的函数
    data = GPS.getSpeed()
    return data
def webot_device_init():# 初始化所有传感器
    KeyBoard.enable(timestep)
    ACC.enable(timestep)

```

```

GPS.enable(timestep)
IMU.enable(timestep)
for leg in range(4):
    robot_motor[leg][0].setPosition(0)
    robot_touch_sensor[leg].enable(timestep)
    for joint in range(3):
        robot_pos_sensor[leg][joint].enable(timestep)
motor_t = list()# 创建一个列表存储各关节扭矩值
motor_t = [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
def set_motor_torque(leg, name, torque): # 设置关节扭矩的函数
    global motor_t
    robot_motor[leg][name].setTorque(torque)
    motor_t[leg][name] = torque
def get_motor_angle(leg, name): # 获取指定关节角度的函数
    angle = robot_pos_sensor[leg][name].getValue()
    return angle * 180.0 / math.pi
def get_all_motor_angle():# 获取所有关节角度的函数
    temp_list = list()
    for leg in range(4):
        temp_list.append([])
        for joint in range(3):
            temp_list[leg].append(get_motor_angle(leg, joint))
    return temp_list
def is_foot_touching(leg): # 判断指定足是否落地的函数
    return robot_touch_sensor[leg].getValue()
def all_is_foot_touching():# 获取所有足是否落地的函数
    temp_list = list()
    for leg in range(4):
        temp_list.append(robot_touch_sensor[leg].getValue())
    return temp_list
class Quadruped_robot_12DOF():# 创建机器人对象
    def __init__(self):
        # 机器人机身参数
        self.L = 0.9

```

```
self.W = 0.3
self.a0 = 0.134
self.L1 = 0.42
self.L2 = 0.42
self.m0 = 1.5
self.m1 = 2
self.m2 = 1.5
self.M = 25 + 4 * 5
# 期望值
self.wz_des = 0.00 # 绕 z 轴的速度 正数 左转
self.Vx_des = 0.00 # 平移速度 正数 左移
self.Vy_des = 0.00 # 前进速度
self.Vy_DES = 0.00
self.H_des = -0.50 # 机体高度
self.Pitch_des = 0.0 / 180.0 * math.pi
self.Roll_des = 0.0 / 180.0 * math.pi
self.Yaw_des = 0.0 / 180.0 * math.pi
self.mode = 0 # 0: 四足稳定站立模式 1: 路面自适应模式 2: 斜坡模式
self.palou = False
self.qidong = 0
# 悬空足系数
self.Kxk = 500
self.Dxk = 25
self.Kyk = 500
self.Dyk = 25
self.Kzk = 500
self.Dzk = 50
self.tk = 0.25 # 悬空时间
self.h_k = 0.30 # 离地高度
self.y_k = 0.125 * self.Vy_des # 跨步距离
self.w_k = 0.125 * self.wz_des # 横跨距离（转弯）
self.x_k = 0.125 * self.Vx_des # 横跨距离（平移）
# 支撑足系数
self.Kx = 800
```

```

self.Dx = 150
self.Ky = 800
self.Dy = 150
self.Kz_f = 800
self.Dz_f = 100
self.Kz_b = 1200
self.Dz_b = 150
self.Kmz = 4
self.Dmz = 0
self.Kmx = -30
self.Dmx = -1
self.Kmy = -30
self.Dmy = -1
self.tz = 0.25 # 支撑时间
self.zh_y = 0 # 支撑足的期望 y 轴位置
# 存储变量
self.Vy = 0 # 当前速度
self.is_touching = [0, 0, 0, 0] # 足端触地状态
self.pos = [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]] # 四足足端当前坐标
for leg in range(4):
    flag = 'L'
    if leg == 1 or leg == 2:
        flag = 'R'
    self.pos[leg] = self.forwardkinematics([0, 1, -2], flag)
self.theta = list()
self.pre_theta = list()
self.motor_v = list()
self.motor_t = list()
self.dot_pos = [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
self.dot_2_pos = [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
self.pre_pos = [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
self.pre_dot_pos = [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
self.Pitch = 0
self.Roll = 0

```

```

self.Yaw = 0
self.pre_eulerAngle = [0, 0, 0]
self.dot_Pitch = 0
self.dot_Roll = 0
self.dot_Yaw = 0
self.dot_Pitch_pre = 0
self.dot_Roll_pre = 0
self.dot_Yaw_pre = 0
self.xiedu_pitch = [0.0, 0.0] # 0
self.xiedu_roll = [0.0, 0.0] # 0
self.time = 0
# 正运动学模型
def forwardkinematics(self, theta, leg): # leg 标识符 左右腿
    a0 = self.a0 if leg == 'L' else -self.a0
    l1 = self.L1 # 大腿长
    l2 = self.L2 # 小腿长
    # 足底位置正解
    s0 = math.sin(theta[0] / 180.0 * math.pi)
    c0 = math.cos(theta[0] / 180.0 * math.pi)
    s1 = math.sin(theta[1] / 180.0 * math.pi)
    c1 = math.cos(theta[1] / 180.0 * math.pi)
    s2 = math.sin(theta[2] / 180.0 * math.pi)
    c2 = math.cos(theta[2] / 180.0 * math.pi)
    s12 = math.sin(theta[1] / 180.0 * math.pi + theta[2] / 180.0 * math.pi)
    c12 = math.cos(theta[1] / 180.0 * math.pi + theta[2] / 180.0 * math.pi)
    x = a0 * c0 + l1 * c1 * s0 + l2 * s0 * c12 # + self.L / 2
    y = -l1 * s1 - l2 * s12 # + self.W / 2
    z = a0 * s0 - l1 * c0 * c1 - l2 * c0 * c12
    return [x, y, z]
# 计算雅可比矩阵
def create_transJ(self, theta, leg): # leg 标识符 左右腿
    a0 = self.a0 if leg == 'L' else -self.a0
    l1 = self.L1 # 大腿长
    l2 = self.L2 # 小腿长

```

```

s0 = math.sin(theta[0] / 180.0 * math.pi)
c0 = math.cos(theta[0] / 180.0 * math.pi)
s1 = math.sin(theta[1] / 180.0 * math.pi)
c1 = math.cos(theta[1] / 180.0 * math.pi)
s2 = math.sin(theta[2] / 180.0 * math.pi)
c2 = math.cos(theta[2] / 180.0 * math.pi)
s12 = math.sin(theta[1] / 180.0 * math.pi + theta[2] / 180.0 * math.pi)
c12 = math.cos(theta[1] / 180.0 * math.pi + theta[2] / 180.0 * math.pi)
transJ = np.zeros((3, 3))
transJ[0, 0] = 11 * c0 * c1 - a0 * s0 + 12 * c0 * c12
transJ[0, 1] = -11 * s0 * s1 - 12 * s0 * s12
transJ[0, 2] = -12 * s0 * s12
transJ[1, 0] = 0
transJ[1, 1] = -11 * c1 - 12 * c12
transJ[1, 2] = -12 * c12
transJ[2, 0] = a0 * c0 + 11 * c1 * s0 + 12 * s0 * c12
transJ[2, 1] = 11 * c0 * s1 + 12 * c0 * s12
transJ[2, 2] = 12 * c0 * s12
return transJ.T

def update_food_touch_sensor(self):# 更新四足触底状态的函数
    self.is_touching = all_is_foot_touching()
def update_theta(self): # 更新各关节角度的函数
    self.theta = get_all_motor_angle()
def get_allmotor_v(self): # 更新各关节速度的函数
    self.motor_v = (np.array(self.theta) - np.array(self.pre_theta)) / timestep * 1000 / 360
    * 60
    self.pre_theta = self.theta
def update_robot_state(self):    # 更新全部状态
    self.key_control()
    self.update_IMU()
    self.update_food_touch_sensor()
    self.update_theta()
    self.get_allmotor_v()
    for leg in range(4):

```

```

        flag = 'L'
        if leg == 1 or leg == 2:
            flag = 'R'
        self.pos[leg] = self.forwardkinematics(self.theta[leg], flag)
    self.update_V_P()
    self.time += timestep * 0.001
def update_IMU(self):    # IMU 获得欧拉角
    eulerAngle = get_IMU_Angle() # Roll Pitch Yaw 当前欧拉角
    self.Roll = eulerAngle[0] # 获得当前 roll
    self.dot_Roll = (eulerAngle[0] - self.pre_eulerAngle[0]) / (0.001 * timestep)
    self.dot_Roll = self.dot_Roll * 0.3 + self.dot_Roll_pre * 0.7
    self.dot_Roll_pre = self.dot_Roll
    self.Pitch = eulerAngle[1] # 获得当前 pitch
    self.dot_Pitch = (eulerAngle[1] - self.pre_eulerAngle[1]) / (0.001 * timestep)
    self.dot_Pitch = self.dot_Pitch * 0.3 + self.dot_Pitch_pre * 0.7 # 低通滤波
    self.dot_Pitch_pre = self.dot_Pitch
    self.Yaw = eulerAngle[2] # 获得当前 yaw
    self.dot_Yaw = (eulerAngle[2] - self.pre_eulerAngle[2]) / (0.001 * timestep)
    self.dot_Yaw = self.dot_Yaw * 0.3 + self.dot_Yaw_pre * 0.7 # 低通滤波
    self.dot_Yaw_pre = self.dot_Yaw
    self.pre_eulerAngle = copy.deepcopy(eulerAngle)
def update_V_P(self):    # 速度，加速度更新
    self.Vy = get_GPS_speed()
    dot_pos = (((np.array(self.pos) - np.array(self.pre_pos))
                / (0.001 * timestep)) * 1.0 + np.array(self.pre_dot_pos) * 0).tolist()
    self.pre_pos = copy.deepcopy(self.pos)
    dot_2_pos = ((np.array(dot_pos) - np.array(self.pre_dot_pos)) / (0.001 *
timestep)).tolist()
    self.pre_dot_pos = copy.deepcopy(dot_pos)
    self.dot_pos = copy.deepcopy(dot_pos)
    self.dot_2_pos = copy.deepcopy(dot_2_pos)
    pass
def cmp_xiedu_pitch(self, zh_f, zh_b):    # 获取斜坡斜度
    a1 = -(self.Pitch - self.Pitch_des) / 180 * np.pi

```

```

    p_f = self.pos[zh_f][1:]
    p_f[0] += self.L / 2
    p_b = self.pos[zh_b][1:]
    p_b[0] -= self.L / 2
    x = np.array(p_b) - np.array(p_f)
    a2 = np.arctan(x[1] / x[0])
    return a1 + a2

def get_xiedu_pitch(self):
    p1 = self.cmp_xiedu_pitch(0, 3)
    p2 = self.cmp_xiedu_pitch(1, 2)
    # self.xiedu_pitch = (p1 + p2) / 2
    self.xiedu_pitch = [p1, p2]

def cmp_xiedu_roll(self, zh_l, zh_r):    # 获取斜坡斜度
    a1 = -(self.Roll - self.Roll_des) / 180 * np.pi
    p_l = []
    p_l.append(self.pos[zh_l][0])
    p_l.append(self.pos[zh_l][2])
    p_l[0] += (self.W / 2 + self.a0)
    p_r = []
    p_r.append(self.pos[zh_r][0])
    p_r.append(self.pos[zh_r][2])
    p_r[0] -= (self.W / 2 + self.a0)
    x = np.array(p_l) - np.array(p_r)
    a2 = -np.arctan(x[1] / x[0])
    return a1 + a2

def get_xiedu_roll(self):
    r1 = self.cmp_xiedu_roll(0, 1)
    r2 = self.cmp_xiedu_roll(3, 2)
    # self.xiedu_roll = (r1 + r2) / 2
    self.xiedu_roll = [r1, r2]

def get_dy(self, H):
    a = self.Pitch / 180 * np.pi
    return np.tan(a) * H

def zh(self, zh_F, zh_B):    # 支撑相

```

```

if zh_F == 0:
    zh_F_RL = 'L'
    zh_B_RL = 'R'
    zh_F_flag = 1
    zh_B_flag = -1
else:
    zh_F_RL = 'R'
    zh_B_RL = 'L'
    zh_F_flag = -1
    zh_B_flag = 1
a = self.Yaw_des - self.Yaw
if a < -20:
    a = -20
elif a > 20:
    a = 20
Mz = self.Kmz * a + self.Dmz * (0 - self.dot_Yaw)
fx = Mz / (self.W * 0.5)
Mx = self.Kmx * (self.Pitch_des - self.Pitch) + self.Dmx * (0 - self.dot_Pitch)
fz = Mx / (self.L * 0.5)
My = self.Kmy * (self.Roll_des - self.Roll) + self.Dmy * (0 - self.dot_Roll)
fyz = My / (self.W * 0.5 + self.a0)
a_pitch = self.xiedu_pitch# 斜坡上前后腿的期望高度
if zh_F == 0:
    d_h11 = self.L / 2 * np.tan(a_pitch[0])
    d_h12 = self.L / 2 * np.tan(a_pitch[1])
else:
    d_h11 = self.L / 2 * np.tan(a_pitch[1])
    d_h12 = self.L / 2 * np.tan(a_pitch[0])
a_roll = self.xiedu_roll
d_h21 = (self.W / 2 + self.a0) * np.tan(a_roll[0])
d_h22 = (self.W / 2 + self.a0) * np.tan(a_roll[1])
Fz_F = self.Kz_f * (self.H_des + d_h11 - zh_F_flag * d_h21 - self.pos[zh_F][2]) +
self.Dz_f * (0 - self.dot_pos[zh_F][2])
Fz_B = self.Kz_b * (self.H_des - d_h12 - zh_B_flag * d_h22 - self.pos[zh_B][2]) +

```

```

self.Dz_f * (0 - self.dot_pos[zh_B][2])
Fz_F = 2 * Fz_F - self.M * 10 * 0.5 - fz + fyz * zh_F_flag
Fz_B = 2 * Fz_B - self.M * 10 * 0.5 + fz + fyz * zh_B_flag
Fy_F = self.Dy * (-self.Vy_des - self.dot_pos[zh_F][1]) + np.sin(self.Pitch / 180 *
np.pi) * self.M * 10 / 2
Fy_B = self.Dy * (-self.Vy_des - self.dot_pos[zh_B][1]) + np.sin(self.Pitch / 180 *
np.pi) * self.M * 10 / 2
Fx_F = self.Dx * ((-self.Vx_des) - self.dot_pos[zh_F][0]) - fx
Fx_B = self.Dx * ((-self.Vx_des) - self.dot_pos[zh_B][0]) + fx
T_F = self.create_transJ(self.theta[zh_F], zh_F_RL) @ np.array([[Fx_F], [Fy_F],
[Fz_F]])
T_B = self.create_transJ(self.theta[zh_B], zh_B_RL) @ np.array([[Fx_B], [Fy_B],
[Fz_B]])
set_motor_torque(zh_F, 0, float(T_F[0, 0]))
set_motor_torque(zh_F, 1, float(T_F[1, 0]))
set_motor_torque(zh_F, 2, float(T_F[2, 0]))
set_motor_torque(zh_B, 0, float(T_B[0, 0]))
set_motor_torque(zh_B, 1, float(T_B[1, 0]))
set_motor_torque(zh_B, 2, float(T_B[2, 0]))
def bai(self, bai_F, bai_B, t):    # 摆动相
    if bai_F == 0:
        bai_F_RL = 'L'
        bai_B_RL = 'R'
        bai_F_flag = 1
        bai_B_flag = -1
    else:
        bai_F_RL = 'R'
        bai_B_RL = 'L'
        bai_F_flag = -1
        bai_B_flag = 1
    # 斜坡补偿
    a_pitch = sum(self.xiedu_pitch)/2
    d_h = self.L / 2 * np.tan(a_pitch)
    a_roll = sum(self.xiedu_roll)/2

```

```

d_h2 = (self.W / 2 + self.a0) * np.tan(a_roll)
dy1 = self.get_dy(1.2 * (self.H_des + d_h - bai_F_flag * d_h2))
dy2 = self.get_dy(1.2 * (self.H_des + d_h - bai_B_flag * d_h2))
Fz = 1.2 * (self.H_des + d_h - bai_F_flag * d_h2) + self.h_k * np.sin(t * np.pi / self.tk)
Bz = 1.2 * (self.H_des - d_h - bai_B_flag * d_h2) + self.h_k * np.sin(t * np.pi / self.tk)
if self.palou:
    if t < self.tk/2:
        Fy = -8 * t * 0.125 * self.Vy_des / self.tk - 0.125 * self.Vy_des - dy1
        By = -8 * t * 0.125 * self.Vy_des / self.tk - 0.125 * self.Vy_des - dy2
    else:
        Fy = 6*2 * (t-self.tk/2) * 0.125 * self.Vy_des / self.tk - 5*0.125 *
        self.Vy_des - dy1
        By = 6*2 * (t-self.tk/2) * 0.125 * self.Vy_des / self.tk - 5*0.125 *
        self.Vy_des - dy2
else:
    Fy = 2 * t * 0.125 * self.Vy_des / self.tk - 0.125 * self.Vy_des - dy1
    By = 2 * t * 0.125 * self.Vy_des / self.tk - 0.125 * self.Vy_des - dy2
a = self.Yaw_des - self.Yaw
if a > 3:
    a = 3
if a < -3:
    a = -3
Fx = (bai_F_flag * self.a0 + t * 0.1 * a / self.tk - 0.05 * a) + t * 0.125 * self.Vx_des /
self.tk
Bx = (bai_B_flag * self.a0 - t * 0.1 * a / self.tk + 0.05 * a) + t * 0.125 * self.Vx_des
/ self.tk
if t > self.tk/2 and is_foot_touching(bai_F):
    Fz = self.pos[bai_F][2]
    Fy = self.pos[bai_F][1]
    Fx = self.pos[bai_F][0]
if t > self.tk/2 and is_foot_touching(bai_B):
    Bz = self.pos[bai_B][2]
    By = self.pos[bai_B][1]
    Bx = self.pos[bai_B][0]

```

```

Fz_F = self.Kzk * (Fz - self.pos[bai_F][2]) + self.Dzk * (0 - self.dot_pos[bai_F][2])
Fz_B = self.Kzk * (Bz - self.pos[bai_B][2]) + self.Dzk * (0 - self.dot_pos[bai_B][2])
Fy_F = self.Kyk * (Fy - self.pos[bai_F][1]) + self.Dyk * (0.0 - self.dot_pos[bai_F][1])
Fy_B = self.Kyk * (By - self.pos[bai_B][1]) + self.Dyk * (0.0 - self.dot_pos[bai_B][1])
Fx_F = self.Kxk * (Fx - self.pos[bai_F][0]) + self.Dxk * (0.0 - self.dot_pos[bai_F][0])
Fx_B = self.Kxk * (Bx - self.pos[bai_B][0]) + self.Dxk * (0.0 - self.dot_pos[bai_B][0])
T_F = self.create_transJ(self.theta[bai_F], bai_F_RL) @ np.array([[Fx_F], [Fy_F], [Fz_F]])
T_B = self.create_transJ(self.theta[bai_B], bai_B_RL) @ np.array([[Fx_B], [Fy_B], [Fz_B]])
set_motor_torque(bai_F, 0, float(T_F[0, 0]))
set_motor_torque(bai_F, 1, float(T_F[1, 0]))
set_motor_torque(bai_F, 2, float(T_F[2, 0]))
set_motor_torque(bai_B, 0, float(T_B[0, 0]))
set_motor_torque(bai_B, 1, float(T_B[1, 0]))
set_motor_torque(bai_B, 2, float(T_B[2, 0]))
def stand(self): # 四足站立状态控制
    if not all_is_foot_touching() == [0, 0, 0, 0]:
        a = self.Yaw_des - self.Yaw
        if a < -20:
            a = -20
        elif a > 20:
            a = 20
        Mz = self.Kmz * a + self.Dmz * (0 - self.dot_Yaw)
        fx = Mz / (self.W * 0.5)
        Mx = self.Kmx * (self.Pitch_des - self.Pitch) + self.Dmx * (0 - self.dot_Pitch)
        My = self.Kmy * (self.Roll_des - self.Roll) + self.Dmy * (0 - self.dot_Roll)
        fz = Mx / (self.L * 0.5)
        fzy = My / (self.W * 0.5 + self.a0)
        d_h = 0
        d_h2 = 0
        Fz_left_F = self.Kz_f * (self.H_des + d_h - d_h2 - self.pos[0][2]) + self.Dz_f *

```

```

( 0 - self.dot_pos[0][2])
Fz_right_F = self.Kz_f * (self.H_des + d_h + d_h2 - self.pos[1][2]) + self.Dz_f
* ( 0 - self.dot_pos[1][2])
Fz_left_B = self.Kz_b * (self.H_des - d_h - d_h2 - self.pos[3][2]) + self.Dz_b *
( 0 - self.dot_pos[3][2])
Fz_right_B = self.Kz_b * (self.H_des - d_h + d_h2 - self.pos[2][2]) + self.Dz_b
* ( 0 - self.dot_pos[2][2])
Fz_left_F = Fz_left_F - self.M * 10 / 4 - fz + fzy
Fz_right_F = Fz_right_F - self.M * 10 / 4 - fz - fzy
Fz_left_B = Fz_left_B - self.M * 10 / 4 + fz + fzy
Fz_right_B = Fz_right_B - self.M * 10 / 4 + fz - fzy # / 2
Fy_left_f = self.Ky * (0.0 - self.pos[0][1]) + self.Dy * (0.0 - self.dot_pos[0][1])
Fy_right_f = self.Ky * (0.0 - self.pos[1][1]) + self.Dy * (0.0 - self.dot_pos[1][1])
Fy_left_b = self.Ky * (0.0 - self.pos[3][1]) + self.Dy * (0.0 - self.dot_pos[3][1])
Fy_right_b = self.Ky * (0.0 - self.pos[2][1]) + self.Dy * (0.0 - self.dot_pos[2][1])
Fx_left_f = self.Kx * (self.a0 - self.pos[0][0]) + self.Dx * (0.0 -
self.dot_pos[0][0]) - fx
Fx_right_f = self.Kx * (-self.a0 - self.pos[1][0]) + self.Dx * (0.0 -
self.dot_pos[1][0]) - fx
Fx_left_b = self.Kx * (self.a0 - self.pos[3][0]) + self.Dx * (0.0 -
self.dot_pos[3][0]) + fx
Fx_right_b = self.Kx * (-self.a0 - self.pos[2][0]) + self.Dx * (0.0 -
self.dot_pos[2][0]) + fx
T_left_F = self.create_transJ(self.theta[0], 'L') @ np.array([[Fx_left_f],
[Fy_left_f], [Fz_left_F]])
T_left_B = self.create_transJ(self.theta[3], 'L') @ np.array([[Fx_left_b],
[Fy_left_b], [Fz_left_B]])
T_right_F = self.create_transJ(self.theta[1], 'R') @ np.array([[Fx_right_f],
[Fy_right_f], [Fz_right_F]])
T_right_B = self.create_transJ(self.theta[2], 'R') @ np.array([[Fx_right_b],
[Fy_right_b], [Fz_right_B]])
set_motor_torque(0, 0, float(T_left_F[0, 0]))
set_motor_torque(0, 1, float(T_left_F[1, 0]))
set_motor_torque(0, 2, float(T_left_F[2, 0]))

```

```

set_motor_torque(1, 0, float(T_right_F[0, 0]))
set_motor_torque(1, 1, float(T_right_F[1, 0]))
set_motor_torque(1, 2, float(T_right_F[2, 0]))
set_motor_torque(2, 0, float(T_right_B[0, 0]))
set_motor_torque(2, 1, float(T_right_B[1, 0]))
set_motor_torque(2, 2, float(T_right_B[2, 0]))
set_motor_torque(3, 0, float(T_left_B[0, 0]))
set_motor_torque(3, 1, float(T_left_B[1, 0]))
set_motor_torque(3, 2, float(T_left_B[2, 0]))
elif all_is_foot_touching() == [0, 0, 0, 0]:
    Fz_left_F = self.Kzk * (self.H_des - self.pos[0][2]) + self.Dzk * (
        0 - self.dot_pos[0][2])
    Fz_right_F = self.Kzk * (self.H_des - self.pos[1][2]) + self.Dzk * (
        0 - self.dot_pos[1][2])
    Fz_left_B = self.Kzk * (self.H_des - self.pos[3][2]) + self.Dzk * (
        0 - self.dot_pos[3][2])
    Fz_right_B = self.Kzk * (self.H_des - self.pos[2][2]) + self.Dzk * (
        0 - self.dot_pos[2][2])
    Fz_left_F = Fz_left_F
    Fz_right_F = Fz_right_F
    Fz_left_B = Fz_left_B
    Fz_right_B = Fz_right_B # / 2
    Fy_left_f = self.Kyk * (0.0 - self.pos[0][1]) + self.Dyk * (0.0 - self.dot_pos[0][1])
    Fy_right_f = self.Kyk * (0.0 - self.pos[1][1]) + self.Dyk * (0.0 -
        self.dot_pos[1][1])
    Fy_left_b = self.Kyk * (0.0 - self.pos[3][1]) + self.Dyk * (0.0 -
        self.dot_pos[3][1])
    Fy_right_b = self.Kyk * (0.0 - self.pos[2][1]) + self.Dyk * (0.0 -
        self.dot_pos[2][1])
    Fx_left_f = self.Kxk * (self.a0 - self.pos[0][0]) + self.Dxk * (0.0 -
        self.dot_pos[0][0])
    Fx_right_f = self.Kxk * (-self.a0 - self.pos[1][0]) + self.Dxk * (0.0 -
        self.dot_pos[1][0])
    Fx_left_b = self.Kxk * (self.a0 - self.pos[3][0]) + self.Dxk * (0.0 -

```

```

self.dot_pos[3][0])
Fx_right_b = self.Kxk * (-self.a0 - self.pos[2][0]) + self.Dxk * (0.0 -
self.dot_pos[2][0])
T_left_F = self.create_transJ(self.theta[0], 'L') @ np.array([[Fx_left_f],
[Fy_left_f], [Fz_left_F]])
T_left_B = self.create_transJ(self.theta[3], 'L') @ np.array([[Fx_left_b],
[Fy_left_b], [Fz_left_B]])
T_right_F = self.create_transJ(self.theta[1], 'R') @ np.array([[Fx_right_f],
[Fy_right_f], [Fz_right_F]])
T_right_B = self.create_transJ(self.theta[2], 'R') @ np.array([[Fx_right_b],
[Fy_right_b], [Fz_right_B]])
set_motor_torque(0, 0, float(T_left_F[0, 0]))
set_motor_torque(0, 1, float(T_left_F[1, 0]))
set_motor_torque(0, 2, float(T_left_F[2, 0]))
set_motor_torque(1, 0, float(T_right_F[0, 0]))
set_motor_torque(1, 1, float(T_right_F[1, 0]))
set_motor_torque(1, 2, float(T_right_F[2, 0]))
set_motor_torque(2, 0, float(T_right_B[0, 0]))
set_motor_torque(2, 1, float(T_right_B[1, 0]))
set_motor_torque(2, 2, float(T_right_B[2, 0]))
set_motor_torque(3, 0, float(T_left_B[0, 0]))
set_motor_torque(3, 1, float(T_left_B[1, 0]))
set_motor_torque(3, 2, float(T_left_B[2, 0]))
def run(self):# 启动函数
    flag = 1
    t = 0.00
    time = 0.0
    velocity = []
    roll = []
    pitch = []
    yaw = []
    y = []
    t0 = []
    t1 = []

```

```

t2 = []
v0 = []
v1 = []
v2 = []
self.pre_theta = [[0, 0, 0],[0, 0, 0],[0, 0, 0],[0, 0, 0]]
self.motor_t = [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
while robot.step(timestep) != -1:
    time += timestep
    if self.mode == 1 or self.mode == 2:
        self.qidong = 1
    else:
        self.qidong = 0
        self.Vy_des = 0
    self.Vy_des += timestep / 1000 * 0.5 * self.qidong
    if self.Vy_des > self.Vy_DES:
        self.Vy_des = self.Vy_DES
    if time >= 5000 and time % 10 == 0:# 获取机器人各传感器数据输出
        velocity.append(GPS.getSpeed())
        roll.append(self.Roll)
        pitch.append(-self.Pitch)
        yaw.append(self.Yaw)
        y.append(self.Vy_des)
        global motor_t
        t0.append(motor_t[0][0])
        t1.append(motor_t[0][1])
        t2.append(motor_t[0][2])
        v0.append(self.motor_v[0][0])
        v1.append(self.motor_v[0][1])
        v2.append(self.motor_v[0][2])
    if len(velocity) == 4000:
        x = np.linspace(0, 40, 4000)
        plt.rcParams['font.family'] = ['STFangsong']
        plt.figure(1)
        plt.ylabel('髋关节扭矩/(Nm)' # y 轴标签

```

```
plt.xlabel('转速/(r/min)') # x 轴标签
plt.scatter(v0, t0, color='red')
plt.figure(2)
plt.ylabel('大腿关节扭矩/(Nm)') # y 轴标签
plt.xlabel('转速/(r/min)') # x 轴标签
plt.scatter(v1, t1, color='red')
plt.figure(3)
plt.ylabel('小腿关节扭矩/(Nm)') # y 轴标签
plt.xlabel('转速/(r/min)') # x 轴标签
plt.scatter(v2, t2, color='red')
plt.figure(4)
plt.subplot(3, 1, 1)
plt.plot(x, t0)
plt.ylabel('髋关节扭矩/(Nm)') # y 轴标签
plt.subplot(3, 1, 2)
plt.plot(x, t1)
plt.ylabel('大腿关节扭矩/(Nm)') # y 轴标签
plt.subplot(3, 1, 3)
plt.plot(x, t2)
plt.ylabel('小腿关节扭矩/(Nm)') # y 轴标签
plt.xlabel('时间/s')
plt.figure(5)
R = plt.subplot(3, 1, 1)
R.set_ylabel('横滚/度') # x 轴标签
R.plot(x, roll, color='b')
P = plt.subplot(3, 1, 2)
P.set_ylabel('俯仰/度') # y 轴标签
P.plot(x, pitch, color='b')
Y = plt.subplot(3, 1, 3)
Y.set_ylabel('偏摆/度') # x 轴标签
Y.set_xlabel('时间/s') # y 轴标签
Y.plot(x, yaw, color='b')
plt.show()
break
```

```

for leg in range(4):
    robot_motor[leg][0].setPosition(0)
self.update_robot_state()
if self.mode == 0:
    self.get_xiedu_roll()
    self.get_xiedu_pitch()
    self.stand()
# trot 步态状态控制
if self.mode == 1 or self.mode == 2:
    if self.mode == 2:
        self.Pitch_des = -sum(self.xiedu_pitch) / 2 / np.pi * 180
    else:
        self.Pitch_des = 0
    if flag:
        zh_F = 0
        zh_B = 2
        bai_F = 1
        bai_B = 3
    else:
        zh_F = 1
        zh_B = 3
        bai_F = 0
        bai_B = 2
    self.zh(zh_F, zh_B)
    self.bai(bai_F, bai_B, t / 1000)
    t += timestep
    if t > 1000 * self.tk / 2 and is_foot_touching(bai_F) and
    is_foot_touching(bai_B):
        flag = not flag
        self.get_xiedu_roll()
        self.get_xiedu_pitch()
        t = 0
    print(
        'Vy_D = {}, Vy = {}, yaw = {}, Vx = {}, H =

```

```
        {}.format(self.Vy_DES, self.Vy_des, self.Yaw_des, self.Vx_des,
                  self.H_des))
    robot.simulationSetMode(0)
def key_control(self):
    key = Keyboard.getKey()
    if key == ord('W'):
        self.Vy_DES += 0.002
        if self.Vy_DES > 2:
            self.Vy_DES = 2
    elif key == ord('S'):
        self.Vy_DES -= 0.002
        if self.Vy_DES < -1:
            self.Vy_DES = -1
    elif key == ord('A'):
        self.Vx_des += 0.001
        if self.Vx_des > 0.6:
            self.Vx_des = 0.6
    elif key == ord('D'):
        self.Vx_des -= 0.001
        if self.Vx_des < -0.6:
            self.Vx_des = -0.6
    elif key == ord('Q'):
        self.Yaw_des += 0.05
        if self.Yaw_des > 180:
            self.Yaw_des -= 360
    elif key == ord('E'):
        self.Yaw_des -= 0.05
        if self.Yaw_des < -180:
            self.Yaw_des += 360
    elif key == ord('Y'):
        self.H_des -= 0.0005
        if self.H_des < -0.8:
            self.H_des = -0.8
    elif key == ord('H'):
```

```
        self.H_des += 0.0005
    if self.H_des > -0.1:
        self.H_des = -0.1
    elif key == ord('I'):
        self.Pitch_des += 0.01
    elif key == ord('K'):
        self.Pitch_des -= 0.01
    elif key == ord('J'):
        self.Roll_des -= 0.01
    elif key == ord('L'):
        self.Roll_des += 0.01
    elif key == ord('0'):
        self.mode = 0
    elif key == ord('1'):
        self.mode = 1
        self.Pitch = 0.0
    elif key == ord('2'):
        self.mode = 2
    elif key == ord('8'):
        self.h_k += 0.001
    elif key == ord('5'):
        self.h_k -= 0.001
    elif key == ord('P'):
        self.palou = not self.palou
    elif key == ord('R'):
        self.Vx_des = 0
        self.Vy_DES = 0.00
        self.H_des = -0.5
        self.Pitch_des = 0.0
        self.Roll_des = 0.0
if __name__ == '__main__':
    q = Quadruped_robot_8DOF()
    webot_device_init()
    q.run()
```